

بسم الله الرحمن الرحيم



Sudan University of Science and Technology

College of Computer Science and Information Technology

Online Payment Integration Platform

Prepared By:

- Abubakr Alzubair Alsaid
- Husam Mohamed Osman Ibrahim Eltahir
- Madani Saifaldwla Madani Bakheet
- Marwa Awad Alrashid Ali

Supervisor: Dr. Sara Ahmed

Date: May, 2026

الآية

قال تعالى:

{ دَعُوهُمْ فِيهَا سُبْحَنَكَ اللَّهُمَّ وَتَحِيَّتُهُمْ فِيهَا سَلَامٌ ۖ وَآخِرُ دَعْوَاهُمْ أَنِ الْحَمْدُ لِلَّهِ رَبِّ
الْعَالَمِينَ }

سورة يونس - الآية 10

الإهداء

نُهدي هذا العمل المتواضع، ثمرةً جهدنا وخلصاً مسيرتنا الدراسية...
إلى والدينا الأحباء، الذين غمرونا بالحب والعطاء، وكانوا الملاذ الذي نعود إليه كلما أرهقنا التعب.
لا تكفيكم كلمات، ولا يوفِّيكُم إنجاز مهما عَظُم.
إلى أساتذتنا الأفاضل، الذين منحونا من علمهم وخبرتهم، ووجَّهونا بصبر وإخلاص حتى أتممنا
هذا المشوار.
إلى عائلاتنا وأهلنا، الذين تحمَّلوا غيابنا وسهرنا، وآمنوا بنا حين احتجنا إلى من يؤمن.
إلى زملائنا الخريجين وسابقينا بالفضل والخبرة، الذين كانوا لنا نِعم العون والقُدوة، ولم يخلوا
علينا بنصحٍ أو توجيهٍ أنار لنا الدرب.
إلى كل من مدَّ لنا يد العون، بكلمة تشجيع، أو نصيحة سديدة، أو ابتسامة في وقتها.
نُهدي هذا العمل بكل ما فيه من تعب وأمل وإصرار، فهو لم يكن يوماً إنجاز فرد، بل كان ثمرة
حبٍّ وعطاءٍ أحاط بنا من كل اتجاه.

"وَقُلْ رَبِّ زِدْنِي عِلْمًا"

شكر وتقدير

الحمد لله والشكر له أولاً وآخراً، العلي القدير الذي وهبنا القوة والصبر لإتمام هذا العمل المتواضع. نتقدم بخالص الشكر والامتنان وعميق التقدير إلى مشرفة المشروع الدكتورة سارة أحمد، على توجيهاتها القيمة، ودعمها المستمر، وملاحظاتها السديدة طوال مراحل هذا المشروع. كما نتوجه ببالغ التقدير والاعتزاز إلى عميدة الكلية الدكتورة وفاء فيصل، ورئيسة قسم هندسة البرمجيات الدكتورة هند الأمين، على قيادتهما الحكيمة وحرصهما الدائم على توفير بيئة أكاديمية متميزة.

ويمتد شكرنا إلى جميع الأساتذة وأعضاء الهيئة التدريسية بـ كلية علوم الحاسوب وتقانة المعلومات، الذين سلّحونا بالمعرفة والعلم اللّذين جعلنا من هذا المشروع ممكناً. ولا يفوتنا أن نعرب عن تقديرنا لزملائنا وسابقينا في هذا المجال، الذين كانوا لنا خير قدوة، وشاركونا خبراتهم القيمة، وقدموا لنا النصح والدعم بكل سخاء كلما احتجنا إليه. وأخيراً، نتوجه بامتناننا الأبدي إلى عائلاتنا وأصدقائنا على حبهم غير المشروط، وصبرهم، وتشجيعهم المتواصل طوال هذه الرحلة.

"من لا يشكر الناس لا يشكر الله" — رسول الله محمد ﷺ.

Abstract

Online payment integration in Sudan is fragmented and inefficient. Developers must negotiate individually with each payment provider, navigate inconsistent technical interfaces, and often rely on manual workarounds such as receipt verification. No unified, developer-facing payment API currently exists in the Sudanese market.

This project presents PhoenixPay, a unified online payment integration platform addressing this gap. It exposes a single RESTful API through which merchants can create invoices, receive HMAC-signed webhook notifications, track balances, and initiate payouts and refunds — without interacting with banking infrastructure directly. The platform implements production-grade security including four authentication layers, idempotent payment confirmation, ACID-compliant transactions, and exponential-backoff webhook delivery. To demonstrate it end-to-end, three supporting components were built or deployed: Apache Fineract to simulate core banking, a custom Bank Channel web application simulating a bank's payment interface, and TaskMaster, a demonstration merchant application exercising the full integration lifecycle.

The system was built with Node.js, Express, Vue 3, and MariaDB, deployed as a Docker Compose stack behind an Nginx reverse proxy. Manual testing across 54 functional, integration, security, and negative test cases achieved a 100% pass rate, demonstrating that a unified payment platform for Sudan can be built securely using open-source tools and the same design patterns proven by global platforms such as Stripe and Flutterwave.

الملخص

لا يزال دمج المدفوعات الإلكترونية في السودان عملية مجزأة وغير فعّالة. إذ يضطر المطورون إلى التفاوض بشكل منفرد مع كل مزود خدمة دفع، والتعامل مع واجهات تقنية غير موحّدة، واللجوء في أحيان كثيرة إلى حلول يدوية كالتحقق من إيصالات الدفع. ولا توجد حالياً في السوق السوداني واجهة برمجية موحّدة للمدفوعات موجّهة للمطورين.

يقدم هذا المشروع PhoenixPay، وهي منصة موحّدة لدمج المدفوعات الإلكترونية تعالج هذه الفجوة. تتيح المنصة واجهة برمجية واحدة من نوع RESTful تمكّن التجار من إنشاء الفواتير، واستقبال إشعارات Webhook موقّعة بـ HMAC، وتتبع الأرصدة، وتنفيذ عمليات الصرف واسترداد المدفوعات — دون الحاجة للتعامل المباشر مع البنية التحتية المصرفية. وتطبّق المنصة معايير أمنية على مستوى الإنتاج، تشمل أربع طبقات للمصادقة، وتأكيد المدفوعات بشكل يؤكد عدم التكرار، ومعاملات مالية متوافقة مع خصائص ACID، وآلية لتسليم إشعارات Webhook بإعادة محاولة متزايدة أُسياً. ولعرض المنصة في سياق متكامل، طُوّرت ثلاثة مكونات داعمة: Apache Fineract لمحاكاة العمليات المصرفية الأساسية، وقناة دفع تحاكي واجهة الدفع للعملاء في البنك، وتطبيق TaskMaster التجريبي للتاجر الذي يستعرض دورة الدمج الكاملة.

طُوّر النظام باستخدام Node.js و Express و Vue 3 و MariaDB، ونُشر عبر حزمة Docker Compose خلف خادم Nginx العكسي. وقد اجتاز النظام جميع الاختبارات اليدوية المنهجية التي شملت 54 حالة اختبار وظيفية وتكاملية وأمنية وسلبية بنسبة نجاح بلغت 100%، مما يُثبت إمكانية بناء منصة موحّدة للمدفوعات في السودان بشكل آمن باستخدام أدوات مفتوحة المصدر، وتطبيق ذات أنماط التصميم التي أثبتت جدواها في منصات عالمية مثل Stripe و Flutterwave.

List of Abbreviations

Abbreviation	Full Form
ACID	Atomicity, Consistency, Isolation, Durability
AML	Anti-Money Laundering
API	Application Programming Interface
B2B	Business to Business
B2C	Business to Customer
C2B	Customer to Business
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IEEE	Institute of Electrical and Electronics Engineers
iPIN	Internet Personal Identification Number
JDBC	Java Database Connectivity
JSON	JavaScript Object Notation
JWT	JSON Web Token
KYC	Know Your Customer
LTS	Long-Term Support
MENA	Middle East and North Africa
OAuth	Open Authorization
ORM	Object-Relational Mapping
OTP	One-Time Password
OWASP	Open Web Application Security Project
RBAC	Role-Based Access Control
REST	Representational State Transfer
RTL	Right-to-Left
SaaS	Software as a Service
SDK	Software Development Kit
SDG	Sudanese Pound (جنيه سوداني)
SHA-256	Secure Hash Algorithm 256-bit
SPA	Single-Page Application
SQL	Structured Query Language

SSL	Secure Sockets Layer
TLS	Transport Layer Security
TTL	Time to Live
UI	User Interface
URL	Uniform Resource Locator
UTC	Coordinated Universal Time

Table of Contents

الإهداء.....	II
شكر وتقدير.....	III
Abstract.....	IV
الملخص.....	V
List of Abbreviations.....	VI
Table of Contents.....	VIII
Chapter 1: Introduction.....	1
1.1 Background.....	1
1.2 Problem Statement.....	3
1.3 Aims & Objectives.....	4
1.4 Scope.....	6
1.5 Motivation.....	9
1.6 Research Methodology.....	10
1.7 Structure of Research.....	12
Chapter 2: Literature Review.....	14
2.1 Introduction.....	14
2.2 Background.....	15
2.3 Case Studies.....	16
2.3.1 Stripe.....	16
Pros:.....	17
Cons:.....	18
Observed Limitations and Addressed Gaps:.....	18
2.3.2 PayPal.....	19
Pros:.....	19
Cons:.....	20
Observed Limitations and Addressed Gaps:.....	21
2.3.3 Flutterwave.....	21
Pros:.....	22
Cons:.....	23
Observed Limitations and Addressed Gaps:.....	23
2.3.4 M-Pesa (Daraja).....	24
Pros:.....	25
Cons:.....	25

Observed Limitations and Addressed Gaps:.....	26
2.3.5 Moyasar.....	26
Pros:.....	27
Cons:.....	28
Observed Limitations and Addressed Gaps:.....	29
2.3.6 CashiPay.....	29
Pros:.....	30
Cons:.....	30
Observed Limitations and Addressed Gaps:.....	31
2.3.7 SyberPay.....	31
Pros:.....	32
Cons:.....	33
Observed Limitations and Addressed Gaps:.....	33
2.3.8 Fragmented Payment Integration in Sudan: Direct Integration and Manual Workarounds.....	34
Challenges Faced by Developers:.....	35
Observed Limitations and Addressed Gaps:.....	36
2.3.9 Demirgüç-Kunt, Klapper, Singer & Ansar (2022): The Global Findex Database 2021.....	37
Relevance to PhoenixPay:.....	37
2.3.10 ECDPM (2024): Gaps in Interoperability as Barriers to Inclusive Digital Payments in Africa.....	38
Relevance to PhoenixPay:.....	39
2.3.11 Ahmad, Green & Jiang (2020): Mobile Money, Financial Inclusion and Development.....	39
Relevance to PhoenixPay:.....	40
2.4 Conclusion.....	41
2.5 Summary of Literature Review.....	42
Payment Platforms and Practices.....	42
Academic Studies.....	44
Chapter 3: System Analysis & Design.....	45
3.1 Introduction.....	45
3.2 Requirements Analysis.....	46
3.2.1 Functional Requirements.....	46
3.2.2 Non-Functional Requirements.....	48
3.3 System Overview: The Three-Entity Model.....	49

3.4 Component 1: Core Banking System — Apache Fineract.....	51
3.4.1 Overview.....	51
3.4.2 Features and Functions.....	52
3.4.3 Users.....	53
3.4.4 Non-functional Requirements and Constraints.....	53
3.4.5 Design.....	54
Key Fineract API calls used in this project:.....	55
3.5 Component 2: Bank Channel.....	55
3.5.1 Overview.....	55
3.5.2 Features and Functions.....	56
3.5.3 Users.....	57
3.5.4 Non-functional Requirements and Constraints.....	57
3.5.5 Design.....	58
PhoenixPay-Facing API (routes/phoenix.js):.....	60
3.5.6 Use Cases.....	61
3.6 Component 3: PhoenixPay System.....	64
3.6.1 Overview.....	64
3.6.2 Features and Functions.....	64
PhoenixPay Backend:.....	64
Merchant Dashboard:.....	65
3.6.3 Users.....	66
3.6.4 Non-functional Requirements and Constraints.....	66
3.6.5 Design.....	67
Authentication Design:.....	67
Key API Endpoints:.....	71
3.6.6 Use Cases.....	71
3.7 Component 4: TaskMaster — Demo Merchant Application.....	76
3.7.1 Overview.....	76
3.7.2 Features and Functions.....	76
Backend (Node.js / Express / MariaDB):.....	76
Frontend (Vue 3 / Pinia / Tailwind CSS):.....	77
3.7.3 Users.....	77
3.7.4 Non-functional Requirements and Constraints.....	78
3.7.5 Design.....	78
System Architecture:.....	78
3.7.6 Use Cases.....	79

Chapter 4: Implementation & Testing.....	83
4.1 Introduction.....	83
4.2 Component 1: Core Banking System — Apache Fineract.....	84
4.2.1 Overview.....	84
4.2.2 Technology Stack.....	85
4.2.3 Setup and Configuration.....	85
4.2.4 Execution Screenshots.....	87
4.2.5 Testing.....	88
4.3 Component 2: Bank Channel.....	88
4.3.1 Overview.....	88
4.3.2 Technology Stack.....	89
Backend:.....	89
Frontend:.....	89
4.3.3 Execution Screenshots.....	90
4.3.4 Testing.....	93
4.4 Component 3: PhoenixPay System.....	93
4.4.1 Overview.....	93
4.4.2 Technology Stack.....	93
Backend:.....	93
Merchant Dashboard:.....	94
4.4.3 Execution Screenshots.....	94
4.4.4 Testing.....	97
4.5 Component 4: TaskMaster — Demo Merchant Application.....	98
4.5.1 Overview.....	98
4.5.2 Technology Stack.....	98
Backend:.....	98
Frontend:.....	98
4.5.3 Execution Screenshots.....	99
4.5.4 Testing.....	101
Chapter 5: Results, Conclusion, and Recommendations.....	102
5.1 Results.....	102
5.1.1 Functional Requirement Coverage.....	102
5.1.2 Non-Functional Requirement Coverage.....	104
5.1.3 Test Results Summary.....	105
5.1.4 System Evaluation — Strengths and Limitations.....	106
5.2 Conclusion.....	108

5.3 Recommendations for Future Work.....	110
References.....	113
Appendices.....	117
Appendix A: Bank Channel Test Case Results.....	117
Appendix B: PhoenixPay Test Case Results.....	120
Appendix C: TaskMaster Test Case Results.....	125

List of Figures

Figure	Title
Figure 3.1	Entity-Component Model
Figure 3.2	Fineract Deployment Architecture
Figure 3.3	Bank Channel Component Architecture
Figure 3.4	Bank Channel Entity-Relationship Diagram
Figure 3.5	Bill-Payment Sequence Diagram
Figure 3.6	Bank Channel Use Case Diagram
Figure 3.7	PhoenixPay Layered Architecture
Figure 3.8	PhoenixPay Data Model
Figure 3.9	Invoice State Machine
Figure 3.10	Payout and Refund State Machines
Figure 3.11	Webhook Delivery with Exponential Backoff
Figure 3.12	PhoenixPay System Use Case Diagram
Figure 3.13	End-to-End Premium Subscription Flow
Figure 3.14	TaskMaster Use Case Diagram
Figure 4.1	MifosX Login Page
Figure 4.2	Fineract Client List (Three Registered Clients)
Figure 4.3	PhoenixPay Settlement Account Detail
Figure 4.4	Bank Channel Login Screen
Figure 4.5	Account Dashboard (Balances)
Figure 4.6	Peer-to-Peer Transfer Form
Figure 4.7	Bill Payment — Invoice Lookup
Figure 4.8	Bill Payment — Confirmation Screen
Figure 4.9	Merchant Dashboard — Developer Registration
Figure 4.10	Application Management
Figure 4.11	API Key Generation (Key Shown Once)
Figure 4.12	Invoice List with Filters
Figure 4.13	Balance View and Payout Request
Figure 4.14	Admin Dashboard
Figure 4.15	Swagger UI (Interactive API Documentation)
Figure 4.16	TaskMaster Login / Registration
Figure 4.17	Dashboard with Todo List
Figure 4.18	Premium Subscription Modal (Invoice Displayed)
Figure 4.19	Premium Features Unlocked
Figure 4.20	Cancel Premium Modal

List of Tables

Table	Title
Table 3.1	Functional Requirements
Table 3.2	Non-Functional Requirements
Table 3.3	Fineract Features Used in This Project
Table 3.4	Fineract Users
Table 3.5	Key Fineract API Calls
Table 3.6	Bank Channel Features and Functions
Table 3.7	Bank Channel Users
Table 3.8	Bank Channel Use Cases
Table 3.9	PhoenixPay Backend Features
Table 3.10	Merchant Dashboard Features
Table 3.11	PhoenixPay Users
Table 3.12	PhoenixPay Non-Functional Requirements
Table 3.13	PhoenixPay Authentication Design
Table 3.14	PhoenixPay Key API Endpoints
Table 3.15	PhoenixPay Use Cases
Table 3.16	TaskMaster Backend Features
Table 3.17	TaskMaster Frontend Features
Table 3.18	TaskMaster Users
Table 3.19	TaskMaster Use Cases
Table 4.1	Fineract Technology Stack
Table 4.2	Bank Channel Backend Technology Stack
Table 4.3	Bank Channel Frontend Technology Stack
Table 4.4	PhoenixPay Backend Technology Stack
Table 4.5	Merchant Dashboard Technology Stack
Table 4.6	TaskMaster Backend Technology Stack
Table 4.7	TaskMaster Frontend Technology Stack
Table 5.1	Functional Requirement Coverage
Table 5.2	Non-Functional Requirement Coverage
Table 5.3	Test Results by Component
Table 5.4	Test Results by Test Type
Table A.1	Bank Channel Test Results
Table B.1	PhoenixPay Test Results
Table C.1	TaskMaster Test Results

Chapter 1: Introduction

1.1 Background

Online payment systems have become a fundamental component of modern digital economies, enabling businesses, mobile applications, and online services to conduct secure and efficient financial transactions. Around the world, digital payment platforms such as PayPal and Stripe have transformed the way developers integrate payment services into their applications by providing unified APIs, developer tools, and automated transaction management systems.

In Sudan, however, integrating online payment solutions remains a major technical and operational challenge. Developers who wish to add payment functionality to their applications often face fragmented ecosystems where each bank or payment provider offers different integration methods, authentication mechanisms, and technical requirements. Furthermore, many providers lack sufficient API documentation, sandbox environments, and developer support, making the integration process difficult, time-consuming, and unreliable.

As a result, many developers resort to inefficient manual solutions such as direct bank transfers, manual receipt verification, or isolated integrations with individual payment providers. These methods increase operational complexity, introduce human error, reduce scalability, and create difficulties in maintaining secure and reliable payment workflows. The absence of a unified integration layer significantly limits the growth of e-commerce and digital services within Sudan.

With the increasing adoption of digital transformation and fintech technologies, automation and centralized payment orchestration have become essential solutions for overcoming these challenges. Unified payment integration platforms simplify communication between developers, merchants, and payment providers by offering a single secure API capable of handling multiple payment operations through a standardized architecture.

The PhoenixPay platform is designed to address these issues by providing a unified online payment integration system tailored to the Sudanese environment. The platform aims to simplify payment integration for developers through a centralized API, developer portal, transaction routing engine, webhook notification services, payout management, refund processing, and merchant ledger management.

The primary deliverable of this graduation project is the **PhoenixPay platform** itself — the unified payment integration API and developer portal. PhoenixPay is the core system, designed and built from scratch to address the identified problem. Three additional components were developed or deployed alongside it to create a complete, demonstrable payment ecosystem:

- **Apache Fineract**, an existing open-source core-banking platform, was deployed from its official Docker image and configured — not developed — to simulate realistic account management, fund transfers, settlement, and payout operations. Fineract is used as-is to represent the core bank.
- **The Bank Channel**, a custom-built web application, was developed specifically for this project to simulate the role of a bank's customer-facing payment interface — the payment network through which end-users initiate payments and which confirms transactions to PhoenixPay.
- **TaskMaster**, a demo merchant web application, was developed specifically to illustrate a complete PhoenixPay integration in action, from invoice creation through webhook-triggered feature activation.

The Bank Channel and TaskMaster are not standalone products; they exist solely to validate and demonstrate PhoenixPay's design in a realistic end-to-end context. By building all four components together, the project demonstrates not just that a unified payment API can be designed, but that a realistic payment ecosystem — covering the merchant, the platform, the bank channel, and the core bank — can be constructed and operated as a coherent system.

1.2 Problem Statement

Integrating online payment methods into applications in Sudan remains a difficult, fragmented, and inefficient process for developers and businesses. While certain digital payment platforms and gateways do exist in the market, they are notoriously difficult to integrate and completely lack public API documentation. Without open, accessible technical references, developers cannot easily evaluate or implement these services. Consequently, there is no unified, developer-friendly payment integration platform that allows businesses to connect multiple Sudanese payment providers through a single, standardized interface. Instead, developers must navigate opaque onboarding processes and interact separately with each provider, routinely dealing with inconsistent APIs, non-existent or outdated documentation, unreliable support, and varying authentication mechanisms.

Traditional approaches to payment integration frequently require manual processes such as maintaining dedicated bank accounts, manually verifying transaction receipts, or building isolated integrations for each provider. These approaches are time-consuming, difficult to maintain, prone to human error, and unsuitable for scalable digital systems.

In addition, developers and merchants lack centralized tools for managing invoices, tracking transaction statuses, handling refunds, monitoring balances, configuring webhooks, and processing payouts. The absence of standardized workflows and automated transaction management creates operational inefficiencies and reduces transaction reliability.

Furthermore, payment confirmations and transaction updates are often handled manually or through inconsistent communication channels, making it difficult to ensure secure, real-time, and traceable payment processing. These challenges negatively affect developers, businesses, and end-users by increasing

delays, reducing trust in digital payment systems, and limiting the adoption of online commerce solutions.

By addressing these challenges, the efficiency, reliability, and scalability of online payment integration in Sudan can be significantly improved through the development of a unified and secure payment integration platform.

Solving this problem, however, requires more than building an API in isolation. Proving that a payment platform works demands a complete ecosystem: a banking backend that manages accounts and executes transfers, a payment channel that simulates how banks confirm transactions, and a real merchant application that exercises the full payment lifecycle from the developer's perspective. Without these surrounding components, the platform cannot be demonstrated, tested, or evaluated in a realistic context.

1.3 Aims & Objectives

The PhoenixPay platform aims to overcome the limitations of fragmented and manual payment integration methods by introducing a centralized and secure online payment integration solution.

The project seeks to simplify payment integration for developers and merchants by providing a unified API capable of handling invoice creation, payment confirmation, transaction management, payouts, refunds, and webhook notifications through a standardized architecture.

In addition, the system aims to enhance transaction reliability, improve operational efficiency, reduce integration complexity, and provide a scalable foundation capable of supporting future payment providers and banking integrations within Sudan's digital payment ecosystem.

To achieve these aims, the following objectives are defined:

1. **Develop a Unified Payment API:** Provide a single standardized API for payment initiation, invoice creation, transaction tracking, payment confirmation, refunds, and payout management.
2. **Simplify Payment Integration:** Reduce the complexity of integrating Sudanese payment providers into applications by offering a centralized integration layer and developer-friendly architecture.
3. **Enhance Transaction Processing and Routing:** Implement a routing engine capable of validating transactions, handling invoice lifecycle states, processing confirmations, and maintaining transaction persistence.
4. **Implement Secure Authentication and Access Control:** Provide secure authentication mechanisms using API keys and JWT sessions while enforcing role-based access control and secure communication protocols.
5. **Implement Webhook-Based Notification Services:** Implement a targeted webhook service to handle successful invoice payment events , enabling secure callback delivery and automated transaction status synchronization between PhoenixPay and integrated merchant systems.
6. **Implement Merchant Ledger and Balance Management:** Maintain internal merchant balances, transaction ledgers, payout eligibility checks, and financial record consistency.
7. **Support Refund and Payout Operations:** Allow merchants to request payouts and process refunds securely while maintaining transaction traceability and auditability.
8. **Provide an Integrated Sandbox and Testing Environment:** Develop the platform as a fully simulated testing environment integrated with Apache Fineract to emulate banking operations, transaction processing, account balances, transfers, refunds, and payout workflows without involving real financial institutions or live banking systems.

9. Improve Monitoring, Logging, and Auditability: Implement audit logging and operational monitoring for tracking transactions, payment events, API activities, and security-sensitive operations.

10. Ensure Scalability and Extensibility: Design the system using scalable and modular architecture principles that allow additional payment providers and services to be integrated in the future with minimal effort.

In addition, the following supporting objectives are defined to enable realistic demonstration and validation of the core PhoenixPay platform:

11. Deploy and Configure Apache Fineract as a Simulated Core Bank: Set up Apache Fineract — an existing open-source core-banking platform, used as-is — as the simulated banking backend, including account provisioning, ledger management, transfer execution, and settlement workflows that support PhoenixPay's payout and refund operations.

12. Design and Implement a Simulated Bank Channel: Build a dedicated bank channel service that emulates the role of a bank's customer-facing payment interface — handling invoice inquiry, payment initiation, and transaction confirmation — to complete the payment loop between the end-user and PhoenixPay.

13. Build a Demo Merchant Application (TaskMaster): Develop a demonstration merchant web application that integrates with PhoenixPay end-to-end, illustrating invoice creation, payment processing, webhook handling, and subscription activation in a realistic usage scenario.

1.4 Scope

The project focuses on the design, development, and evaluation of PhoenixPay as a unified online payment integration platform tailored for the Sudanese digital payment environment. The platform is intended to simplify

payment integration for developers and merchants by providing centralized payment orchestration, standardized APIs, and simulated financial transaction workflows.

The **core deliverable** — the PhoenixPay platform — includes the development of:

- A unified payment integration API
- Developer and merchant authentication mechanisms
- Application and API key management
- Invoice creation and transaction lifecycle management
- Payment inquiry and confirmation workflows
- Webhook-based event notification services
- Merchant ledger and balance management
- Refund and payout processing modules
- Audit logging and transaction monitoring services
- Developer support and documentation interfaces

The PhoenixPay platform is internally organized into two sub-systems:

- A **Users Sub-System** responsible for developer management, authentication, merchant application registration, and access control
- A **Payment Sub-System** responsible for transaction routing, invoice processing, webhook orchestration, refunds, payouts, settlement simulation, and financial workflow coordination

The project also includes three **supporting components** developed or deployed to demonstrate and validate the core platform:

- **Apache Fineract** (used as-is, not developed) — deployed from its official Docker image and configured to emulate core-banking operations

including account balances, transfers, transaction posting, invoice payments, refund operations, settlement simulation, and payout processing

- **A simulated Bank Channel** — a custom-built web application that emulates a bank's customer-facing payment interface, handling payment confirmation, account operations, and banking transaction flows within the testing environment

- **TaskMaster** — a custom-built demonstration merchant application used to illustrate a complete PhoenixPay integration and validate the end-to-end transaction lifecycle

The platform is intended solely for testing, simulation, demonstration, and academic purposes. No real financial transactions, live banking integrations, or production payment infrastructures are involved in the implementation of the system.

However, the project does not include:

- Full commercial deployment
- Integration with live banking systems
- Real-money transaction processing
- Regulatory or financial compliance certification
- Mobile application development
- Advanced fraud detection systems
- Production-grade distributed infrastructure

These areas are considered possible future enhancements beyond the scope of the current prototype implementation.

1.5 Motivation

With the rapid growth of digital services and fintech technologies, the demand for reliable and scalable online payment integration solutions continues to increase. However, developers in Sudan still face significant challenges when attempting to integrate payment services into their applications due to fragmented provider ecosystems, inconsistent APIs, limited technical documentation, and the absence of unified integration standards.

The PhoenixPay platform is motivated by the need to simplify and modernize payment integration within the Sudanese digital ecosystem. The project aims to demonstrate how centralized payment orchestration, standardized APIs, and automated transaction workflows can reduce integration complexity and improve transaction management for developers and merchants.

In addition, the project seeks to provide a practical academic prototype that simulates realistic payment and banking workflows within a secure sandbox environment. Instead of relying on live banking systems or real financial transactions, the platform integrates with Apache Fineract to emulate core banking operations such as account balances, transfers, refunds, settlements, and payout processing for testing and demonstration purposes.

The motivation behind the project also includes applying modern software engineering and fintech concepts in a real-world-inspired environment, including:

- API design
- Transaction lifecycle management
- Event-driven webhook systems
- Merchant ledger management
- Secure authentication and authorization
- Audit logging

- Payment routing
- Scalable backend architecture

Furthermore, the project contributes academically by exploring how unified payment integration platforms can support digital transformation and fintech innovation within emerging markets such as Sudan, while providing a safe testing environment for studying payment workflows without involving production banking infrastructure.

An important part of this motivation was the decision to build the surrounding ecosystem — not just the PhoenixPay API — as fully working deliverables. Deploying and configuring Apache Fineract demonstrates how an open-source core bank can underpin a payment platform. Building the bank channel shows how a payment network's confirmation interface can be simulated for development and testing purposes. Developing TaskMaster proves that the integration is practical and usable from a merchant developer's perspective. Together, these components transform the project from a theoretical API design into a working, end-to-end payment environment.

1.6 Research Methodology

This project follows a two-part methodology that separates understanding the problem from building the solution. A multiple-case study was used to investigate the fragmented state of online payment integration in Sudan and to derive the system's requirements from existing practice, and the Agile Scrum framework was used to develop and validate the PhoenixPay platform and its supporting components.

The case study examined existing payment platforms as individual cases to establish what a successful integration platform requires and where current provision falls short. Seven platforms were studied — the global benchmarks

Stripe and PayPal, the regional aggregators Flutterwave and Moyasar, the mobile-money service M-Pesa, and the local Sudanese providers CashIPay and SyberPay — supported by an analysis of current Sudanese integration workarounds and by academic studies of financial inclusion. As these are commercial systems, the study relied on secondary data: official documentation, public sandboxes and developer portals, technical reports, domain experts advice, and peer-reviewed literature. Comparing the cases revealed a common set of proven design patterns — RESTful APIs, API-key authentication, idempotent payment confirmation, signed webhook callbacks, and internal merchant ledgers — together with the gaps that no locally available provider fills. These findings, summarized in Section 2.5, were translated directly into the functional and non-functional requirements specified in Chapter 3.

For development, the four-member team adopted Scrum, an iterative and incremental Agile framework suited to evolving requirements and to a system of interdependent components built within a fixed academic timeframe. The requirements formed a product backlog from which work was selected and delivered as a working increment each iteration. Rather than appointing fixed role-holders, the team shared the Product Owner and Scrum Master responsibilities collectively. Work was organized into twelve one-week sprints, each opening with sprint planning and closing with a review of the increment and a short retrospective, supported by brief daily stand-ups. Successive sprints built the system in dependency order: the core banking backend (Apache Fineract) and the bank channel first, then the PhoenixPay API with its ledger, webhook, payout, and refund services, then the developer portal and the TaskMaster demo application, and finally end-to-end integration and testing.

Validation was continuous: every increment was tested before being accepted, and the completed system was verified as a whole by running the full stack under Docker Compose and executing 54 functional, security, and

integration test cases (Chapter 4), all of which passed. The decisive check was an end-to-end test of the complete payment loop — from a TaskMaster user initiating a subscription, through the bank channel confirming the payment, to PhoenixPay crediting the merchant ledger and delivering the webhook that activates the feature — confirming that the system meets the objectives defined in Section 1.3.

1.7 Structure of Research

The remainder of this report is organized into four chapters as follows:

Chapter 2 — Literature Review establishes the theoretical background of digital payments and financial inclusion, then presents the multiple-case study that underpins the project's problem analysis: a structured review of eleven sources, comprising seven existing payment platforms — global (Stripe, PayPal), regional (Flutterwave, Moyasar), mobile-money (M-Pesa), and local Sudanese (CashiPay, SyberPay) — together with an examination of current Sudanese integration workarounds and three academic studies of financial inclusion. The chapter concludes by consolidating these findings into a comparison table and identifying the specific gap that PhoenixPay addresses.

Chapter 3 — System Analysis & Design specifies the platform's functional and non-functional requirements, introduces the three-entity model (Bank, PhoenixPay, Merchant) that frames how the four components relate, and then examines each component in depth — covering its purpose, users, architecture, database schema, API design, security model, state machines, and use cases. PhoenixPay, as the core deliverable, receives the most detailed treatment.

Chapter 4 — Implementation & Testing documents how each component was built or deployed — the technology stacks, the configuration of

Apache Fineract, and the implementation of the Bank Channel, the PhoenixPay backend and developer portal, and the TaskMaster demo application, together with the Docker-based deployment — and then reports the results of the 54 manual functional, integration, security, and negative test cases executed against the running system.

Chapter 5 — Results & Recommendations evaluates the platform against its defined requirements, summarizes the test outcomes, discusses the strengths and limitations of the implementation, and concludes with a prioritized set of recommendations for evolving the prototype toward a production-ready system.

Chapter 2: Literature Review

2.1 Introduction

This chapter examines existing online payment integration platforms and systems relevant to the Sudanese digital payment landscape. Based on a review of established global, regional, and local payment platforms, it explores their core functionalities, technical approaches, and developer-facing features, while identifying recurring limitations and gaps. In doing so, it highlights the common challenges faced by developers attempting to integrate payment services into their applications and provides insights that inform the design and development of PhoenixPay. These examples demonstrate the fragmented state of payment infrastructure in markets similar to Sudan and outline the areas that the proposed platform seeks to address.

The digital payment landscape in developing economies has grown rapidly in recent years, yet significant gaps remain in access, integration, and interoperability. According to [18], only 57% of adults in developing countries made or received a digital payment, compared to 64% worldwide, highlighting a persistent gap that unified payment platforms can help close. In Africa specifically, the payment systems landscape remains fragmented, with limited interoperability between providers acting as a key barrier to financial inclusion and developer adoption [19]. Sudan reflects this broader pattern, where multiple payment providers such as Bankak, Fawry, O-Cash, and Cashier operate in isolation, each with different integration requirements, limited documentation, and no unified developer interface.

This chapter is structured as follows. Section 2.2 provides the theoretical background contextualising the problem within the broader fintech and payment integration literature. Section 2.3 presents a detailed review of eleven existing payment platforms, approaches, and academic studies, examining each system's features, strengths, and limitations, and explaining how PhoenixPay addresses the

identified gaps. Section 2.4 presents the conclusion drawn from the review, and Section 2.5 provides a summary comparison table of all reviewed systems.

2.2 Background

The rapid growth of digital commerce has made reliable online payment infrastructure a critical component of modern economies. However, in many developing economies, including Sudan, this infrastructure remains fragmented, underdocumented, and difficult to integrate. Recent empirical evidence underscores that such payment fragmentation heavily restricts the adoption of digital transaction systems, whereas network interoperability and unified API layers can accelerate user and merchant financial engagement by over 50% within a short timeframe [30]. Furthermore, systematic insights into fintech ecosystems within Sub-Saharan Africa confirm that seamless technical integration, system usability, and standardized infrastructures are the primary structural drivers for sustainable mobile money adoption and broader financial inclusion [31].

The concept of financial inclusion — defined as the availability of and equal access to useful and affordable financial services — is central to understanding why unified payment infrastructure matters beyond its technical dimensions. Research consistently shows that fragmented and inaccessible payment systems are among the primary barriers to financial inclusion in developing economies. The Global Findex report [18] finds that while financial account ownership has grown globally, significant gaps remain in developing countries, particularly in the adoption of digital payments for everyday transactions. In Africa, this challenge is compounded by limited interoperability between payment providers. As noted in [19], the majority of instant payment systems across Africa are not interoperable with one another, meaning that funds, data, and developer integrations cannot flow freely between providers — a

situation that directly mirrors the challenges PhoenixPay seeks to resolve in Sudan.

The experience of mobile money platforms in sub-Saharan Africa offers an important precedent for understanding how unified payment infrastructure can drive financial inclusion. As demonstrated in [20], a review of the African mobile money landscape shows that platforms which successfully aggregate payment functionality — most notably M-Pesa in Kenya — achieved widespread adoption precisely because they offered a single, accessible interface for both users and developers. This lesson is directly applicable to the Sudanese context, where a similar aggregation approach at the developer API level could unlock significant growth in digital commerce and financial access. PhoenixPay draws on this precedent by providing a unified API layer that abstracts the complexity of Sudan's fragmented payment ecosystem, enabling developers to integrate multiple providers through a single, well-documented interface.

2.3 Case Studies

2.3.1 Stripe

Stripe is a globally recognised online payment platform founded in 2010, designed to simplify the process of accepting and managing payments over the internet. It serves as both a payment gateway and a payment processor, providing businesses and developers with a unified interface to handle transactions, subscriptions, refunds, and payouts across more than 135 currencies and dozens of payment methods. Stripe is widely regarded as the benchmark for developer-facing payment API design, and its architectural decisions and documentation standards have influenced the design of numerous payment platforms worldwide, including PhoenixPay.

Stripe's core offering is built around a RESTful API specified using industry-standard conventions, supported by official Software Development Kits (SDKs) in multiple programming languages including Python, Node.js, Ruby, Java, and PHP. The platform provides a comprehensive developer dashboard, sandbox testing environment, webhook-based event notifications, and detailed API reference documentation. Authentication is handled through API keys, with a clear separation between publishable and secret keys — a model that PhoenixPay adopts directly in its own API key design. Stripe also enforces HTTPS for all communications and provides idempotency key support to prevent duplicate transaction processing [13].

Pros:

- Provides a unified, well-documented RESTful API that allows developers to integrate multiple payment methods through a single interface, significantly reducing integration complexity.
- Offers a comprehensive sandbox testing environment with test API keys, simulated payment scenarios, and detailed error responses, enabling developers to validate integrations before going live.
- Supports webhook-based event notifications with automatic retry policies, allowing merchant applications to receive real-time updates on payment events such as successful charges and refund completions.
- Implements a clean and secure API key model — separating publishable and secret keys — with support for key rotation and revocation, setting a strong security baseline for payment API design.
- Maintains extensive, consistently updated documentation with code samples in multiple programming languages, interactive API explorers, and integration guides tailored to different use cases.

Cons:

- Stripe does not support direct integration with Sudanese banks or local mobile wallet providers such as Bankak, Fawry, O-Cash, or Cash, making it effectively inaccessible for developers building payment solutions in Sudan.
- The platform does not support the Sudanese Pound (SDG) as a currency, which is a fundamental requirement for any payment system operating within Sudan's financial ecosystem.
- Stripe's availability is restricted in Sudan due to international banking regulations and sanctions, meaning Sudanese developers and businesses cannot create Stripe accounts or process live transactions through the platform.
- While Stripe's feature set is comprehensive, its complexity can be overwhelming for developers new to payment integration, particularly in markets where technical support and local developer communities are limited.

Observed Limitations and Addressed Gaps:

Stripe represents the global gold standard in payment API design and provides an invaluable reference model for developer experience, documentation quality, and security architecture. However, its inaccessibility in Sudan — due to currency limitations, lack of local provider support, and regulatory restrictions — means it cannot serve the needs of Sudanese developers directly. PhoenixPay addresses these limitations by adopting Stripe's proven design principles — including its API key model, RESTful architecture, webhook notification system, and sandbox testing environment — while building natively on Sudan's local payment ecosystem. Where Stripe serves a global market, PhoenixPay is purpose-built for Sudan, supporting SDG as its system currency and designed to

integrate with local providers such as Bankak, Fawry, O-Cash, and Cashier through a single, unified API.

2.3.2 PayPal

PayPal is one of the world's oldest and most widely recognised online payment platforms, founded in 1998 and currently serving over 400 million active accounts across more than 200 countries and regions. It operates as both a consumer digital wallet and a merchant payment platform, enabling individuals and businesses to send, receive, and manage money online. PayPal offers a REST-based API that allows developers to integrate payment functionality into their applications, supporting features such as one-time payments, recurring billing, refunds, and dispute management. Its brand recognition and global reach have made it a default payment option for many e-commerce platforms worldwide [22].

PayPal's developer platform provides REST APIs with OAuth 2.0 authentication, official SDKs for multiple programming languages, a sandbox testing environment, and a developer dashboard for managing applications and credentials. It supports webhooks for real-time event notifications and provides detailed API reference documentation. PayPal's payment flow separates the concepts of orders, authorisations, and captures, giving developers fine-grained control over when funds are reserved and when they are actually collected.

Pros:

- Offers a globally recognised and trusted brand that increases consumer confidence at checkout, reducing payment abandonment rates for merchants who integrate it.

- Provides a well-structured REST API with OAuth 2.0 authentication, official SDKs, and a sandbox environment, giving developers a solid foundation for building and testing payment integrations.
- Supports a wide range of payment scenarios including one-time payments, subscriptions, invoicing, and peer-to-peer transfers, making it versatile across different business models.
- Includes buyer and seller protection policies that reduce the financial risk of fraud and disputes, adding an additional layer of trust for both merchants and customers.
- Provides webhook support for real-time event notifications, enabling merchant applications to respond automatically to payment events without continuous polling.

Cons:

- PayPal's availability in Sudan is severely restricted due to international sanctions and banking regulations. Sudanese developers and businesses cannot create PayPal merchant accounts or process live transactions through the platform, making it practically inaccessible for the Sudanese market.
- The platform does not support the Sudanese Pound (SDG), which is a fundamental requirement for any payment system intended to operate within Sudan's financial ecosystem.
- PayPal's API documentation, while extensive, is inconsistent across different API versions. The coexistence of legacy NVP/SOAP APIs and newer REST APIs creates confusion for developers navigating between older and newer integration patterns.

- Transaction and currency conversion fees can be significant, particularly for businesses in developing markets operating on thin margins, making PayPal a costly option even where it is technically available.
- PayPal does not provide direct integration with local Sudanese payment providers such as Bankak, Fawry, O-Cash, or Cashi, meaning it cannot serve as a unified solution for Sudan's fragmented payment landscape.

Observed Limitations and Addressed Gaps:

PayPal's global reach and consumer trust make it a powerful platform in markets where it is fully available. However, its inaccessibility in Sudan — driven by regulatory restrictions, lack of SDG support, and absence of local provider integration — renders it unsuitable as a payment solution for Sudanese developers and businesses. Furthermore, its inconsistent documentation across API versions highlights a documentation quality challenge that PhoenixPay directly addresses through a single, consistently maintained OpenAPI 3.0 specification. PhoenixPay fills the gap left by PayPal in the Sudanese market by providing a locally accessible, fully documented payment platform that supports SDG and integrates natively with Sudan's existing payment providers, offering Sudanese developers the reliability and developer experience that PayPal provides in other markets.

2.3.3 Flutterwave

Flutterwave is an African-founded financial technology company established in 2016, headquartered in San Francisco with operations across more than 30 African countries. It was built specifically to solve the payment fragmentation problem in Africa by providing a single, unified API that aggregates multiple local payment methods — including mobile money, bank transfers, card payments, and digital wallets — behind one integration layer. Flutterwave serves as one of the closest real-world analogues to what PhoenixPay

aims to achieve in Sudan, operating on the same core principle of abstracting fragmented local payment infrastructure behind a developer-friendly unified interface [23].

Flutterwave's technical offering includes a RESTful API with API key authentication, official SDKs for multiple programming languages, a sandbox testing environment, webhook notifications, and a merchant dashboard for managing transactions, payouts, and refunds. It supports payments in multiple African currencies and provides a hosted payment page option for merchants who prefer a low-code integration approach. Its platform supports a wide range of payment scenarios including one-time payments, recurring billing, split payments, and bulk payouts.

Pros:

- Aggregates multiple African payment methods — including mobile money, bank transfers, USSD, and card payments — behind a single unified API, directly addressing the fragmentation problem that developers face across African markets.
- Provides a well-documented RESTful API with sandbox testing, webhook support, and official SDKs, offering a strong developer experience comparable to global platforms like Stripe within the African context.
- Supports payments in multiple African currencies, demonstrating that a locally focused platform can handle the currency diversity of the African continent without requiring developers to manage currency conversion manually.
- Offers both API-based and hosted payment page integration options, giving developers flexibility in how deeply they integrate the platform into their applications.

- Has processed billions of dollars in transactions across Africa, demonstrating proven scalability and reliability at significant transaction volumes.

Cons:

- Flutterwave does not currently support direct integration with Sudanese payment providers such as Bankak, Fawry, O-Cash, or Cash, meaning it cannot serve as a unified solution for Sudan's specific payment ecosystem despite its broader African coverage.
- The platform does not support the Sudanese Pound (SDG) as a transaction currency, which is a fundamental requirement for payment operations within Sudan.
- Flutterwave's availability and feature set vary significantly across its supported countries, meaning developers in less-covered markets cannot rely on consistent API behaviour or support.
- While its documentation is generally good, some advanced features and country-specific integration details are insufficiently documented, requiring developers to rely on community support or direct contact with Flutterwave's team.
- The platform has faced regulatory challenges in some African markets, including temporary suspensions of operations, which raises reliability concerns for developers building production systems on top of it.

Observed Limitations and Addressed Gaps:

Flutterwave is perhaps the most conceptually similar platform to PhoenixPay, built on the same foundational principle of unifying fragmented African payment infrastructure behind a single developer API. Its success across more than 30 African countries demonstrates that the unified payment platform model is viable and valuable in African markets. However, Flutterwave's lack of

support for Sudanese payment providers and SDG currency means it cannot directly serve Sudan's payment ecosystem. PhoenixPay addresses this gap by applying Flutterwave's aggregation model specifically to the Sudanese context, providing native support for local providers such as Bankak, Fawry, O-Cash, and Cashi, with SDG as the system currency and documentation tailored to the needs of Sudanese developers.

2.3.4 M-Pesa (Daraja)

M-Pesa is a mobile money platform launched in 2007 by Safaricom in Kenya, and is widely regarded as one of the most successful financial technology innovations in the developing world. The name M-Pesa combines the Swahili word for money ("pesa") with the prefix "M" for mobile, reflecting its core proposition of enabling financial transactions through mobile phones. Over the past two decades, M-Pesa has expanded beyond Kenya to operate in several African countries including Tanzania, Ghana, Egypt, and Mozambique, serving tens of millions of active users. Its success has been extensively studied as a model for how mobile-based payment infrastructure can drive financial inclusion in markets where traditional banking infrastructure is limited or inaccessible [20].

The Daraja API is M-Pesa's developer-facing interface, provided through Safaricom's developer portal. It exposes a set of REST-based endpoints that allow merchant applications to initiate payment requests, receive payment confirmations, process refunds, and query transaction status. The Daraja API supports key payment flows including Customer to Business (C2B), Business to Customer (B2C), and Business to Business (B2B) transactions. Authentication is handled through OAuth 2.0, and the platform provides a sandbox environment for testing integrations. Webhook-style callback URLs are used to notify merchant applications of payment outcomes in real time [24].

Pros:

- Demonstrates conclusively that a mobile-based payment platform can achieve massive scale and widespread adoption in a developing economy with limited traditional banking infrastructure, providing a strong proof of concept for PhoenixPay's goals in Sudan.
- Provides a developer-facing REST API through the Daraja portal, giving merchants and developers a programmatic interface to integrate M-Pesa payments into their applications.
- Supports multiple payment flows including C2B, B2C, and B2B transactions, covering a wide range of real-world payment scenarios for both consumer and business use cases.
- Includes a sandbox testing environment and OAuth 2.0 authentication, reflecting good practices in developer-facing API design that are directly relevant to PhoenixPay's own design decisions.
- Has been the subject of extensive academic research demonstrating its positive impact on financial inclusion, poverty reduction, and economic participation among previously unbanked populations [20].

Cons:

- M-Pesa and the Daraja API are primarily available in Kenya and a limited number of other African countries, and do not extend to Sudan or support Sudanese payment providers, making direct adoption impossible for Sudanese developers.
- The Daraja API documentation, while publicly available, is considered inconsistent and difficult to navigate in places, with some endpoints poorly documented and requiring developers to rely on community resources for clarification.

- M-Pesa's payment model is tied to Safaricom's mobile network infrastructure, meaning it is not provider-agnostic and cannot be extended to aggregate other payment methods or banks behind a single interface.
- The platform does not support the Sudanese Pound (SDG) and its currency support is limited to the markets in which it directly operates.
- Integration complexity can be significant for developers unfamiliar with M-Pesa's specific authentication flows and callback-based confirmation model, particularly without strong local developer support resources.

Observed Limitations and Addressed Gaps:

M-Pesa's success across Africa provides one of the most compelling real-world demonstrations that unified, mobile-accessible payment infrastructure can transform financial participation in developing economies — a lesson that is directly applicable to Sudan. As highlighted in [20], M-Pesa's success was driven not only by its technology but by its ability to meet users and developers where they are, offering a simple, accessible interface in a market with limited alternatives. However, M-Pesa's geographic limitations, provider-specific architecture, and documentation inconsistencies mean it cannot serve as a solution for Sudan's fragmented payment ecosystem. PhoenixPay draws on M-Pesa's success as a model while addressing its limitations — providing a provider-agnostic unified API that is specifically designed for Sudan's payment landscape, with comprehensive OpenAPI 3.0 documentation and a sandbox environment that removes the integration barriers M-Pesa's Daraja API still presents to developers.

2.3.5 Moyasar

Moyasar is a Saudi Arabian payment platform founded in 2019, designed to simplify online payment integration for businesses and developers operating in

the Middle Eastern and North African (MENA) region. It provides a unified payment API that aggregates multiple local payment methods including Mada (Saudi Arabia's national debit network), Visa, Mastercard, and Apple Pay, behind a single developer-friendly interface. Moyasar is particularly relevant to the PhoenixPay context because it demonstrates that the unified payment platform model is viable and valuable not only in Western markets but also in Arabic-speaking, Islamic-finance-aware markets that share cultural, regulatory, and economic characteristics with Sudan [21].

Moyasar's technical offering includes a RESTful API formally documented using the OpenAPI standard, API key authentication with separate publishable and secret keys, a sandbox testing environment, webhook notifications for real-time event updates, and a merchant dashboard for managing transactions and API credentials. It supports both direct API integration and hosted payment page options, giving developers flexibility in their integration approach. All API communications are conducted over HTTPS, and the platform provides clear, consistent error responses following standard HTTP status code conventions.

Pros:

- Provides a unified API that aggregates multiple MENA-specific payment methods behind a single integration point, demonstrating that the payment aggregation model works effectively in Arabic-speaking markets with regulatory and cultural characteristics similar to Sudan.
- Offers clean, well-structured OpenAPI documentation in both Arabic and English, making it one of the few payment platforms to prioritise Arabic-language developer resources — a significant consideration for Sudan's developer community.

- Implements a familiar and secure API key model with separate publishable and secret keys, consistent with the design approach adopted by Stripe and directly reflected in PhoenixPay's own authentication design.
- Provides a sandbox environment and webhook support, enabling developers to test integrations thoroughly before going live and receive real-time notifications of payment events.
- Demonstrates that a regionally focused payment platform can achieve strong developer adoption by combining global API design best practices with local payment method support.

Cons:

- Moyasar is designed specifically for the Saudi Arabian market and does not support Sudanese payment providers such as Bankak, Fawry, O-Cash, or Cashi, making it inaccessible as a direct payment solution for Sudanese developers and businesses.
- The platform does not support the Sudanese Pound (SDG), as its currency support is limited to Saudi Riyal (SAR) and other currencies relevant to the MENA region.
- Moyasar's merchant registration and account verification processes are restricted to businesses legally registered in Saudi Arabia, excluding Sudanese businesses from direct access to the platform.
- While its documentation is well-structured, Moyasar's developer community and third-party integration resources are significantly smaller than those of global platforms like Stripe or PayPal, limiting the availability of community support for developers encountering integration challenges.
- The platform's feature set, while sufficient for the Saudi market, does not include advanced features such as split payments or multi-provider routing

that would be necessary for a truly unified payment platform serving multiple providers simultaneously.

Observed Limitations and Addressed Gaps:

Moyasar is the most geographically and culturally proximate reference platform to PhoenixPay among the systems reviewed, demonstrating that the unified payment API model can be successfully applied in Arabic-speaking markets with regulatory and financial characteristics similar to Sudan. Its adoption of OpenAPI documentation standards, dual API key authentication, and webhook-based notifications provides a strong regional precedent for PhoenixPay's own design decisions. However, Moyasar's restriction to the Saudi Arabian market, its lack of SDG support, and its exclusion of Sudanese payment providers mean it cannot serve Sudan's payment ecosystem. PhoenixPay addresses these gaps by applying Moyasar's regionally sensitive approach to the Sudanese context, providing a platform that is not only technically aligned with modern API design standards but also natively integrated with Sudan's local payment providers and financial regulations.

2.3.6 CashIPay

CashIPay is the online payment and checkout solution offered by Cashi — a financial platform operated by Alsoug (Soug al-Sudan), Sudan's leading e-commerce marketplace, and licensed by the Central Bank of Sudan. While the broader Cashi ecosystem includes consumer products such as the MyCashi mobile wallet and a nationwide Cashi Points agent network, CashIPay is specifically designed for web-based merchants who need to accept digital payments within their online applications. It provides payment link generation and a checkout integration layer, enabling e-commerce businesses to accept Cashi wallet payments from customers without requiring card infrastructure. A REST

API for programmatic payment initiation is listed on the official product page as forthcoming [25], [26].

Pros:

- Operated by an entity fully licensed by the Central Bank of Sudan, providing regulatory legitimacy within the Sudanese financial system — a rare attribute among Sudanese merchant payment tools.
- Targets web-based and online merchants directly, positioning CashIPay closer to a developer-facing payment integration product than a purely consumer-facing wallet service.
- Offers instant settlement on payment confirmation, addressing one of the core merchant concerns in markets where settlement delays create cash-flow problems.
- Eliminates card dependency by routing payments through the CashIPay wallet network, which is relevant in Sudan where card acceptance infrastructure remains inconsistent across banks and merchants.
- Part of an ecosystem that already serves a large base of active merchants and wallet users, giving CashIPay immediate network reach that a new payment API would take years to build independently.

Cons:

- The REST API described on the CashIPay product page is listed as "coming soon" at the time of writing, meaning no public API documentation, endpoint reference, or sandbox environment is currently available for merchant developers to evaluate or integrate against [25].
- Without published developer documentation, merchants cannot independently implement a CashIPay integration — adoption currently

requires direct engagement with Cashi's support team via WhatsApp, which does not scale to a developer self-service model.

- No publicly documented webhook system exists, meaning merchant applications cannot receive automated, real-time payment confirmations — a fundamental limitation for building event-driven payment flows.
- Integration is currently limited to Cashi wallet users and does not support card payments, bank transfers, or other payment methods, restricting the addressable audience for merchants who integrate it.

Observed Limitations and Addressed Gaps:

CashiPay represents a meaningful step toward a developer-facing payment integration product in Sudan: its positioning as an online checkout solution for web merchants, combined with the announced REST API, signals recognition that Sudanese developers need a programmatic payment integration layer. However, in its current state, the absence of published API documentation, a sandbox environment, and webhook support means CashiPay does not yet provide the self-service developer experience that characterises mature payment platforms. PhoenixPay addresses the same need — online payment integration for web-based Sudanese merchants — but does so with a fully documented OpenAPI 3.0 specification, a working sandbox, idempotent payment confirmation, and HMAC-signed webhook delivery implemented and operational today. CashiPay's roadmap and PhoenixPay's design converge on the same vision; PhoenixPay demonstrates what that vision looks like when the API layer is fully realised.

2.3.7 SyberPay

SyberPay is the merchant-facing payment gateway product of Syber Group, a Sudanese technology company founded in 2015 with operations across Sudan, the UAE, Egypt, and Rwanda. Distinct from the SyberPayPlus consumer

mobile wallet, the SyberPay payment gateway enables businesses to accept online payments from customers using local ATM and debit cards from more than 30 Sudanese banks, internet PIN (iPIN), OTP-authenticated cards, and mobile wallets. SyberPay pioneered PIN-based card processing in Sudan and is integrated into e-commerce platforms including WooCommerce, OpenCart, and Magento through official and community-maintained plugins. The gateway serves more than 320 active merchants and connects to 36 or more Sudanese banks [27], [28].

Pros:

- Provides genuine merchant-facing payment gateway functionality — accepting card payments from 30+ Sudanese banks through a single integration — making it the most technically complete local payment gateway product identified for the Sudanese market.
- Offers ready-to-use integration plugins for major e-commerce platforms (WooCommerce, OpenCart, Magento), lowering the barrier to integration for merchants who operate on those platforms without requiring custom API development.
- Supports multiple card authentication methods — PIN, iPIN, and OTP — demonstrating security-conscious design and alignment with the authentication capabilities of Sudan's banking infrastructure.
- Has demonstrated commercial scale across 320+ merchants and 36+ banks, confirming that the payment gateway model is operationally viable in the Sudanese market.
- Community-maintained integration packages confirm that a REST API exists and document its core endpoints, including payment initiation (`POST /api/transactions/syber/payment`), webhook notification (`POST /api/transactions/syber/notify`), and cancellation callbacks [29].

Cons:

- SyberPay does not publish a formal, publicly accessible developer API documentation portal. The most detailed technical information available at time of writing comes from a community-contributed GitHub package rather than an official Syber Group API reference, meaning developers cannot rely on authoritative, version-controlled documentation [29].
- The absence of an official sandbox environment means developers must integrate against a live system or rely on community resources to understand expected API behaviour before building and testing their integrations.
- No official webhook documentation is published; webhook endpoint patterns are inferred from the community GitHub package, which may not reflect the full or current API specification.
- SyberPay's plugin-first integration model suits standard e-commerce platforms but does not provide the flexibility needed by developers building custom merchant applications — such as a SaaS platform that requires programmatic invoice creation, status querying, and event-driven payment confirmation.

Observed Limitations and Addressed Gaps:

SyberPay's payment gateway demonstrates that card-based online payment infrastructure is technically achievable in Sudan and has reached meaningful merchant adoption. Its support for 30+ banks and multiple authentication methods represents a real and significant contribution to Sudan's payment ecosystem. However, the absence of official API documentation, a public sandbox, and a self-service developer portal means the integration experience does not meet the standard that modern developer-facing payment platforms — Stripe, Flutterwave, Moyasar — have established. PhoenixPay complements this infrastructure by

providing the developer experience layer that is missing: a fully documented OpenAPI 3.0 specification, a working sandbox, an idempotent payment confirmation endpoint, HMAC-signed webhooks with exponential-backoff retry, and a complete audit trail — all accessible without any direct contact with the platform operator.

2.3.8 Fragmented Payment Integration in Sudan: Direct Integration and Manual Workarounds

In the absence of a unified payment integration platform in Sudan, developers building applications that require payment functionality have historically resorted to two broad categories of manual workarounds. These workarounds are not formal platforms or documented systems — they are ad hoc practices that have emerged out of necessity in a market where no standardised developer-facing payment API is publicly available. Bankak and Fawry are among the most widely used payment services in Sudan, with Bankak serving as a leading interbank transfer platform and Fawry providing bill payment and digital wallet services. However, neither provider offers publicly accessible developer documentation or a standardised API that merchants can integrate into their applications in a consistent and reliable way. As a result, developers have been forced to adopt the following approaches:

The first approach is **direct provider contact and custom integration**. Developers who wish to integrate Bankak or Fawry into their applications must contact each provider individually and attempt to negotiate access to whatever technical interface may be available. This process is slow, unpredictable, and yields inconsistent results. Because no public documentation exists for these providers, developers have no way to evaluate the integration requirements in advance, compare providers, or build on shared community knowledge. The resulting integrations — where they are achievable at all — are built on privately

shared information that is subject to change without notice, making them fragile and difficult to maintain.

The second approach is **manual receipt verification**. Many Sudanese merchants — particularly smaller businesses — bypass automated payment processing entirely by asking customers to make a bank transfer manually and then upload a photo or screenshot of the transfer receipt. The merchant then manually verifies the receipt, confirms the payment, and fulfils the order. This process is highly inefficient, prone to human error and fraud, difficult to scale, and provides no programmatic confirmation of payment status that an application can act upon automatically.

Challenges Faced by Developers:

Regarding direct provider contact and custom integration:

- The absence of public documentation means developers have no way to verify what features, capabilities, or integration requirements are available in advance, creating significant uncertainty that undermines confidence in building production systems on top of these integrations.
- The process of negotiating access with each provider individually is slow, unpredictable, and inaccessible to most developers, effectively limiting payment integration to those with the time and connections to navigate it.
- Integrations built on privately shared information are fragile and subject to breaking without warning if the provider changes their internal systems, with no public changelog or developer communication to notify affected developers.

Regarding manual receipt verification:

- This approach is highly vulnerable to fraud, as receipts can be forged or reused, and there is no automated mechanism to verify the authenticity or uniqueness of a payment confirmation.

- It provides no webhook or callback mechanism, forcing merchant applications to rely entirely on manual checks to determine payment status, which is both inefficient and unreliable.
- It provides no standardised error handling, audit logging, or idempotency guarantees, making it fundamentally incompatible with the requirements of reliable production-grade payment processing.
- It does not scale — as transaction volumes grow, the manual verification effort increases significantly, creating an operational bottleneck that limits business growth.

Observed Limitations and Addressed Gaps:

The prevalence of manual payment workarounds in Sudan represents the most direct evidence of the gap that PhoenixPay is designed to fill. These workarounds are not edge cases — they are the current standard practice for many Sudanese developers and businesses attempting to accept digital payments. They are slow, error-prone, unscalable, and fundamentally incompatible with the requirements of modern digital commerce. PhoenixPay addresses these limitations by providing a unified, well-documented REST API that abstracts the complexity of Sudan's payment providers behind a single integration point, eliminating the need for direct provider negotiation. It replaces manual receipt verification with automated payment confirmation through a webhook notification system, provides idempotency guarantees to prevent duplicate processing, and maintains a full audit log of all payment events. As a prototype, PhoenixPay currently demonstrates these capabilities through integration with Apache Fineract as a demo core banking backend, with the architecture designed to support real provider integration in future development.

2.3.9 Demirgüç-Kunt, Klapper, Singer & Ansar (2022): The Global Findex Database 2021

The Global Findex Database 2021, published by the World Bank, is the most comprehensive source of internationally comparable data on financial inclusion, account ownership, and digital payment adoption across 123 economies. Compiled from surveys conducted in 2021 and published in 2022, the report provides the empirical foundation for understanding where digital payment adoption stands globally and in developing economies specifically [18].

The report's central finding for developing economies is that the share of adults making or receiving digital payments grew from 35% in 2014 to 57% in 2021 — a substantial increase that nonetheless leaves a significant proportion of adults in developing countries outside the digital payment ecosystem. Globally, the equivalent figure is 64%, confirming a persistent gap between developed and developing markets. The report also documents that the COVID-19 pandemic accelerated digital payment adoption: approximately 40% of adults in developing economies who made a digital merchant payment in 2021 reported doing so for the first time after the pandemic began, driven by the need to avoid physical cash handling.

Of particular relevance to the Sudanese context is the report's Sub-Saharan Africa data, which shows that 33% of adults in the region hold a mobile money account — a form of financial access that in many cases substitutes entirely for formal banking. In Sudan specifically, the data reveals that 92% of adults who use mobile money do not hold a formal financial account, meaning mobile money operates outside the formal financial system rather than complementing it.

Relevance to PhoenixPay:

These findings directly contextualise the problem PhoenixPay addresses. The 57% digital payment adoption rate in developing economies, and the even

lower implied rate for Sudan given the informal character of its mobile money usage, confirms a large underserved population. The predominance of mobile money over formal banking in Sudan shapes PhoenixPay's design decisions: by building on local payment infrastructure (including mobile wallet providers) rather than requiring card accounts, PhoenixPay is aligned with the payment reality of the Sudanese market as documented by Global Findex data.

2.3.10 ECDPM (2024): Gaps in Interoperability as Barriers to Inclusive Digital Payments in Africa

Published in April 2024 by the European Centre for Development Policy Management (ECDPM), this policy briefing examines the state of interoperability between digital payment systems across Africa and its consequences for financial inclusion. The report focuses on instant payment systems — the infrastructure layer through which mobile money, bank transfers, and digital wallet payments are routed — and documents the extent to which fragmentation between these systems creates barriers for individuals, merchants, and developers [19].

The report's central finding is that, as of June 2023, 32 active domestic and regional instant payment systems existed across Africa, yet only approximately 44% of cross-domain payment systems enable all-to-all connectivity between different mobile money operators and bank accounts. The majority of instant payment systems across the continent remain siloed, with technical barriers including legacy system incompatibility, mismatched data formats, and incompatible security protocols preventing interoperability. Ghana is identified as the only African country with fully interoperable multiple instant payment systems. The report notes that this fragmentation disproportionately affects small-scale traders and economically marginalised populations, with over 400 million adults in Sub-Saharan Africa remaining excluded from formal financial services.

The ECDPM report recommends policy harmonisation, open APIs, and financial incentives for interoperability as the primary instruments for closing these gaps, and frames interoperability as a prerequisite for the success of the African Continental Free Trade Area digital economy.

Relevance to PhoenixPay:

The interoperability gaps documented by ECDPM at the African infrastructure level are directly mirrored at the developer integration level in Sudan, where no unified API layer connects the multiple payment providers operating in the market. PhoenixPay addresses exactly this gap — not by resolving interoperability between payment networks directly, but by providing an abstraction layer above them: a single, documented REST API that gives developers one integration point regardless of which underlying payment provider processes the transaction. The ECDPM report thus provides both the empirical evidence of the problem and the conceptual framing — open APIs as an interoperability mechanism — that underpins PhoenixPay's design.

2.3.11 Ahmad, Green & Jiang (2020): Mobile Money, Financial Inclusion and Development

In [20], a comprehensive literature review of research on mobile money, financial inclusion, and economic development is presented, with a specific focus on the African experience. Published in the *Journal of Economic Surveys* (Volume 34, Issue 4, pp. 753–792, DOI: 10.1111/joes.12372), the paper synthesises a substantial body of empirical work to assess what is known — and what remains contested — about the role of mobile money in driving financial inclusion and broader economic development outcomes in developing economies.

The paper uses Kenya's M-Pesa as the primary reference case, examining how its mechanics, adoption trajectory, and impact on financial inclusion have

been documented in the literature. The authors identify M-Pesa's success as a function not only of its technology but of its ability to provide a simple, accessible, and affordable payment interface in a market where traditional banking infrastructure was limited — a lesson the authors argue is applicable across Sub-Saharan Africa and analogous developing markets. A central research question the paper addresses is whether mobile money *substitutes for* or *complements* conventional financial services: the evidence suggests that in many contexts mobile money creates a parallel informal financial system rather than acting as a gateway into formal banking, a pattern consistent with the Global Findex finding that 92% of Sudan's mobile money users lack a formal account [18].

The paper concludes by identifying four research gaps requiring further investigation: variation in mobile money take-up across demographic groups; the causal mechanisms linking mobile money to financial inclusion; the regulatory frameworks that best support mobile money providers; and the degree of substitutability between mobile money and conventional finance.

Relevance to PhoenixPay:

The analysis of M-Pesa in [20] directly informs the case study presented in section 2.3.4 of this chapter and provides theoretical grounding for PhoenixPay's design philosophy. The finding that successful mobile money platforms achieve adoption by offering a *single, accessible interface* in markets with limited alternatives is the same principle that motivates PhoenixPay's unified API approach. The substitutability findings also inform PhoenixPay's architecture: by operating as an abstraction layer above both mobile wallet providers and bank-based payment channels, PhoenixPay does not require developers to choose between mobile money and formal banking infrastructure — it accommodates both through a single integration point.

2.4 Conclusion

The seven systems reviewed in this chapter collectively demonstrate both the potential and the limits of existing payment integration approaches as solutions for Sudan's payment ecosystem. Global platforms such as Stripe and PayPal set the benchmark for developer experience — offering well-documented REST APIs, sandbox environments, webhook delivery with retry, and secure API key authentication — but are inaccessible in Sudan due to regulatory restrictions and the absence of SDG support. Regional platforms such as Flutterwave and Moyasar demonstrate that the unified payment aggregation model is viable in African and Arabic-speaking markets respectively, and that global API design patterns translate successfully when combined with local payment method support. M-Pesa's Daraja API illustrates how a mobile-first payment platform can achieve transformative financial inclusion at scale in a developing economy, providing a compelling precedent for the Sudanese context. Taken together, these platforms confirm that the design patterns underpinning modern payment integration — RESTful APIs, API key authentication, idempotent payment confirmation, signed webhook callbacks, and internal merchant ledgers — are well-proven and directly applicable to the Sudanese market.

The most directly relevant finding, however, comes from section 2.3.8, which documents the current state of payment integration practice in Sudan. The absence of any publicly accessible, developer-facing payment API has forced Sudanese developers into time-consuming, fragile, and unscalable workarounds — direct provider negotiation and manual receipt verification. These workarounds are not edge cases; they represent the current standard for digital payment processing in the Sudanese market and demonstrate a concrete, measurable gap in the local payment infrastructure. PhoenixPay is designed to fill precisely this gap: by providing a unified, well-documented REST API backed by a local core-banking simulation, it offers Sudanese developers the same

integration experience that Stripe, PayPal, Flutterwave, Moyasar, and M-Pesa provide in their respective markets — built natively for Sudan's currency, providers, and regulatory context.

2.5 Summary of Literature Review

The table below provides a side-by-side comparison of all platforms and approaches reviewed in this chapter, summarising their key characteristics and their relevance to PhoenixPay.

Payment Platforms and Practices

Platform / System	Type	SDG Support	API Style	Webhook Support	Sandbox	Sudan Accessible	Key Lesson for PhoenixPay
Stripe	Global payment gateway	No	REST (OpenAPI)	Yes (with retry)	Yes	No	Gold standard for API design, key model, and developer documentation
PayPal	Global digital wallet + gateway	No	REST (OAuth 2.0)	Yes	Yes	No	Consumer trust model; highlights cost of inconsistent API versioning
Flutterwave	Pan-African payment aggregator	No	REST	Yes	Yes	No	Proves aggregation model works in Africa; gap is Sudan-specific provider support

M-Pesa (Daraja)	Mobile money platform	No	REST (OAuth 2.0)	Callback-based	Yes	No	Demonstrates transformative financial inclusion via mobile-first unified access
Moyasar	MENA payment platform	No	REST (OpenAPI)	Yes	Yes	No	Closest regional analogue; validates bilingual OpenAPI docs and dual-key auth
CashiPay	Sudanese online checkout / payment link API	Yes	REST (coming soon)	No	No	Yes	Licensed Sudanese merchant checkout product; REST API and webhooks not yet publicly available
SyberPay	Sudanese payment gateway (30+ banks)	Yes	REST (undocumented officially)	Partial (community-evidenced)	No	Yes	Most complete local gateway; gap is absent official API docs, sandbox, and webhook spec
Manual workarounds (Sudan)	Ad hoc practice	Yes (SDG)	None	None	None	Yes	Defines the gap: no unified API, no automation, no idempotency, no audit trail

PhoenixPay	Local payment integration platform	Yes	REST (OpenAPI 3.0)	Yes (HMAC + retry)	Yes	Yes	Applies global design patterns natively to Sudan's payment ecosystem
-------------------	------------------------------------	------------	---------------------------	---------------------------	------------	------------	--

Academic Studies

Study	Type	Key Metric / Finding	Relevance to PhoenixPay
Demirgüç-Kunt et al. (2022) — Global Findex 2021	World Bank empirical report	57% digital payment adoption in developing economies; 92% of Sudan mobile money users lack a formal account	Quantifies the adoption gap PhoenixPay targets; confirms mobile-first design is appropriate for Sudan
ECDPM (2024) — Interoperability in Africa	Policy briefing	32 active payment systems in Africa; only 44% enable all-to-all interoperability	Confirms that payment fragmentation is structural; validates PhoenixPay's unified-API abstraction approach
Ahmad, Green & Jiang (2020) — Mobile Money Review	Journal literature review	M-Pesa's success driven by single accessible interface; mobile money often substitutes for rather than complements formal banking	Supports unified API design philosophy; explains why a single integration point drives adoption in developing markets

Chapter 3: System Analysis & Design

3.1 Introduction

This chapter presents the analysis and design of the PhoenixPay system and the supporting components that together form a complete, demonstrable payment ecosystem. The chapter begins with the formal requirements specification, followed by a three-entity conceptual model that frames how the components relate to one another, and then examines each component in depth — covering its purpose, functional capabilities, intended users, non-functional constraints, and the architectural and database design decisions made during development.

The **PhoenixPay platform** is the core deliverable of this project — the unified payment integration API and developer portal that all other components exist to support. The three supporting components are:

1. **Core Banking System — Apache Fineract**, an existing open-source core-banking engine deployed from its official Docker image and configured (not developed) to simulate realistic banking operations. Fineract is used as-is to represent the core bank.
2. **Bank Channel**, a custom-built customer-facing web application developed for this project to simulate the role of a bank's payment interface, giving bank customers access to their Fineract accounts and enabling them to pay PhoenixPay invoices.
3. **TaskMaster**, a custom-built demo task-management SaaS application developed to demonstrate a complete PhoenixPay merchant integration, illustrating the full payment lifecycle from the merchant's perspective.

3.2 Requirements Analysis

This section formally specifies the functional and non-functional requirements for the PhoenixPay platform. These requirements define what the core system must do and the quality attributes it must satisfy. The supporting components (Apache Fineract, Bank Channel, and TaskMaster) are not subject to independent requirements; their design is driven by PhoenixPay's integration needs.

3.2.1 Functional Requirements

The following twenty-four functional requirements were identified for the PhoenixPay platform:

ID	Description	Component
FREQ-1	Developer registration with email validation and secure password (8+ characters, mixed case, digits, and symbols)	PhoenixPay Backend
FREQ-2	Developer authentication via JWT with 8-hour token lifetime and proactive refresh	PhoenixPay Backend + Merchant Dashboard
FREQ-3	Merchant application creation, update, activation, and deactivation with audit logging	PhoenixPay Backend
FREQ-4	API key generation (bcrypt-hashed, 16-character prefix for fast lookup), rotation, and revocation	PhoenixPay Backend
FREQ-5	Per-application webhook configuration: URL, signing secret, and enabled/disabled flag	PhoenixPay Backend
FREQ-6	Invoice creation with positive SDG amount, description, optional merchant reference, and optional expiry date	PhoenixPay Backend
FREQ-7	Invoice listing with status and date-range filters and pagination	PhoenixPay Backend
FREQ-8	Invoice cancellation (pending invoices only) by the issuing merchant application	PhoenixPay Backend

FREQ-9	Automatic invoice expiry: status set to <code>expired</code> when <code>due_date</code> passes, checked lazily on GET and on confirmation	PhoenixPay Backend
FREQ-10	Bank Channel invoice inquiry authenticated by channel shared secret, returning amount and merchant name	PhoenixPay Backend
FREQ-11	Idempotent payment confirmation with a unique database constraint on (<code>invoice_id</code> , <code>bank_reference</code>) to prevent duplicate ledger credits	PhoenixPay Backend
FREQ-12	Atomic ledger credit on payment confirmation: Sequelize transaction wraps invoice status update and balance increment	PhoenixPay Backend
FREQ-13	HMAC-SHA256 signed <code>invoice.paid</code> webhook delivery carrying the full transaction payload	PhoenixPay Backend
FREQ-14	Exponential-backoff webhook retry with delivery attempts at 0 s, 30 s, 5 min, 10 min, and 15 min; marked <code>failed</code> after 5 attempts	PhoenixPay Backend
FREQ-15	Startup reconciliation: stale <code>pending</code> webhook deliveries are re-queued when the server boots	PhoenixPay Backend
FREQ-16	Per-developer balance ledger tracking his <code>available_balance</code>	PhoenixPay Backend
FREQ-17	Payout account registration: store a merchant's Fineract client ID and account ID with a label	PhoenixPay Backend
FREQ-18	Payout initiation with minimum-threshold enforcement and available-balance validation	PhoenixPay Backend
FREQ-19	Payout execution via Bank Channel <code>/api/phoenix/payout</code> → Fineract POST <code>/accounttransfers</code> ; Fineract transfer ID stored on the payout record	Bank Channel + PhoenixPay Backend
FREQ-20	Refund initiation by a merchant application with amount, reason, and available-balance validation	PhoenixPay Backend

FREQ-21	Refund execution via Bank Channel <code>/api/phoenix/refund</code> → Fineract transfer; invoice status updated to <code>refunded</code> or <code>partially_refunded</code>	Bank Channel + PhoenixPay Backend
FREQ-22	Admin system configuration management: <code>payout_min_threshold</code> and <code>invoice_default_expiry_days</code> configurable at runtime	PhoenixPay Backend
FREQ-23	Comprehensive audit trail: every sensitive action recorded with actor email, IP Address, resource type and ID, action name, metadata JSON, and UTC timestamp	PhoenixPay Backend
FREQ-24	Admin audit log query with date-range and action-type filters and pagination	PhoenixPay Backend

3.2.2 Non-Functional Requirements

ID	Requirement	Implementation Approach
NFREQ-1	Multi-layer security controls	JWT authentication, bcrypt-hashed passwords, HMAC-SHA256 webhook signatures with timing-safe verification, channel shared secret authenticated via <code>crypto.timingSafeEqual()</code> , RBAC admin access, and per-endpoint rate limiting
NFREQ-2	Stateless backend design	JWT-based sessions with no server-side session state; all persistent state stored in MariaDB
NFREQ-3	Durable, ACID-compliant financial storage	MariaDB with Sequelize database transactions for all balance updates; <code>DECIMAL(15, 2)</code> for every monetary column; UTC timestamps; Docker volume-mounted database directories
NFREQ-4	Reliable webhook delivery	Five-attempt exponential-backoff retry; startup reconciliation re-queues any deliveries that were still in

		pending status when the server last stopped
NFREQ-5	Idempotent payment confirmation	Unique database constraint on (<code>invoice_id</code> , <code>bank_reference</code>) in the <code>transactions</code> table prevents duplicate ledger credits even under concurrent retries
NFREQ-6	Complete audit trail	Every sensitive and money-related operation is logged to <code>audit_logs</code> with actor email, IP Address, resource type, resource ID, action name, metadata JSON field, and UTC timestamp

3.3 System Overview: The Three-Entity Model

Before examining each component individually, it is useful to understand the overall system through the lens of the three actors it serves: the **Bank**, **PhoenixPay**, and the **Merchant**. Each actor is associated with one or two system components, and the interactions between them define the end-to-end payment flow.

Bank — The bank owns and operates the financial infrastructure. In this project the bank is represented by two components: the Core Banking System (Apache Fineract), which holds client accounts and records all financial transactions, and the Bank Channel, which is the customer-facing web portal through which bank customers view their accounts, make peer-to-peer transfers, and—crucially—pay invoices issued by merchants through PhoenixPay.

PhoenixPay — PhoenixPay is the payment intermediary. It accepts invoice creation requests from merchants, manages payment confirmations received from the Bank Channel, maintains a balance ledger for each merchant, and enables payouts from that balance by routing requests through the Bank Channel (which executes the actual Fineract transfers). PhoenixPay never

communicates with Fineract directly — all banking operations are mediated by the Bank Channel. PhoenixPay is implemented as two sub-components: the PhoenixPay Backend (REST API) and the Merchant Dashboard (developer portal SPA).

Merchant — The merchant is a business that uses PhoenixPay to collect payments from customers. In this project, TaskMaster serves as a demonstration merchant — a task-management SaaS that integrates PhoenixPay for premium subscriptions, exercising every major merchant-facing PhoenixPay feature end-to-end.

The diagram below shows which components belong to which entity and the communication links between them.

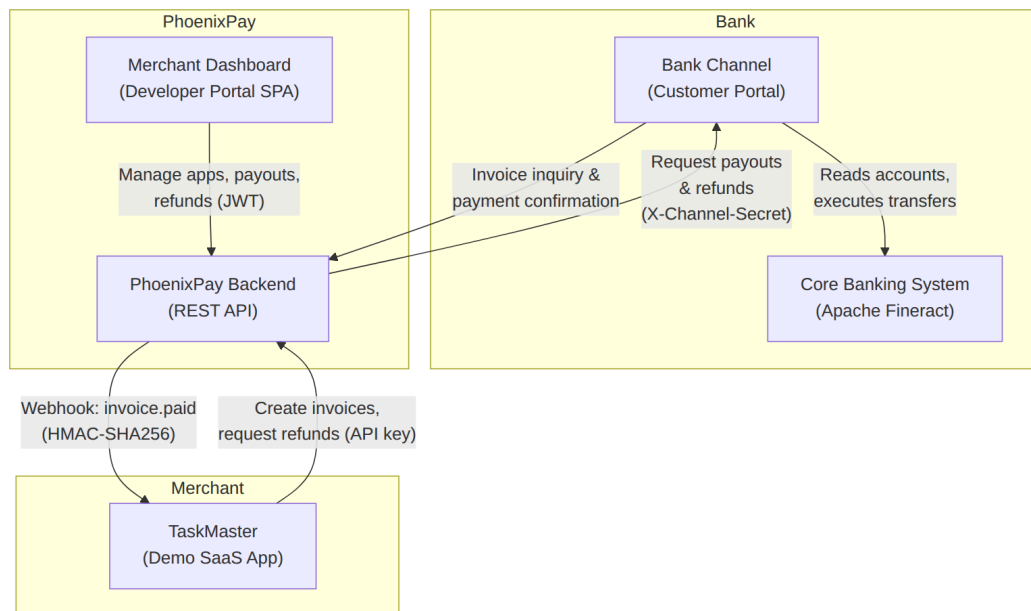


Figure 3.1 — Entity-Component Model

A payment begins when a merchant application creates an invoice on PhoenixPay using its API key and receives an invoice ID in return. The merchant presents that ID to the payer, who navigates to the Bank Channel to complete the payment. The Bank Channel first inquires the invoice details from PhoenixPay

(authenticated via a shared channel secret), then executes a Fineract inter-account transfer moving funds from the payer's account into PhoenixPay's settlement account, and finally sends a payment confirmation back to PhoenixPay. Upon receiving that confirmation, PhoenixPay credits the merchant's internal balance and fires a webhook to the merchant application. When the merchant later requests a payout through the Merchant Dashboard, PhoenixPay forwards the instruction to the Bank Channel, which executes a second Fineract transfer from the settlement account to the merchant's registered bank account. Throughout this entire flow, PhoenixPay never communicates with Apache Fineract directly — all fund movements are mediated by the Bank Channel, which is the only component that holds Fineract credentials.

3.4 Component 1: Core Banking System — Apache Fineract

3.4.1 Overview

Apache Fineract is an open-source, production-grade core-banking platform maintained by the Apache Software Foundation. It was originally developed by MifosX and donated to Apache in 2016, and is widely used in microfinance institutions across the developing world. Fineract provides a comprehensive REST API for managing financial entities — clients, savings accounts, loans, deposits, and inter-account transfers — making it suitable as the banking backbone for this project.

In the context of this graduation project, Apache Fineract serves as the authoritative ledger of record. Every SDG balance, every fund transfer, and every account statement ultimately resides in Fineract. Only the Bank Channel communicates with Fineract directly — PhoenixPay routes all its banking operations (payouts, refunds, account lookups) through the Bank Channel, which

acts as a secure intermediary. This design ensures that core banking credentials remain exclusively within the bank's domain.

Fineract is deployed as a Spring Boot application in a Docker container. It connects to a shared MariaDB 11.4 database. A companion Angular web interface (MifosX Community App) is deployed alongside it, providing bank administrators with a graphical tool for client and account setup, transaction monitoring, and account management.

3.4.2 Features and Functions

The following features of Apache Fineract are actively used by this project:

Feature	Fineract API Endpoint	Description
Client registry	GET /clients/:id	Retrieve a client's profile including office ID and display name
Account listing	GET /clients/:id/accounts	List all savings accounts belonging to a client
Account search	GET /search?query=...&resource=savingsaccount	Find a savings account by account number
Balance and statement	GET /savingsaccounts/:id?associations=transactions	Retrieve account balance and full transaction history
Fund transfer	POST /accounttransfers	Atomically transfer funds from one savings account to another
Multi-tenancy	Request header Fineract-Platform-TenantId: default	All API calls are scoped to the configured tenant

Every bank customer in the system has a corresponding Fineract client entity with one or more savings accounts denominated in SDG. PhoenixPay itself is also a Fineract client (Client ID 1), holding the settlement account into which all incoming invoice payments flow before being disbursed as payouts.

The MifosX web interface (proxied at `/core-bank/` by Nginx) is the bank administrator's tool for registering new clients, opening savings accounts, and monitoring the ledger without writing code.

3.4.3 Users

User	Type	How They Interact
Bank Administrators	Human	MifosX web UI — create clients, open accounts, monitor transactions
Bank Channel (system)	Service	Fineract REST API — read account data, execute transfers on behalf of bank customers and PhoenixPay

Note: PhoenixPay does **not** access Fineract directly. All PhoenixPay banking operations (payouts, refunds, account lookups) are routed through the Bank Channel, which acts as a secure intermediary. This ensures that Fineract credentials remain exclusively within the bank's domain.

3.4.4 Non-functional Requirements and Constraints

- **Startup time:** Fineract takes approximately 300 seconds to complete its Spring Boot context initialisation and database migration sequence.

Docker Compose applies a TCP-level health check (`nc -z localhost 8080`) with a 300-second start period, and the Bank Channel service is gated behind this check via `condition: service_healthy`.

PhoenixPay does not depend on Fineract directly — it communicates with the Bank Channel, which in turn depends on Fineract.

- **Authentication:** All calls to the Fineract REST API use HTTP Basic Authentication with the credential pair `mifos:password`.

- **Network:** In the development and demonstration environment, Fineract runs with SSL disabled (`FINERACT_SERVER_SSL_ENABLED=false`) on

port 8080, communicating over plain HTTP within the Docker network. A production deployment would require TLS termination and CA-signed certificates.

- **Scalability:** The deployment is single-node and not horizontally scaled. This is appropriate for a proof-of-concept demonstration; a production system would require a load-balanced, database-replicated Fineract cluster.
- **Persistence:** The MariaDB data directory is Docker volume-mounted so that all account balances and transfer records survive container restarts.
- **Currency:** Only SDG (Sudanese Pound) savings accounts are used in this project. All amounts are stored as `DECIMAL(15, 2)`.

3.4.5 Design

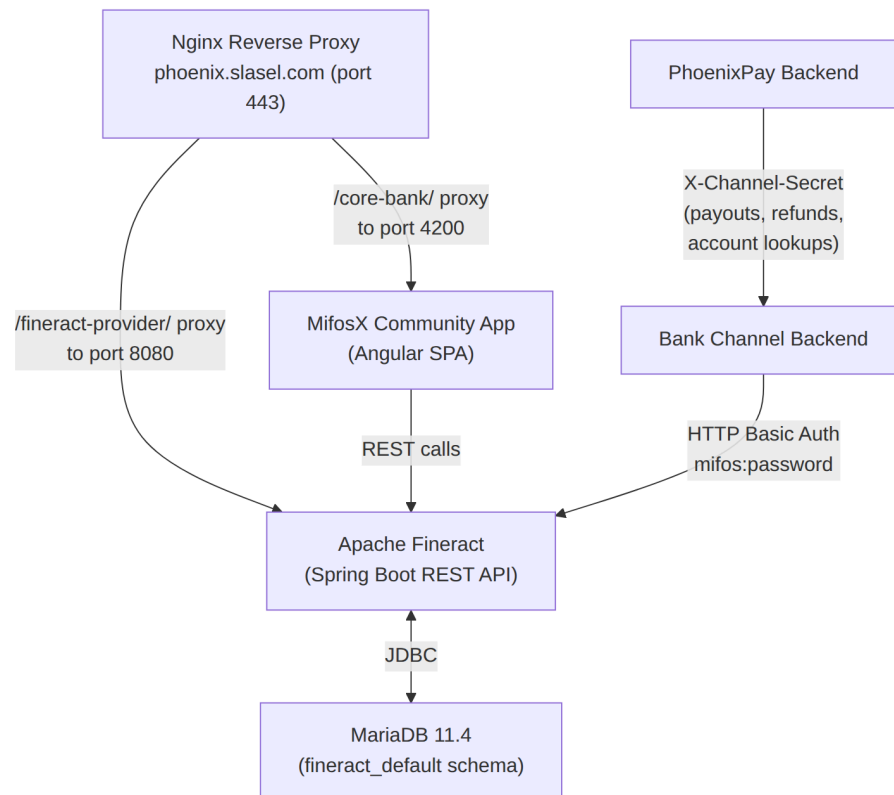


Figure 3.2 — Fineract Deployment Architecture

The `POST /accounttransfers` endpoint is the most critical Fineract call in the system. Its request body specifies the source and destination by (officeId, clientId, accountId) triplets along with the transfer amount in SDG and a date formatted as "dd MMMM yyyy". Fineract returns a `resourceId` identifying the transfer, which is stored by the calling system for reconciliation purposes.

Key Fineract API calls used in this project:

Method	Endpoint	Called By	Purpose
GET	/clients/:id	Bank Channel	Fetch client profile (name, officeId)
GET	/clients/:id/accounts	Bank Channel	List client savings accounts
GET	/savingsaccounts/:id?associations=transactions	Bank Channel	Account balance and statement
GET	/search?query=:accNo&resource=savingsaccount	Bank Channel	Find account by account number
POST	/accounttransfers	Bank Channel	Execute inter-account fund transfer

3.5 Component 2: Bank Channel

3.5.1 Overview

The Bank Channel is a custom-built web application that provides bank customers with a self-service interface to their Fineract accounts. It consists of a Vue 3 single-page application (frontend) and a Node.js/Express backend API. Both components are deployed in Docker containers and served through Nginx.

The Bank Channel occupies a critical position in the overall payment flow: it is the **only** component that communicates with the core banking system (Apache Fineract). All fund movements — whether initiated by a bank customer (bill payments, peer-to-peer transfers) or by PhoenixPay (payouts, refunds) — are

executed through the Bank Channel. When a bank customer pays a PhoenixPay invoice, the Channel executes a Fineract inter-account transfer and then notifies PhoenixPay. When PhoenixPay requests a payout or refund, the Channel receives the request (authenticated via `X-Channel-Secret`), executes the corresponding Fineract transfer, and returns the transfer ID. This design ensures that Fineract credentials remain exclusively within the bank's domain and that PhoenixPay never has direct access to the core banking API.

3.5.2 Features and Functions

Feature	Description
Login and session management	JWT-based authentication: bcrypt password verification, 8-hour access token, token refresh endpoint In the context of this graduation project, Apache Fineract serves as the authoritative ledger of record
Account dashboard	View all savings account balances, sourced from Fineract via API proxy
Transaction history	Per-account statement lines, sourced from Fineract
Peer-to-peer transfer	Look up a recipient by account number; execute a Fineract inter-account transfer
Invoice bill payment	Enter a PhoenixPay invoice ID; retrieve invoice details; execute Fineract payment; confirm to PhoenixPay
PhoenixPay payout/refund proxy	Receive payout and refund requests from PhoenixPay (authenticated via <code>X-Channel-Secret</code>); execute the corresponding Fineract transfers and return transfer IDs
PhoenixPay account lookup proxy	Serve Fineract client and account data to PhoenixPay for payout account registration
Local audit trail	Every TRANSFER and BILL_PAYMENT is recorded in the Bank Channel's local <code>transactions</code> table

3.5.3 Users

User	Type	Interaction Mode
Bank Customers	Human	Vue 3 SPA — view accounts, transfer funds, pay invoices
PhoenixPay Backend (system)	Service	REST API — request payouts, refunds, and account lookups (authenticated via <code>X-Channel-Secret</code>)

Bank customers are pre-registered in the system: their login credentials exist in the Bank Channel's `users` table, and their financial accounts exist in Fineract. The `fineract_client_id` column in the `users` table bridges the two systems. The seeded test users are **PhoenixPay** (Fineract Client ID 1 — the settlement account holder), **TaskMaster** (Client ID 2), and **Mohamed** (Client ID 3).

3.5.4 Non-functional Requirements and Constraints

- **CORS:** The Bank Channel API accepts requests from any origin. This is acceptable for a demonstration environment.
- **Rate limiting:** No rate limiting is implemented. In a production deployment, rate limiting on the bill-payment endpoint would be essential to prevent abuse.
- **Idempotent payment confirmation:** After a successful Fineract transfer, the Bank Channel retries the payment confirmation call to PhoenixPay up to three times with exponential backoff. It distinguishes 4xx responses (definitive rejection — stop retrying) from 5xx responses (transient failure — retry). This ensures the merchant's PhoenixPay balance is credited even if the confirmation endpoint is temporarily unavailable after the Fineract transfer succeeds.

- **Deployment:** The backend runs in a `node:20-alpine` Docker container on port 3333, exposed only to localhost. Docker Compose starts it only after both Fineract (`service_healthy`) and PhoenixPay Backend (`service_started`) are ready.

3.5.5 Design

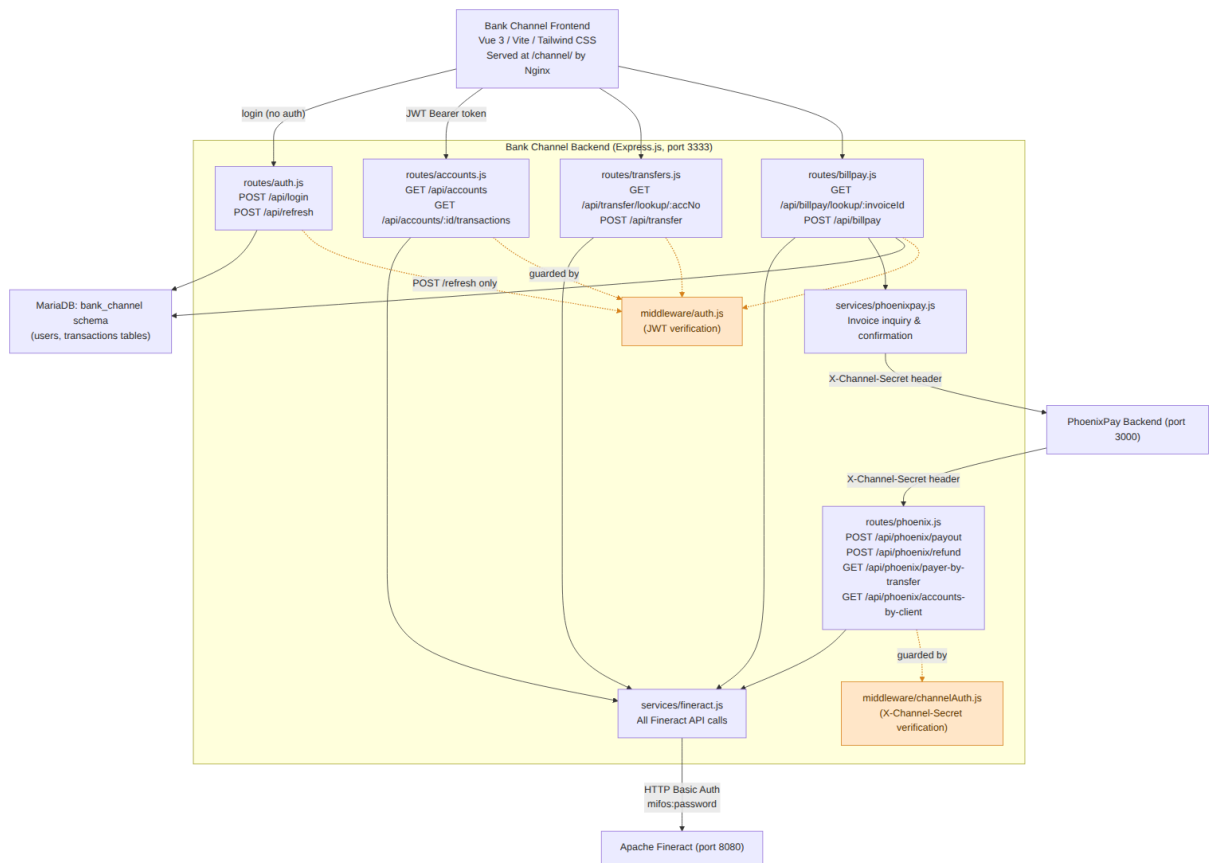


Figure 3.3 — Bank Channel Component Architecture

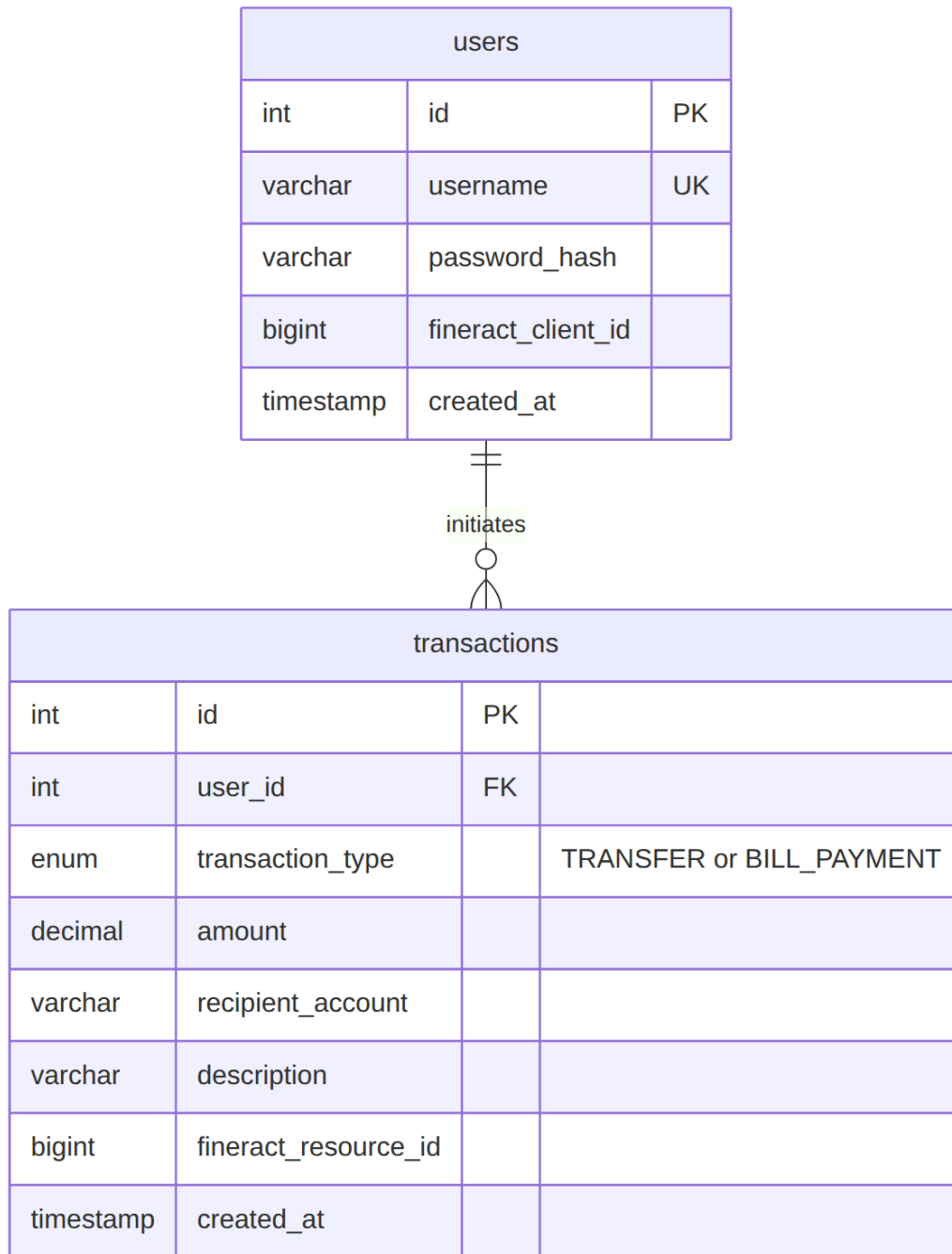


Figure 3.4 — Bank Channel Entity-Relationship Diagram

The schema is intentionally minimal: account balances and transaction details are the authoritative responsibility of Fineract; the Bank Channel only keeps a local record of operations it has initiated, indexed by the Fineract `resourceId` for reconciliation.

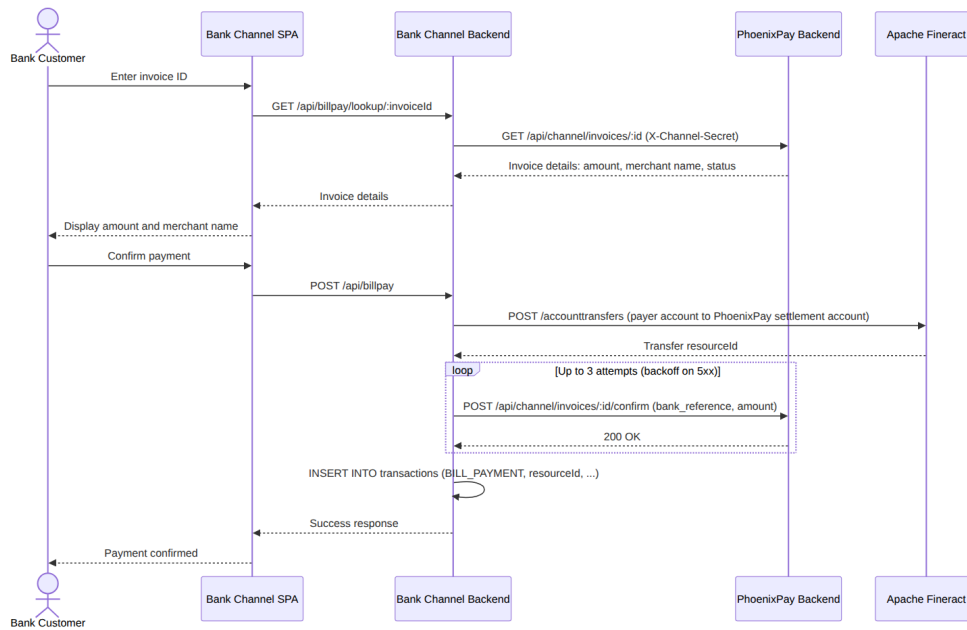


Figure 3.5 — Bill-Payment Sequence Diagram

The most important design detail in this sequence is the ordering: the Fineract transfer is committed before the confirmation call to PhoenixPay. If the confirmation call fails after a successful transfer, the retry loop ensures PhoenixPay is eventually notified. PhoenixPay's idempotency constraint (unique index on `(invoice_id, bank_reference)`) ensures that even if the Bank Channel sends the confirmation more than once, the merchant's balance is only credited once.

PhoenixPay-Facing API (`routes/phoenix.js`):

In addition to serving bank customers directly, the Bank Channel exposes a set of API endpoints that PhoenixPay calls to execute banking operations. This design ensures that PhoenixPay never holds Fineract credentials or communicates with the core banking system directly — all fund movements are mediated by the Bank Channel.

Method	Endpoint	Purpose
POST	/api/phoenix/payout	Execute a payout transfer from the PhoenixPay settlement account to a merchant's bank account
POST	/api/phoenix/refund	Execute a refund transfer (settlement account self-transfer for audit trail)
GET	/api/phoenix/accounts-by-client?client_id=X	Look up a Fineract client's profile and savings accounts

These endpoints are authenticated using the same bidirectional shared secret (`CHANNEL_API_SECRET`) that the Bank Channel uses when calling PhoenixPay for invoice inquiry and payment confirmation. PhoenixPay sends the secret in the `X-Channel-Secret` header, and the Bank Channel validates it with `crypto.timingSafeEqual()` via dedicated `channelAuth` middleware. This symmetric authentication model is appropriate because both services are internal to the platform operator.

The payout endpoint receives the merchant's Fineract client and account IDs from PhoenixPay (which stores them on payout account records), resolves the PhoenixPay settlement account details from its own environment configuration (`PHOENIX_FINERACT_CLIENT_ID`, `PHOENIX_FINERACT_ACCOUNT_ID`), and executes the inter-account transfer via Fineract. The transfer's `resourceId` is returned to PhoenixPay, which stores it for reconciliation.

3.5.6 Use Cases

The Bank Channel has two actors: the **Bank Customer**, a human user who interacts through the Vue 3 SPA, and the **PhoenixPay Backend**, a system actor that calls the `routes/phoenix.js` endpoints to execute fund movements on behalf of PhoenixPay.

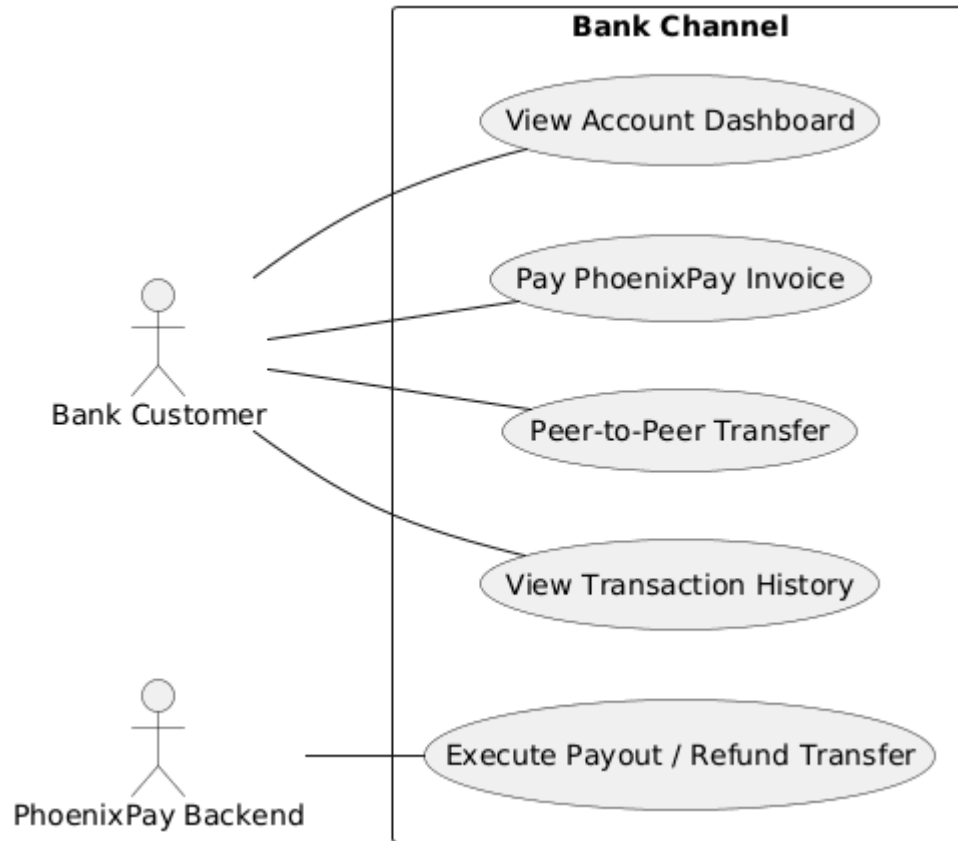


Figure 3.6 — Bank Channel Use Case Diagram

ID	Use Case	Actor	Preconditions	Main Flow	Postconditions
BC-01	View Account Dashboard	Bank Customer	User is logged in	SPA requests account list from backend; backend calls Fineract GET /clients/:id/accounts; account balances displayed in dashboard	Account balances visible
BC-02	View Transaction History	Bank Customer	User has selected an account	Backend calls Fineract GET /savingsaccounts/:id?associations=transactions; statement lines displayed	Transaction history visible

				chronologically	
BC-03	Peer-to-Peer Transfer	Bank Customer	User logged in; recipient account exists in Fineract	User enters recipient account number; backend resolves account via Fineract search; user confirms amount; backend posts POST /accounttransfers; local transactions row inserted	Funds transferred; audit row created in local DB
BC-04	Pay PhoenixPay Invoice	Bank Customer	User logged in; invoice is in pending status	User enters invoice ID; backend fetches invoice details from PhoenixPay via channel secret; user confirms; backend executes Fineract transfer then notifies PhoenixPay (up to 3 retries)	Invoice marked paid; merchant balance credited; local audit row created
BC-05	Execute Payout / Refund Transfer	PhoenixPay Backend	Valid X-Channel-Secret header; settlement account has sufficient funds	PhoenixPay POSTs to /api/phoenix/payout or /api/phoenix/refund; Bank Channel calls Fineract POST /accounttransfers; returns fineract_transfer_id to PhoenixPay	Funds transferred; transfer ID returned to PhoenixPay for reconciliation

3.6 Component 3: PhoenixPay System

3.6.1 Overview

PhoenixPay is the payment-processing platform at the centre of this project — the core deliverable of this graduation project. It is composed of two sub-components that together form one integrated product:

- **PhoenixPay Backend** — a Node.js/Express REST API implementing all payment logic: invoice management, payment confirmation, balance ledger, payout and refund processing (via the Bank Channel), HMAC-signed webhook delivery, and a full audit trail.
- **Merchant Dashboard** — a Vue 3 single-page application (developer portal) that provides merchant developers with a graphical interface for every backend capability.

PhoenixPay serves three distinct client types, each authenticated by a different mechanism: the Merchant Dashboard authenticates with JWTs, merchant applications (such as TaskMaster) authenticate with API keys, and the Bank Channel authenticates with a shared channel secret.

3.6.2 Features and Functions

PhoenixPay Backend:

Feature	Description
Developer authentication	Registration, login, proactive JWT refresh (8-hour tokens, refresh before expiry)
Application management	Create, activate, and deactivate merchant applications; configure webhook URL, secret, and enabled flag
API key management	Generate keys (shown once; stored as bcrypt hash with 16-char prefix), rotate, revoke
Invoice lifecycle	Create, list with status/date filters, cancel, and auto-expire on <code>due_date</code>

Payment confirmation	Idempotent endpoint for Bank Channel — credits merchant balance atomically inside a Sequelize transaction
Balance ledger	Tracks <code>available_balance</code> per merchant developer
Payouts	Initiate payout via the Bank Channel, which executes a Fineract <code>POST /accounttransfers</code> from the settlement account to the merchant's registered account
Refunds	Initiate refund via the Bank Channel, which executes a Fineract transfer; update invoice status to <code>refunded</code> or <code>partially_refunded</code>
Webhook delivery	HMAC-SHA256 signed <code>invoice.paid</code> events with exponential-backoff retry (up to 5 attempts) and startup reconciliation
Audit trail	Every sensitive action logged with actor identity, IP Address, resource type and ID, and metadata
Admin capabilities	System config management, paginated audit log query, platform statistics
API documentation	OpenAPI 3.0 specification served interactively at <code>/api-docs</code> via Swagger UI

Merchant Dashboard:

Feature	Description
Authentication	Login, register, silent JWT refresh (Axios 401 interceptor retries the original request after obtaining a new token)
Application management	CRUD, webhook configuration, API key display (shown once) / rotation / revocation
Invoice monitoring	Paginated list with status and date-range filters
Balance view	Available balances; payout eligibility indicator
Payout account management	Register Fineract client/account IDs as named payout destinations
Payouts	Request payout, track status with Fineract transfer ID
Refund history	List refunds with Fineract transfer ID and current status
Webhook delivery log	Per-application delivery attempts with HTTP response status and truncated response body
Admin views	System configuration editor, audit log browser with date and action filters
Localisation	Full English / Arabic (RTL) support via <code>vue-i18n</code>

3.6.3 Users

User	Type	Authentication	Accessible Scope
Merchant Developers	Human	JWT (x-auth-token header)	Own applications, invoices, balance, payouts, refunds
Bank Channel (system)	Service	Shared channel secret (X-Channel-Secret header)	Channel invoice inquiry and payment confirmation only
Admin	Human	JWT + admin role check	System config, full audit log, platform statistics

3.6.4 Non-functional Requirements and Constraints

Requirement	Implementation Detail
Idempotency	Unique DB constraint on (invoice_id, bank_reference) in the transactions table prevents double-credit on retried confirmations; duplicate requests return the cached response
ACID transactions	Sequelize database transactions wrap payment confirmation + balance increment, and payout/refund + balance decrement, atomically
Rate limiting	Login: 10 req/15 min; invoice creation: 100 req/15 min; payment confirmation: 30 req/15 min; payouts: 20 req/15 min; refunds: 20 req/15 min
Currency	SDG only — enforced at the express-validator middleware layer before any business logic executes
Monetary precision	DECIMAL(15, 2) for all monetary columns; no floating-point arithmetic
Webhook reliability	Exponential-backoff retry with delays 0 s, 30 s, 5 min, 10 min, 15 min; server startup reconciliation re-queues any deliveries that were pending when the server last stopped
Fail-closed security	Channel auth middleware returns HTTP 500 if CHANNEL_API_SECRET is not configured, preventing accidental deployment without security credentials

3.6.5 Design

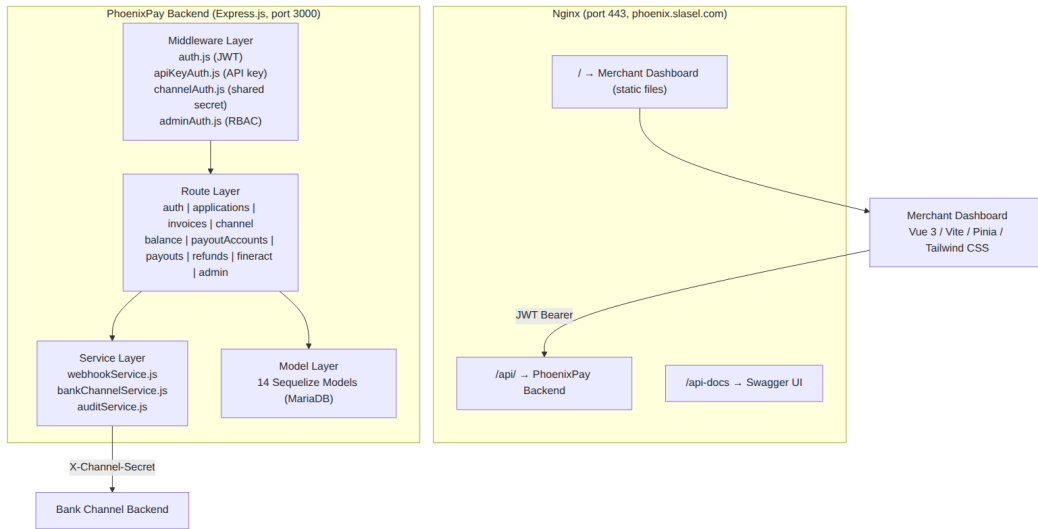


Figure 3.7 — PhoenixPay Layered Architecture

Authentication Design:

Layer	Request Header	Used By	Verification Mechanism
JWT	<code>x-auth-token</code>	Merchant Dashboard	HMAC-SHA256 signed token, 8 h TTL, proactive refresh
API Key	<code>Authorization: Bearer <key></code>	Merchant Apps (e.g., TaskMaster)	Extract 16-char prefix → DB lookup → <code>bcrypt.compare()</code> full key against stored hash
Channel Secret	<code>X-Channel-Secret</code>	Bank Channel	<code>crypto.timingSafeEqual()</code> against <code>CHANNEL_API_SECRET</code> env var; fails closed (HTTP 500) if unconfigured
Admin RBAC	JWT + role check	Admin users	DB query on <code>user_roles</code> table after JWT verification; returns 403 if role is absent

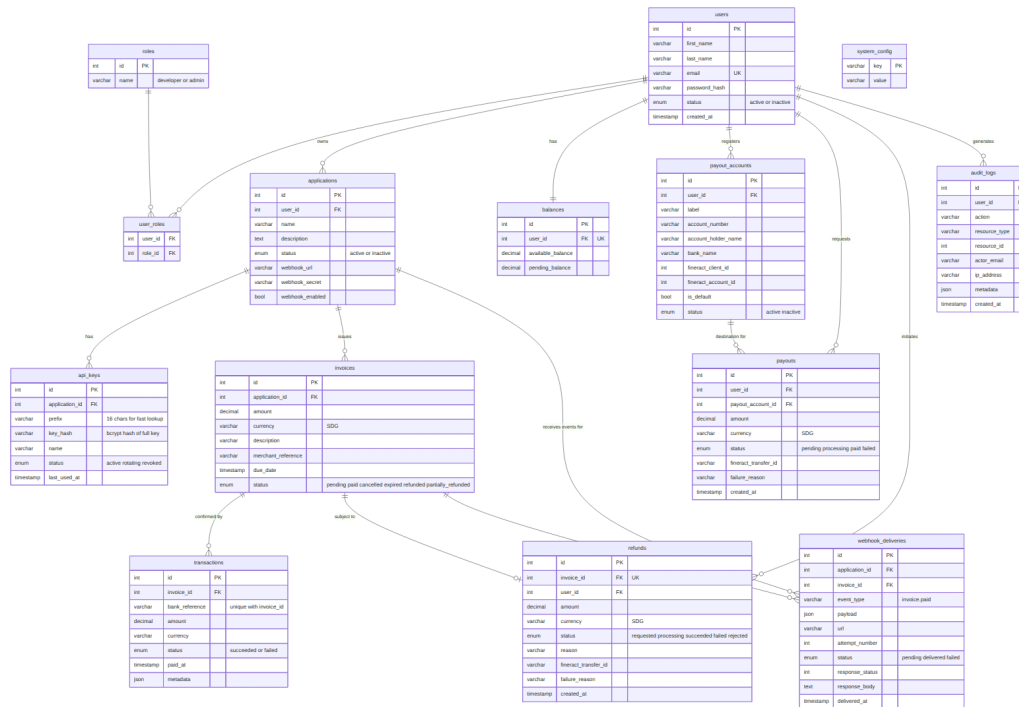


Figure 3.8 — PhoenixPay Data Model

All monetary columns (amount, available_balance, pending_balance) use DECIMAL(15,2) to eliminate floating-point rounding errors. Timestamps are stored in UTC.

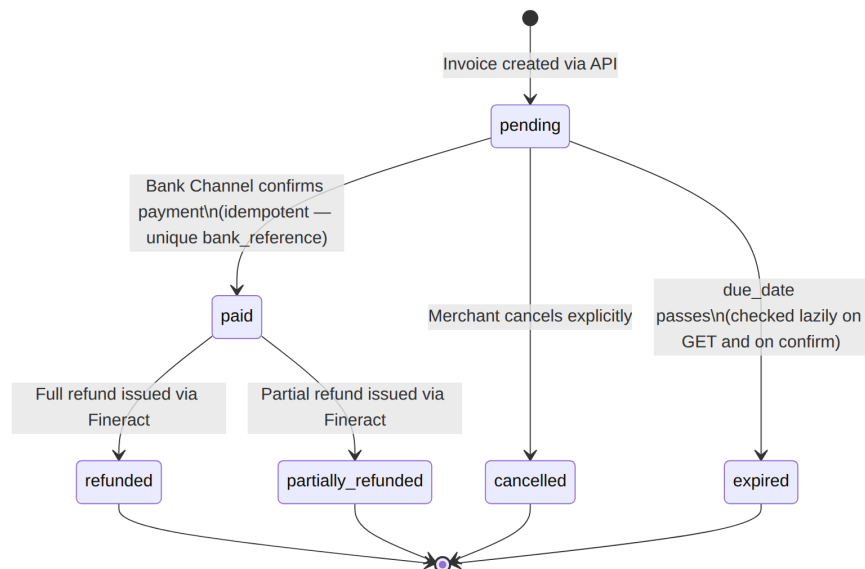


Figure 3.9 — Invoice State Machine

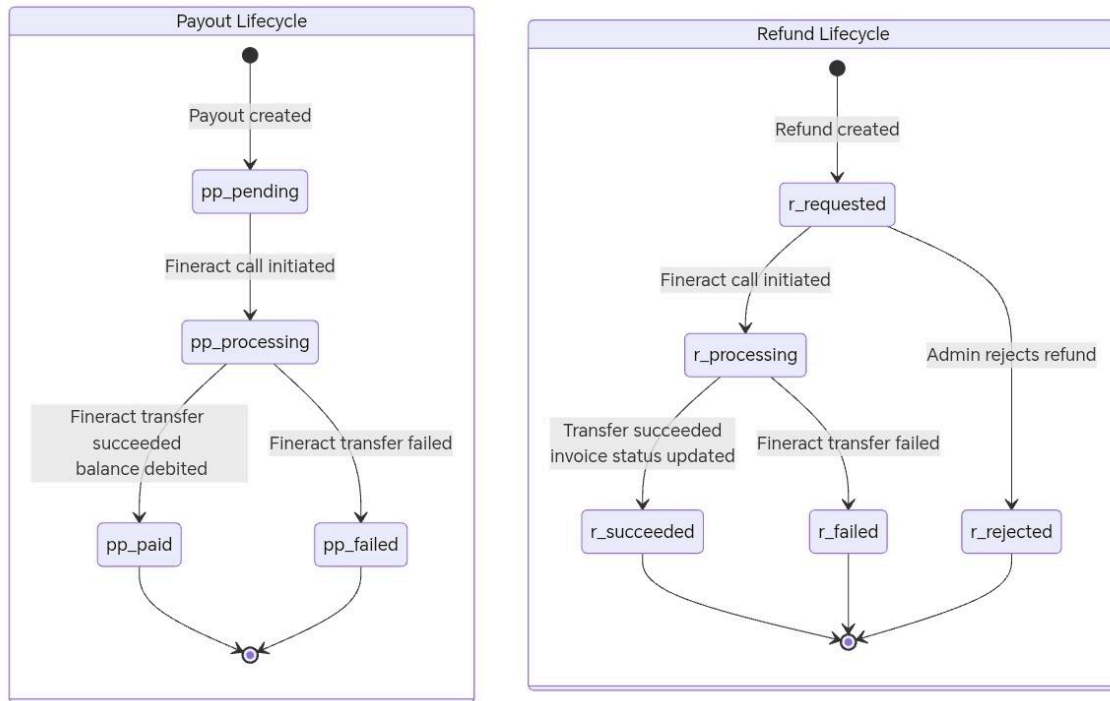


Figure 3.10 — Payout and Refund State Machines

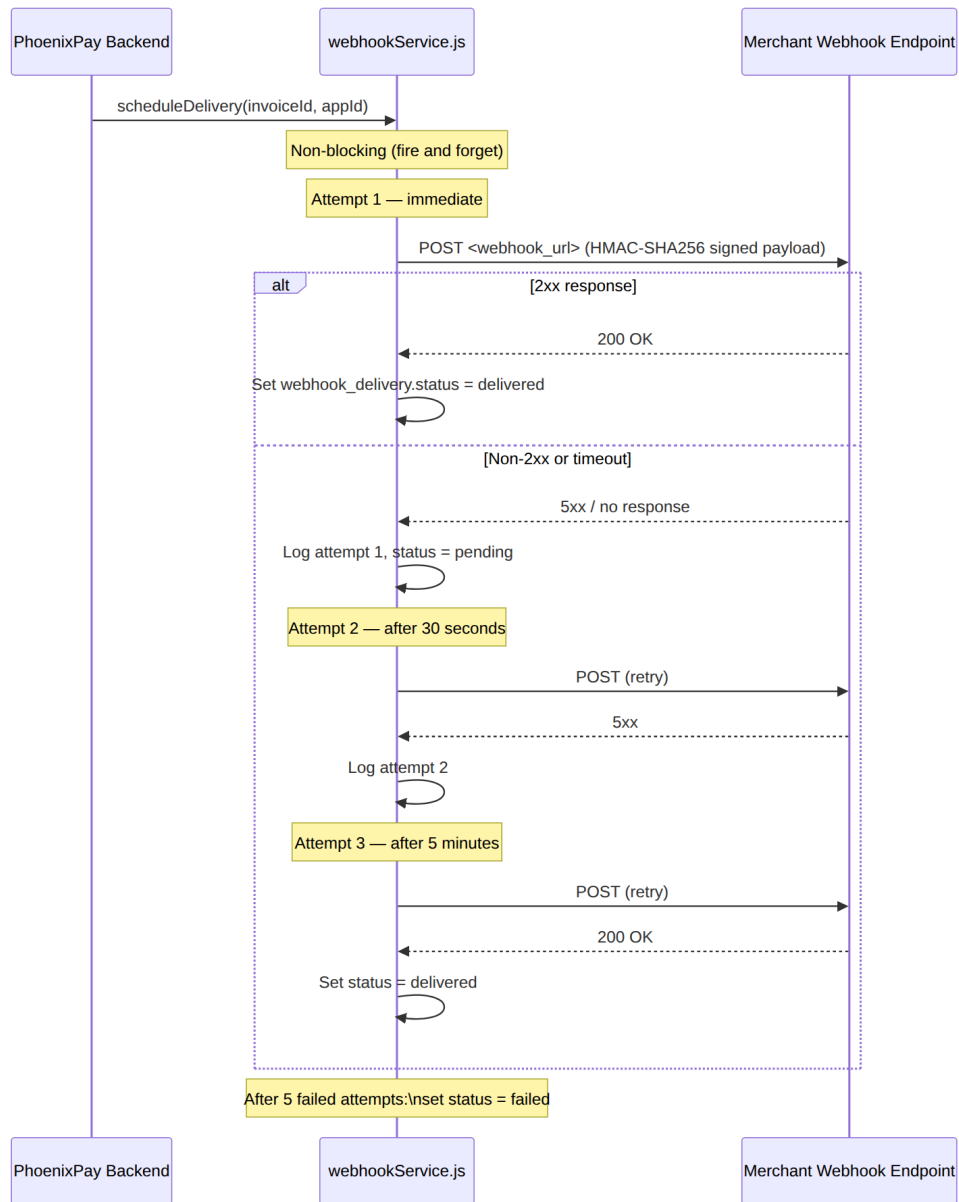


Figure 3.11 — Webhook Delivery with Exponential Backoff

The webhook payload carries the `invoice_id`, `amount`, `currency`, `merchant_reference`, `transaction_reference`, and `paid_at` timestamp. The `X-PhoenixPay-Signature` header contains `sha256=<HMAC-SHA256 hex>` computed over the raw JSON body using the application's `webhook_secret`. Merchants verify this signature using `crypto.timingSafeEqual()` before trusting the payload.

Key API Endpoints:

Auth	Method	Path	Purpose
Public	POST	/api/auth/register	Developer registration
Public	POST	/api/auth/login	JWT issuance
JWT	POST	/api/auth/refresh	Proactive token refresh
JWT	GET/POST	/api/applications	List / create applications
JWT	GET/PUT	/api/applications/:id	View / update application
JWT	POST	/api/applications/:id/key/reset	Rotate API key
JWT	GET	/api/applications/:appId/webhook-deliveries	Webhook delivery log
API Key	POST	/api/invoices	Create invoice
API Key	GET	/api/invoices	List invoices
API Key	POST	/api/invoices/:id/cancel	Cancel invoice
Channel Secret	GET	/api/channel/invoices/:id	Invoice inquiry by Bank Channel
Channel Secret	POST	/api/channel/invoices/:id/confirm	Idempotent payment confirmation
JWT	GET	/api/balance	Available balance
JWT	GET/POST	/api/payout-accounts	List / register payout accounts
JWT	GET/POST	/api/payouts	List / request payouts
API Key	POST	/api/refunds	Initiate refund
JWT	GET	/api/refunds	List refunds
Admin JWT	GET/PUT	/api/admin/config	System configuration
JWT	GET	/api/fineract/accounts-by-client	Look up Fineract client accounts for payout registration
Admin JWT	GET	/api/admin/audit-logs	Paginated audit log
Public	GET	/api-docs	Interactive Swagger UI

3.6.6 Use Cases

PhoenixPay serves four actors: **Merchant Developer** (human user of the dashboard, authenticated by JWT), **Merchant App** (system client such as

TaskMaster, authenticated by API key), **Bank Channel** (system actor using the shared channel secret), and **Admin** (human user with the `admin` role).

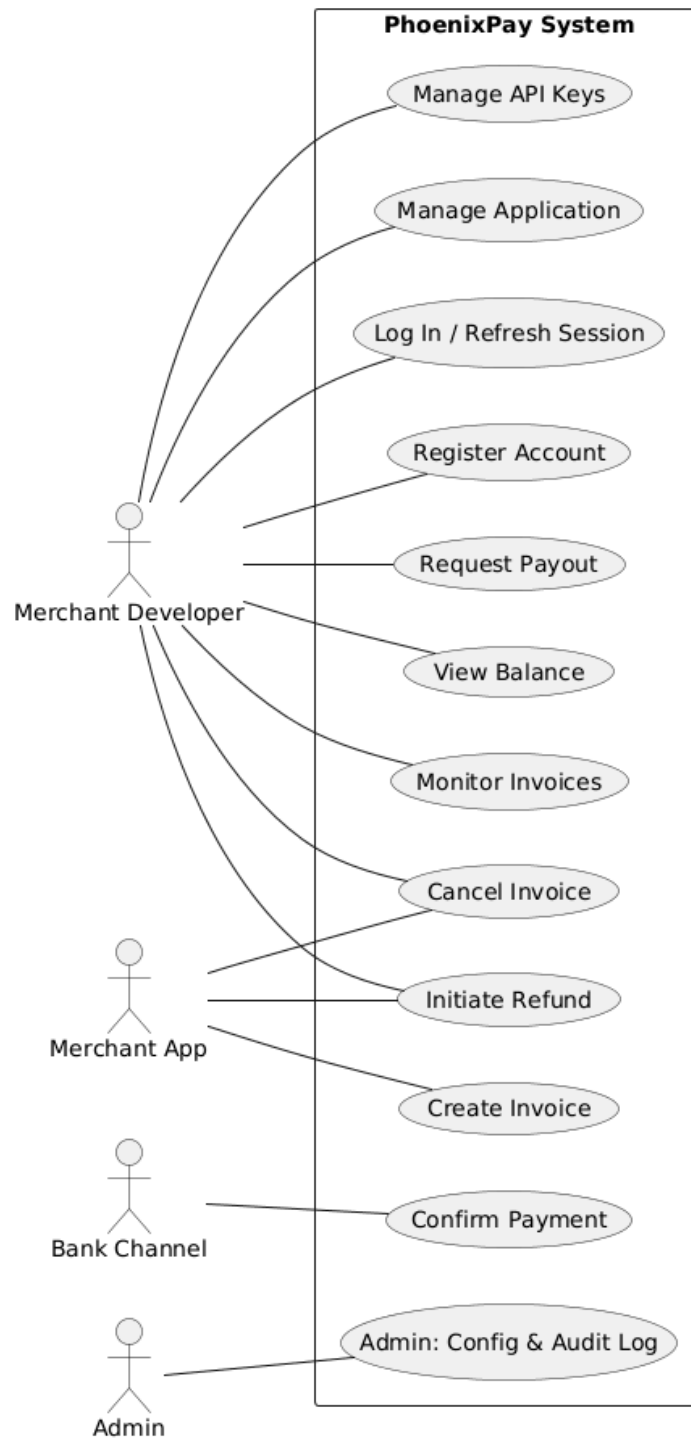


Figure 3.12 — PhoenixPay System Use Case Diagram

ID	Use Case	Actor	Preconditions	Main Flow	Postconditions
PP-01	Register Developer Account	Merchant Developer	Email not already registered	Developer POSTs email and password to /api/auth/register; password bcrypt-hashed; developer role assigned; JWT issued; balance row seeded	Account active; developer can create applications
PP-02	Log In / Refresh Session	Merchant Developer	Account exists	Developer POSTs credentials; JWT returned; Merchant Dashboard stores token and schedules proactive refresh via the Axios 401 interceptor	Active JWT session; subsequent requests carry x-auth-token header
PP-03	Create & Manage Application	Merchant Developer	Developer logged in	Developer creates application with name and optional webhook URL / secret / enabled flag; status defaults to inactive; developer activates or deactivates as needed	Application record persisted; audit log entry created
PP-04	Generate / Rotate / Revoke API Key	Merchant Developer	Application exists	Developer requests key generation; full key displayed once and stored as bcrypt hash with 16-character prefix; rotation revokes the active key and	API key active; 16-char prefix stored for fast lookup during authentication

				issues a replacement	
PP-05	Create Invoice	Merchant App	Valid API key; currency is SDG	Merchant App POSTs amount, currency, description, and optional due_date; server applies configured default expiry if omitted; invoice persisted with status pending	Invoice ID returned to caller; invoice immediately available for payment
PP-06	Monitor & Filter Invoices	Merchant Developer	Developer logged in	Dashboard GETs /api/invoices with optional status, date-range, and pagination parameters; filtered list returned	Invoice list reflects current status across all applications
PP-07	Cancel Invoice	Merchant App / Merchant Developer	Invoice in pending status	Caller POSTs to /api/invoices/:id/cancel; invoice status set to cancelled; action recorded in audit log	Invoice permanently non-payable
PP-08	Confirm Payment (Idempotent)	Bank Channel	Invoice pending; valid channel secret; bank_reference not already recorded	Bank Channel POSTs bank_reference and amount; server opens a Sequelize transaction: inserts transactions row, increments balances.available_balance, sets invoice to paid; webhook	Balance credited atomically; invoice paid; idempotency guaranteed by unique constraint on (invoice_id, bank_reference)

				delivery queued	
PP-09	View Balance	Merchant Developer	Developer logged in	Dashboard GETs /api/balance; server returns available_balance and payout eligibility flag	Current balance visible in dashboard
PP-10	Request Payout	Merchant Developer	Payout account registered; available balance $\geq$ configured minimum threshold	Developer submits payout amount; server calls Bank Channel /api/phoenix/payout; on success, balance decremented and payout record updated with Fineract transfer ID	Funds disbursed to merchant's registered Fineract account; payout record created
PP-11	Initiate Refund	Merchant App / Merchant Developer	Invoice in paid status; available balance covers refund amount	Caller POSTs refund with amount and reason; server calls Bank Channel /api/phoenix/refund; invoice status updated to refunded or partially_refunded; action audit-logged	Refund transfer completed; invoice status updated; balance adjusted
PP-12	Admin: Config & Audit Log	Admin	User holds admin role	Admin reads or writes system_configs keys (payout_min_threshold, invoice_default_expiry_days); queries paginated audit log filtered by date	Platform parameters updated; audit history reviewed without exposing raw DB

				range and action type	
--	--	--	--	--------------------------	--

3.7 Component 4: TaskMaster — Demo Merchant Application

3.7.1 Overview

TaskMaster is a task-management SaaS application built to demonstrate a complete, real-world PhoenixPay merchant integration. Its existence answers the question: "If I were a merchant using PhoenixPay, what would my integration look like?" TaskMaster offers free task management and a paid premium tier. The premium subscription is handled entirely through PhoenixPay: a PhoenixPay application is created for TaskMaster, an API key is issued, and the backend uses that key to create invoices, receive webhook notifications, and issue refunds.

From the end user's perspective, TaskMaster is a simple productivity app with a paywall. From an integration-engineering perspective, it exercises every major PhoenixPay merchant capability: programmatic invoice creation, out-of-band payment detection via both polling and webhooks, and refund processing.

3.7.2 Features and Functions

Backend (Node.js / Express / MariaDB):

Feature	Description
User registration	Email format validation, minimum 8-character password, bcrypt hashing
Login and session	JWT authentication with 24-hour access tokens and a refresh endpoint
Todo CRUD	Create todo items (task text), toggle completion status, delete

Premium subscription	Calls PhoenixPay <code>POST /api/invoices</code> with API key; stores returned invoice ID locally
Premium status polling	Queries PhoenixPay <code>GET /api/invoices/:id</code> for invoice status to detect payment without waiting for a webhook
Webhook receiver	Validates <code>X-PhoenixPay-Signature</code> HMAC-SHA256 header; sets <code>users.is_premium = TRUE</code> on <code>invoice.paid</code> event
Premium cancellation	Calls PhoenixPay <code>POST /api/refunds</code> for 50% of the premium amount; clears premium status
Idempotent DB recovery	Handles the edge case where a Fineract refund transfer succeeds but the local DB update fails, without leaving the user in an inconsistent state

Frontend (Vue 3 / Pinia / Tailwind CSS):

Feature	Description
Dashboard	Task statistics (total / pending / completed) and interactive todo list
Premium modal	Displays PhoenixPay invoice ID, payment instructions, and polls for paid status at regular intervals
Cancel premium modal	Shows calculated refund amount (50%) and triggers the cancellation flow
Responsive layout	Mobile-friendly via Tailwind CSS breakpoints and a collapsible sidebar
Localisation	Full English / Arabic support with RTL layout flip via <code>vue-i18n</code>

3.7.3 Users

User	Type	Interaction
End Users	Human	Vue SPA — manage tasks, subscribe to and cancel premium
PhoenixPay (system)	Service	HTTP POST to TaskMaster's webhook endpoint with HMAC-signed payload

3.7.4 Non-functional Requirements and Constraints

- **Webhook HMAC verification:** The webhook handler uses `crypto.timingSafeEqual()` to compare the computed and received signatures, preventing timing-attack-based forgery.
- **Raw body capture:** The Express application applies `express.raw({ type: 'application/json' })` middleware specifically to the webhook route. This captures the exact raw bytes of the request body before any JSON parsing, ensuring that the HMAC is computed over precisely the same bytes that PhoenixPay signed. Any re-serialisation of a parsed JSON object may alter whitespace and invalidate the signature.
- **Dual payment detection:** Premium activation is detected via both webhook (push, low latency) and status polling (pull, fallback). Polling handles scenarios where the webhook delivery is delayed or the user completes payment while the SPA is not waiting for a webhook.
- **Configurability:** The premium amount and currency are configured via `PREMIUM_AMOUNT` and `PREMIUM_CURRENCY` environment variables, defaulting to 9.99 SDG.

3.7.5 Design

System Architecture:

TaskMaster is a standard two-tier web application. The Express backend exposes a REST API consumed by the Vue 3 SPA. The backend stores user accounts and todos in MariaDB, and communicates with PhoenixPay via two integration channels: outbound API calls (invoice creation, refund requests) authenticated by a PhoenixPay API key, and an inbound webhook endpoint that receives payment events from PhoenixPay.

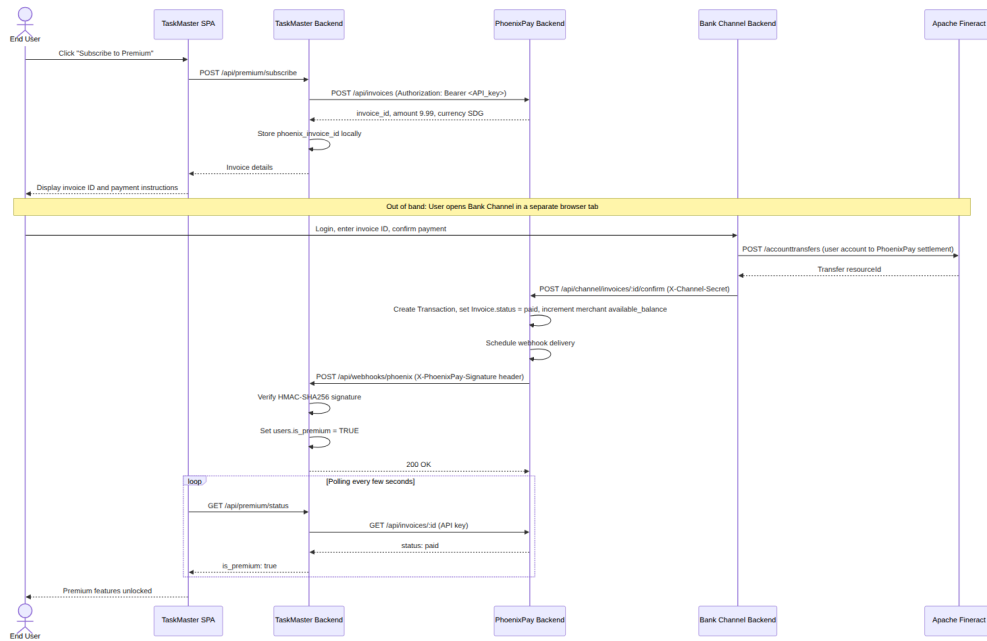


Figure 3.13 — End-to-End Premium Subscription Flow

The sequence illustrates the complementary roles of the webhook and polling mechanisms: the webhook provides near-instant activation when it succeeds, while polling ensures the SPA eventually reflects the correct state regardless of webhook delivery timing.

3.7.6 Use Cases

TaskMaster has two actors: **End User**, a human who manages tasks and purchases premium features through the SPA, and **PhoenixPay**, a system actor that delivers signed webhook events when an invoice is paid.

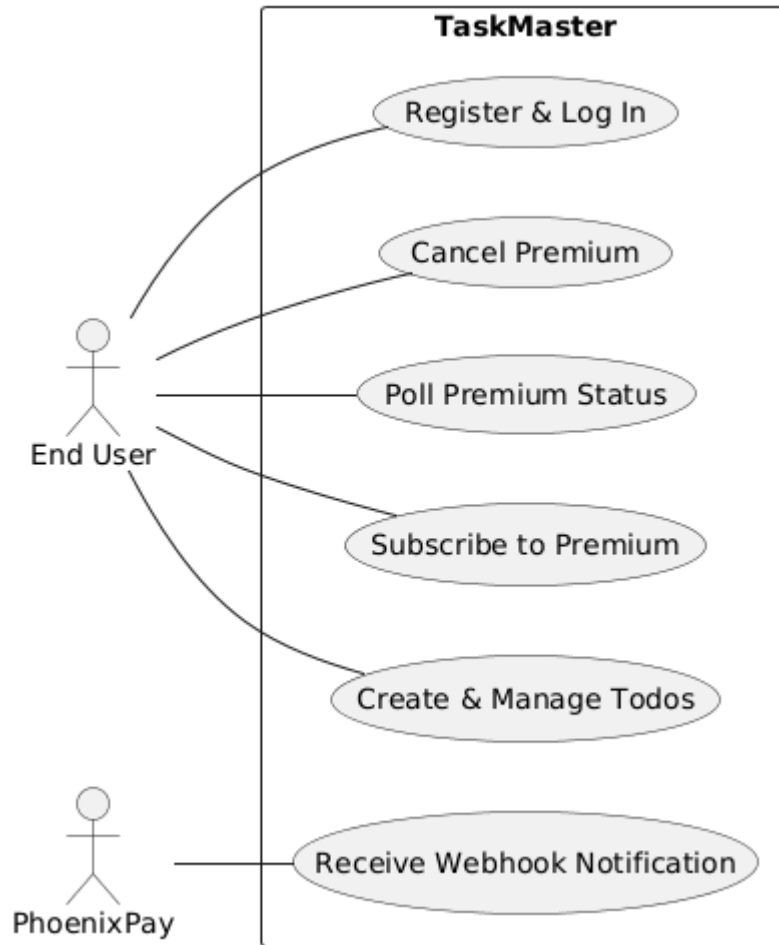


Figure 3.14 — TaskMaster Use Case Diagram

ID	Use Case	Actor	Preconditions	Main Flow	Postconditions
TM-01	Register & Log In	End User	—	User submits email and password (minimum 8 characters); backend bcrypt-hashes the password on registration; login returns a 24-hour JWT stored by the SPA	Session active; user can access their todo list
TM-02	Create & Manage Todos	End User	User logged in	User creates todo items, toggles	Todo list reflects current state; task

				completion status, and deletes items via the SPA; all changes persisted to the MariaDB todos table	statistics updated in dashboard
TM-03	Subscribe to Premium	End User	User not yet premium	SPA POSTs to /api/premium/subscribe; backend calls PhoenixPay POST /api/invoices with its API key; PhoenixPay returns an invoice ID which is stored locally; SPA displays the invoice ID and payment instructions	PhoenixPay invoice created with status pending; user is shown how to pay via the Bank Channel
TM-04	Poll Premium Status	End User (SPA)	Pending invoice exists in local DB	SPA periodically GETs /api/premium/status; backend queries PhoenixPay GET /api/invoices/:id; if status is paid, backend sets users.is_premium = TRUE	Premium unlocked in SPA when polling detects the paid status; serves as fallback when webhook delivery is delayed
TM-05	Receive Webhook Notification	PhoenixPay	Webhook endpoint reachable; WEBHOOK_SECRET configured	PhoenixPay POSTs an HMAC-SHA256 signed invoice.paid payload; TaskMaster verifies the X-PhoenixPay-Signature header using crypto.timingSafeEqual() over the	User premium status updated immediately without waiting for the next poll cycle

				raw request bytes; sets users.is_premium = TRUE on success	
TM-06	Cancel Premium	End User	User is premium; original invoice is paid	SPA POSTs to /api/premium/cancel; backend calls PhoenixPay POST /api/refunds for 50% of the premium amount; on success, users.is_premium is set to FALSE	Partial refund processed; premium features disabled; invoice status updated to partially_refunded

Chapter 4: Implementation & Testing

4.1 Introduction

This chapter documents the implementation and testing of the PhoenixPay platform and the three supporting components that together constitute the complete payment ecosystem. The PhoenixPay system — the core deliverable — receives the most detailed coverage. Apache Fineract, used as-is from its official Docker image, is documented in terms of deployment and configuration only. The Bank Channel and TaskMaster, both custom-built to demonstrate and validate PhoenixPay, are documented with their technology stacks, execution screenshots, and test results.

Because the project's primary focus was the complexity of integrating multiple independently designed systems — Apache Fineract, a custom Bank Channel, a payment-processing backend, and a demonstration merchant application — no dedicated automated test framework was wired up during development. Testing was instead conducted as **systematic manual testing** against the running Docker Compose stack. This approach is appropriate and well-established for prototype-phase academic projects, and the results are recorded here using standard academic test-case notation.

Four test types are applied across the three custom-built components:

- **Functional testing** — verifies that each feature behaves as specified under normal, valid inputs. These tests exercise the positive path and confirm that the system produces the correct output for well-formed requests.
- **Integration testing** — verifies correct interaction between system components. Integration tests cross the boundary between two services (for example, Bank Channel calling the Fineract REST API, or PhoenixPay calling the Bank Channel payout proxy) and confirm that data is transmitted and processed correctly end-to-end.

- **Security testing** — verifies that authentication, authorisation, and data-protection mechanisms function correctly under invalid or adversarial inputs. These tests include sending requests without credentials, with wrong credentials, and with mismatched secrets.
- **Negative testing** — verifies that the system responds gracefully to invalid inputs, boundary violations, and error conditions. These tests confirm that validation rules are enforced and that appropriate error codes are returned.

Apache Fineract was not implemented as part of this project; it was deployed from an official Docker image and configured. Testing of Fineract itself was therefore not undertaken; its correctness is assumed by virtue of its status as a mature, community-tested open-source platform. Its integration with the rest of the system is validated indirectly through the Bank Channel integration tests in Section 4.3.4.

4.2 Component 1: Core Banking System — Apache Fineract

4.2.1 Overview

Apache Fineract is a third-party, open-source core-banking platform and was not implemented as part of this project. It was deployed from its official Docker image and configured to serve as the financial backbone of the system. The project's contribution with respect to Fineract was limited to deployment configuration and the one-time creation of three client accounts via the MifosX web interface.

4.2.2 Technology Stack

Layer	Technology	Notes
Core Banking Platform	Apache Fineract	openmf/fineract:1.11 Docker image; Spring Boot / Java backend
Database	MariaDB 11.4	Docker volume-mounted for data persistence across restarts
Administrator UI	MifosX Community App	Angular SPA served at /core-bank/ by Nginx
Containerisation	Docker	Managed by Docker Compose; health-checked before dependent services start

4.2.3 Setup and Configuration

Deployment of Apache Fineract involved the following steps:

- 1. Image pull and environment configuration.** The official Apache Fineract Docker image was pulled and configured via environment variables: database host, port, the credential pair `mifos:password`, and the tenant identifier `default`. The MariaDB service was started first; Fineract connects to it on startup.
- 2. Initialisation wait.** Apache Fineract requires approximately 300 seconds to complete its Spring Boot context initialisation and database migration sequence. A Docker Compose health check on `GET /fineract-provider/api/v1/clients?limit=0` gates all dependent services, ensuring the Bank Channel does not attempt to connect before Fineract is ready.
- 3. Client account creation.** After Fineract initialised, the MifosX Community App web interface was used to create three client accounts, each with an associated SDG-denominated savings account:

- **Client 1 — PhoenixPay:** holds the settlement savings account into which all invoice payments flow. Client ID 1 and account ID 1 were configured into both the Bank Channel

(`PHOENIX_FINERACT_CLIENT_ID`,
`PHOENIX_FINERACT_ACCOUNT_ID`) and the PhoenixPay backend environment files.

- **Client 2 — TaskMaster:** represents the demo merchant. Its savings account serves as the destination for payout transfers from the PhoenixPay settlement account.

- **Client 3 — Mohamed:** represents a regular bank customer. This account is the source of funds in bill-payment and peer-to-peer transfer tests.

4. **Environment propagation.** The Fineract client IDs and account IDs assigned during step 3 were noted and inserted into the `.env` files of the Bank Channel and PhoenixPay backends before bringing those services online.

4.2.4 Execution Screenshots

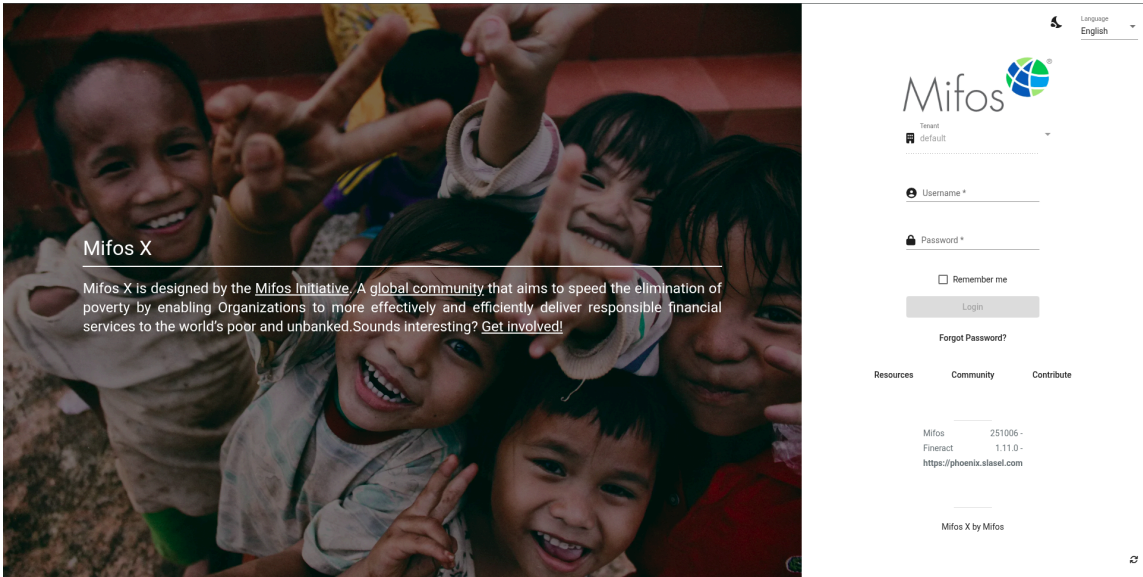


Figure 4.1 — MifosX Login Page

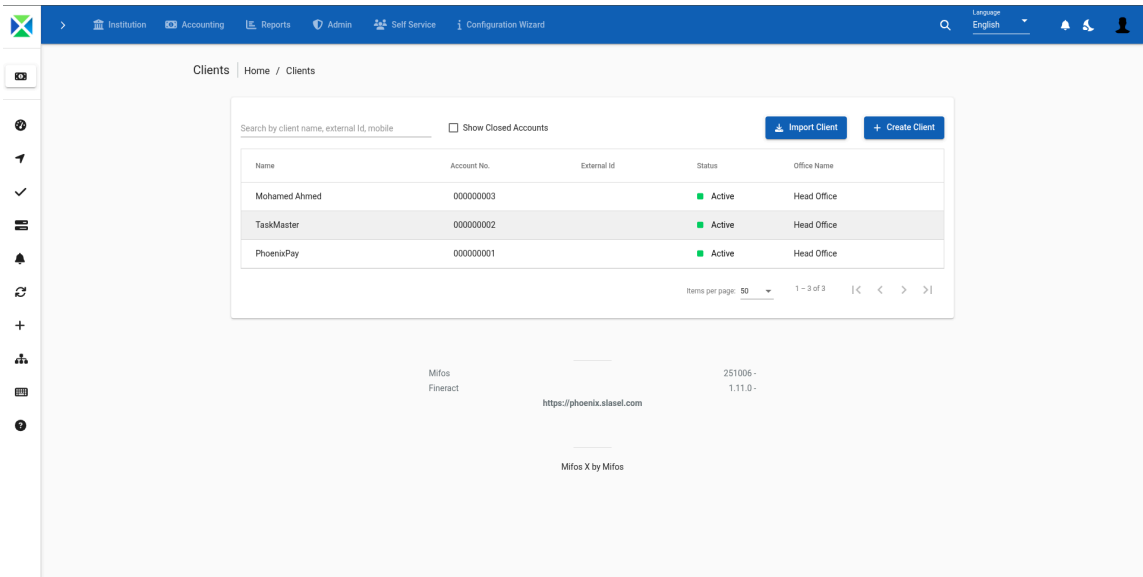


Figure 4.2 — Fineract Client List (Three Registered Clients)

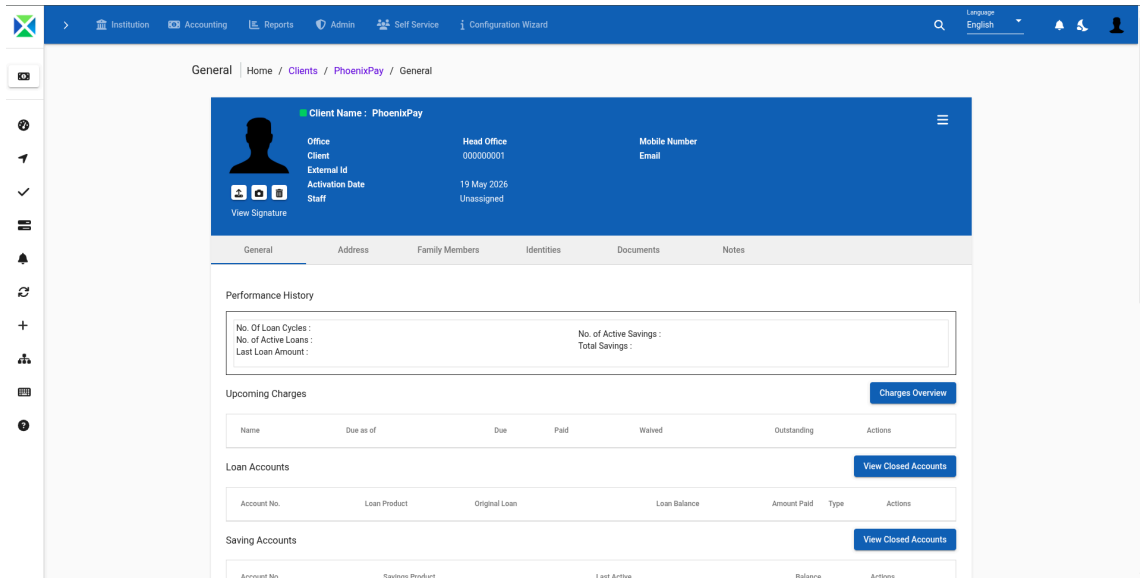


Figure 4.3 — PhoenixPay Settlement Account Detail

4.2.5 Testing

Apache Fineract is a production-grade, community-tested open-source platform maintained by the Apache Software Foundation. No independent testing of Fineract itself was performed; the platform is assumed correct. The integration between Fineract and the custom-built components is verified indirectly through the Bank Channel integration tests documented in Section 4.3.4, which call the Fineract REST API (POST /accounttransfers, GET /savingsaccounts/:id?associations=transactions, GET /search) and confirm that the expected transfer outcomes and balance changes are produced.

4.3 Component 2: Bank Channel

4.3.1 Overview

The Bank Channel is a fully custom-built application consisting of a Node.js/Express backend API and a Vue 3 single-page application. It is the only

system component that communicates directly with Apache Fineract, acting as a secure intermediary for all fund movements. It serves two actors: bank customers (via the SPA) and PhoenixPay (via the authenticated `routes/phoenix.js` endpoints).

4.3.2 Technology Stack

Backend:

Layer	Technology	Version
Runtime	Node.js	20 LTS
Framework	Express.js	4.18.2
Database Driver	MySQL2	3.6.5
Authentication	jsonwebtoken	9.0.3
Password Hashing	bcrypt	6.0.0
HTTP Client	Axios	1.6.0
Containerisation	Docker (node:20-alpine)	—

Frontend:

Layer	Technology	Version
Framework	Vue 3	3.5.24
Build Tool	Vite	7.2.4
State Management	Pinia	3.0.4
Routing	Vue Router	5.0.1
HTTP Client	Axios	1.13.4
CSS Framework	Tailwind CSS	4.1.18
Internationalisation	vue-i18n	10.0.8

Database: MariaDB 11.4 (MySQL-compatible) — `bank_channel` schema; 2 tables: `users` (`id`, `username`, `password_hash`, `fineract_client_id`) and `transactions` (local audit trail of all transfers and bill payments)

4.3.3 Execution Screenshots

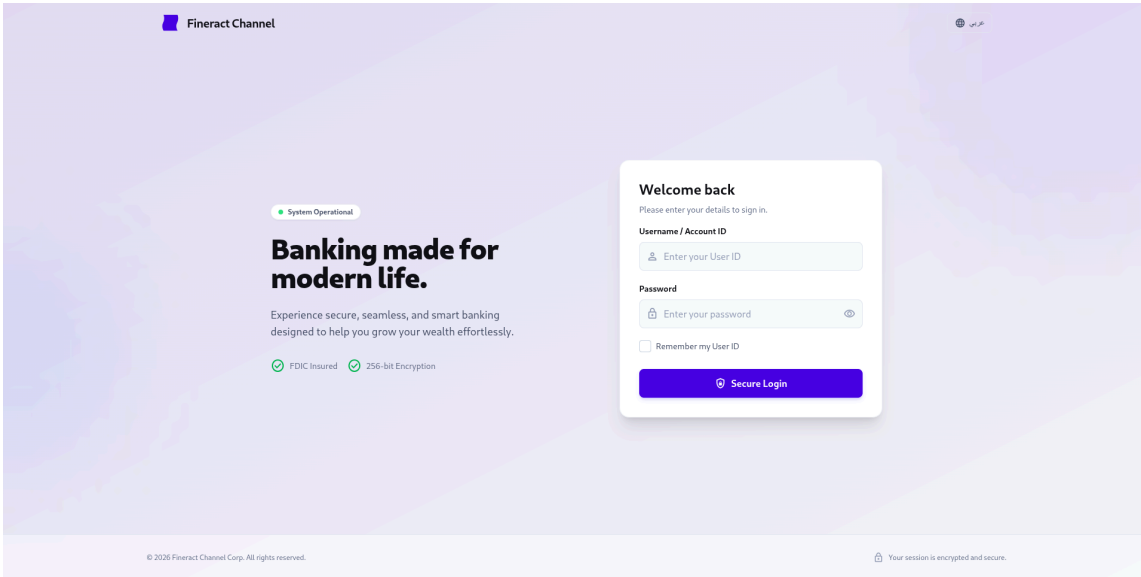


Figure 4.4 — Bank Channel Login Screen

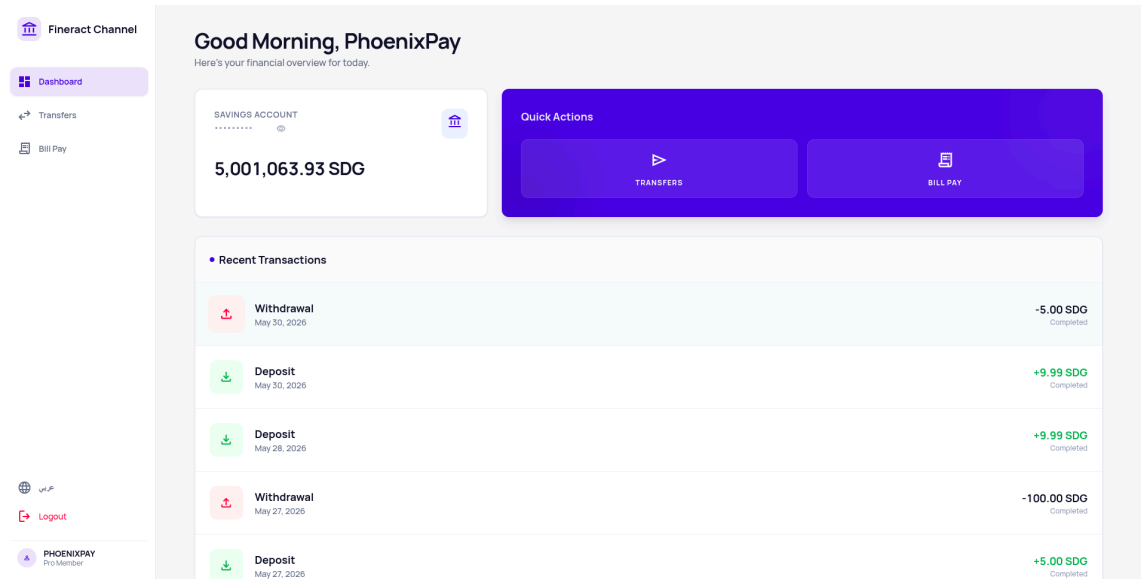


Figure 4.5 — Account Dashboard (Balances)

Fineract Channel

Dashboard

Transfers

Bill Pay

Logout

PHOENIXPAY
Pro Member

Transfer Funds

Inter-bank transfer made easy

Recipient Account

Enter account number

Continue

Transfer Funds

Inter-bank transfer made easy

RECIPIENT

TaskMaster

888888882

Amount (SDO)

Enter amount

Remarks (Optional)

What is this for?

Transfer

Change Recipient

Figure 4.6 — Peer-to-Peer Transfer Form

91

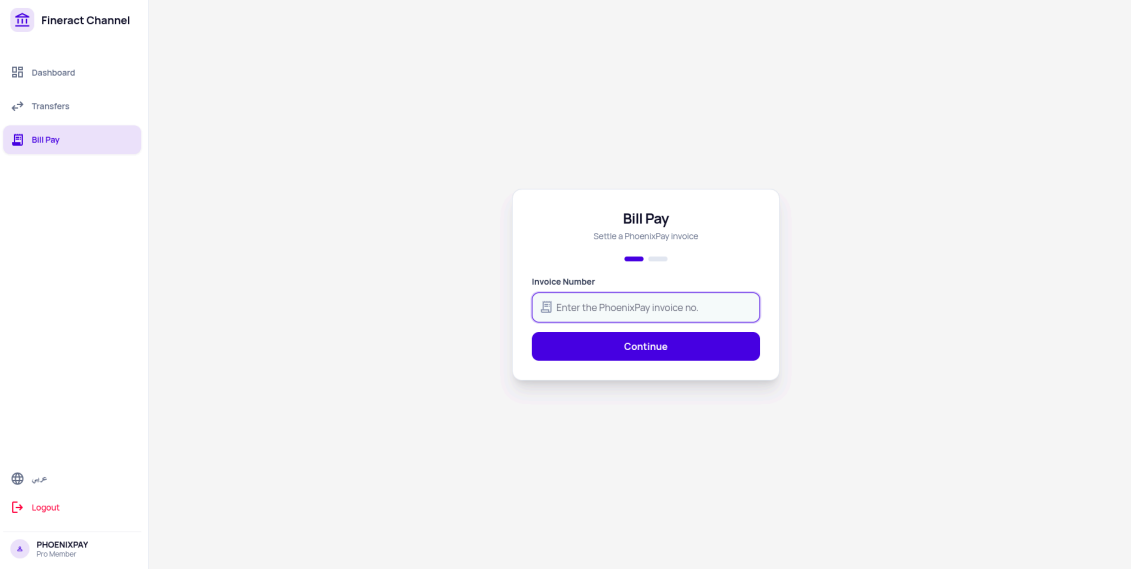


Figure 4.7 — Bill Payment — Invoice Lookup

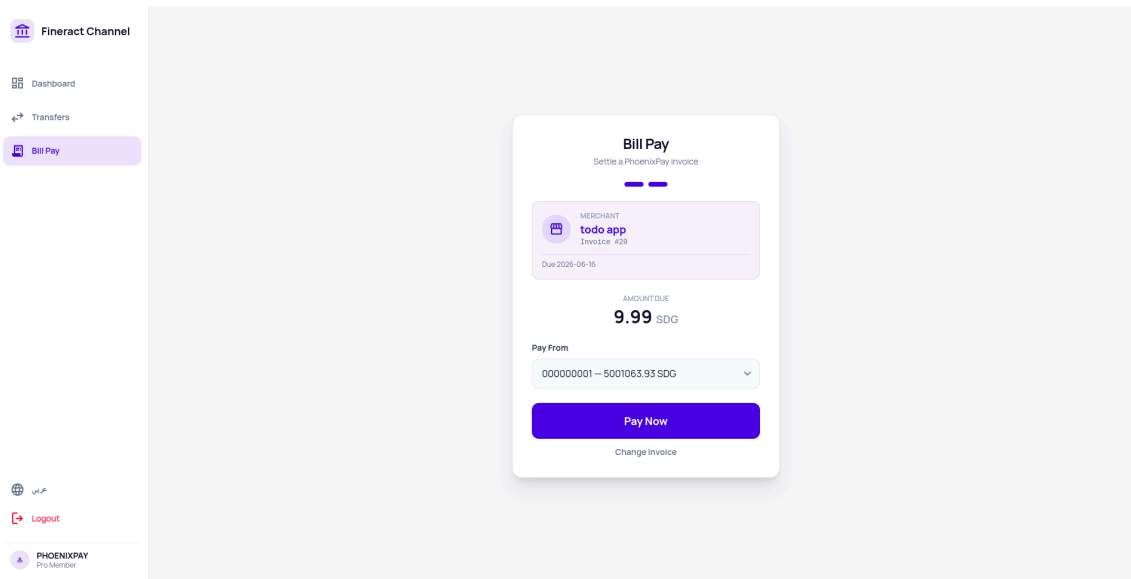


Figure 4.8 — Bill Payment — Confirmation Screen

4.3.4 Testing

All tests were executed manually against the running Docker Compose stack. The table in Appendix A records each test case with its type, objective, input steps, expected result, actual result, and pass/fail status.

4.4 Component 3: PhoenixPay System

4.4.1 Overview

PhoenixPay is the payment-processing platform at the centre of the project, comprising a Node.js/Express REST API backend and a Vue 3 Merchant Dashboard. The backend implements 14 Sequelize models across four authentication layers (JWT, API key, channel secret, and admin RBAC), rate limiting on all sensitive endpoints, idempotent payment confirmation backed by a database unique constraint, HMAC-SHA256 webhook delivery with exponential-backoff retry, and a full audit trail. The Merchant Dashboard provides a graphical interface for every backend capability, with full English and Arabic localisation.

4.4.2 Technology Stack

Backend:

Layer	Technology	Version
Runtime	Node.js	20 LTS
Framework	Express	5.2.1
ORM	Sequelize	6.37.7
Database Driver	MySQL2	3.16.2
Authentication	jsonwebtoken	9.0.3
Password Hashing	bcryptjs	3.0.3
Input Validation	express-validator	7.3.1
Rate Limiting	express-rate-limit	8.5.2
HTTP Client	Axios	1.16.1

API Documentation	swagger-ui-express	5.0.1
Containerisation	Docker (node:20-alpine)	—

Merchant Dashboard:

Layer	Technology	Version
Framework	Vue 3	3.5.24
Build Tool	Vite	7.3.1
State Management	Pinia	3.0.4
Routing	Vue Router	5.0.2
HTTP Client	Axios	1.13.4
CSS Framework	Tailwind CSS	3.4.17
Internationalisation	vue-i18n	10.0.8

Database: MariaDB 11.4 (MySQL-compatible) — 14 Sequelize models; all monetary columns typed `DECIMAL(15, 2)`; timestamps stored in UTC

4.4.3 Execution Screenshots

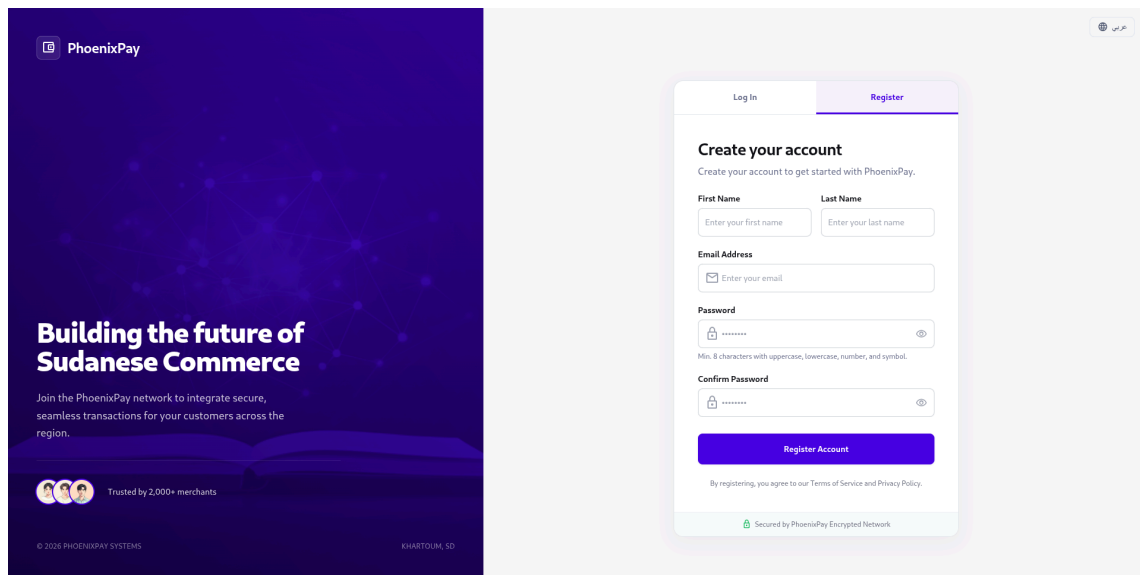


Figure 4.9 — Merchant Dashboard — Developer Registration

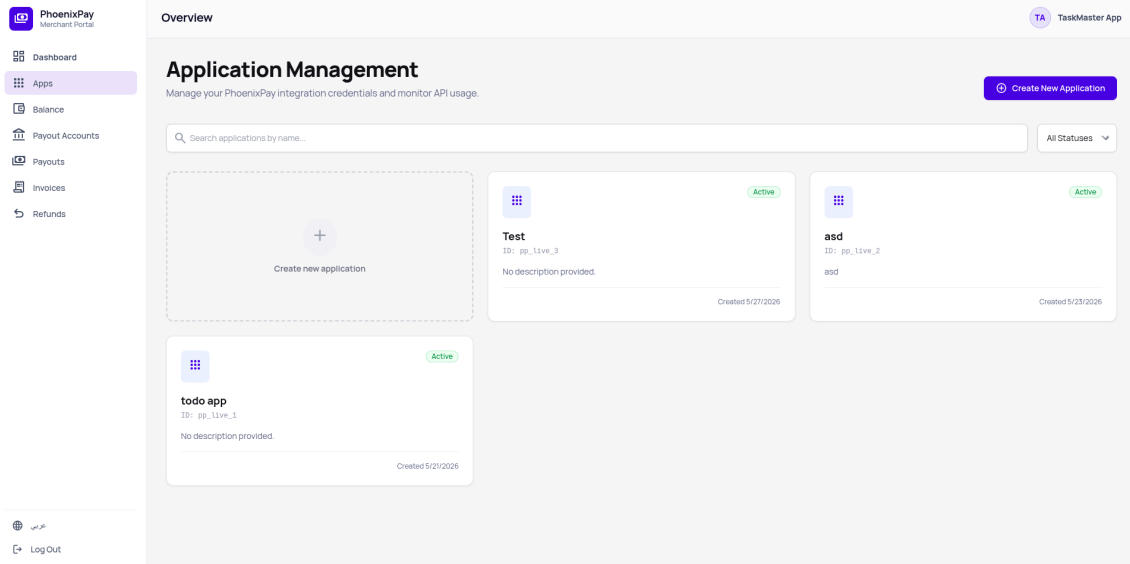


Figure 4.10 — Application Management

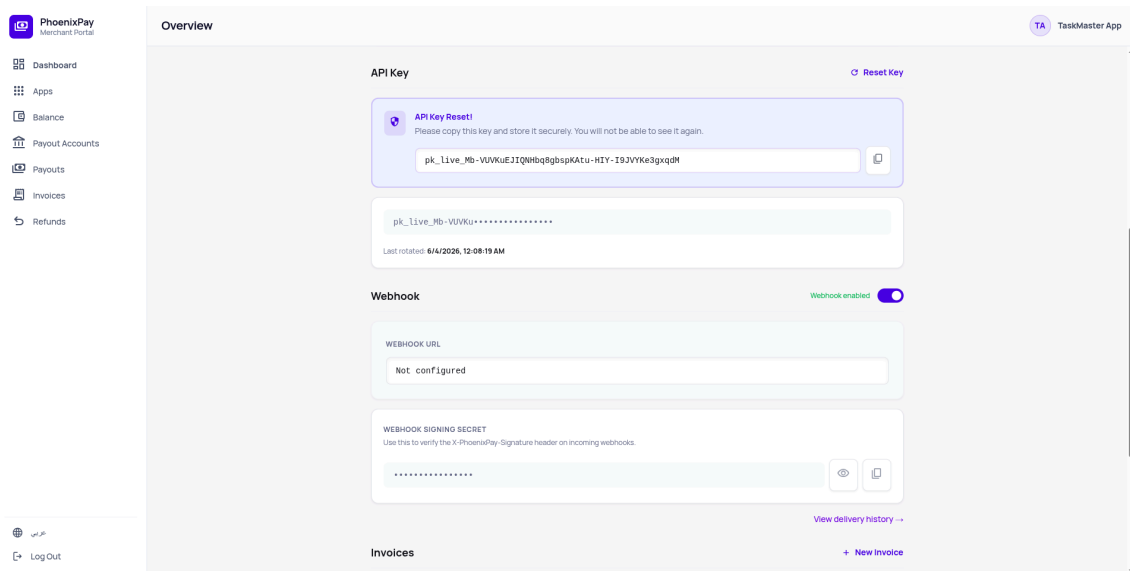


Figure 4.11 — API Key Generation (Key Shown Once)

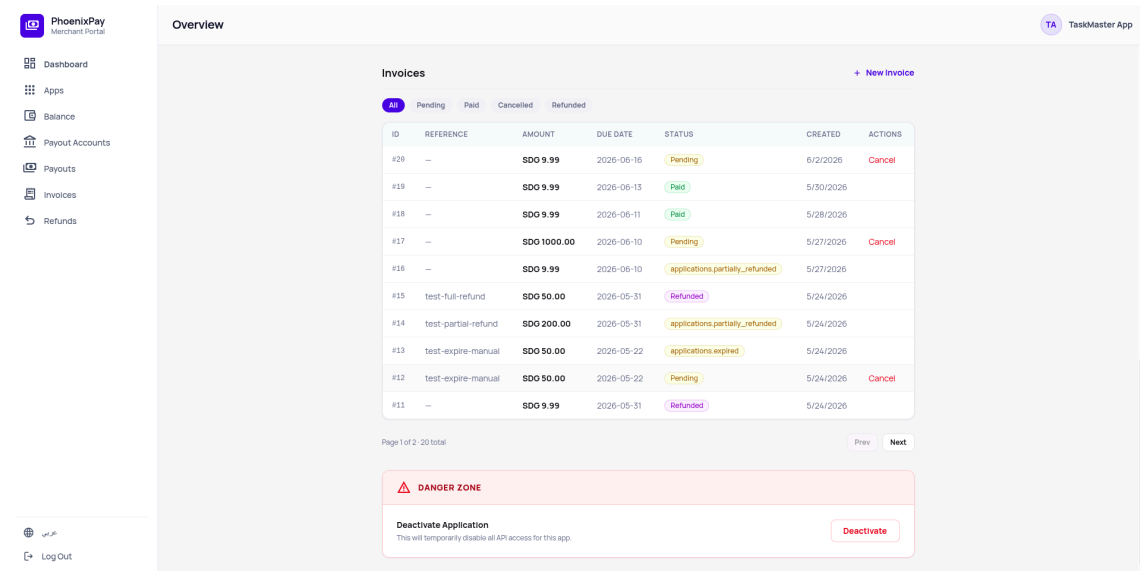


Figure 4.12 — Invoice List with Filters

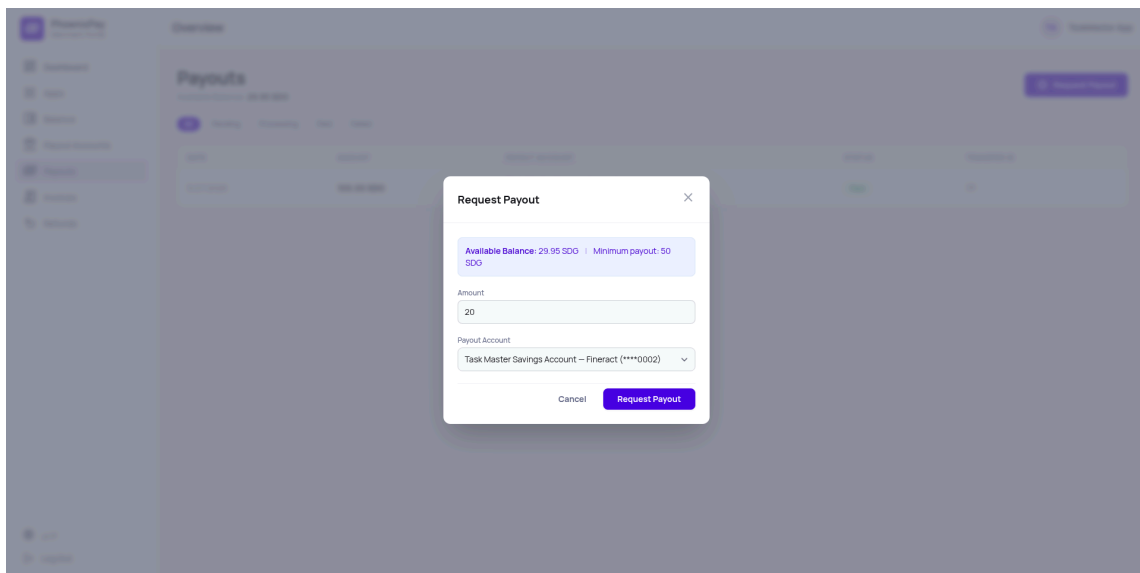


Figure 4.13 — Balance View and Payout Request

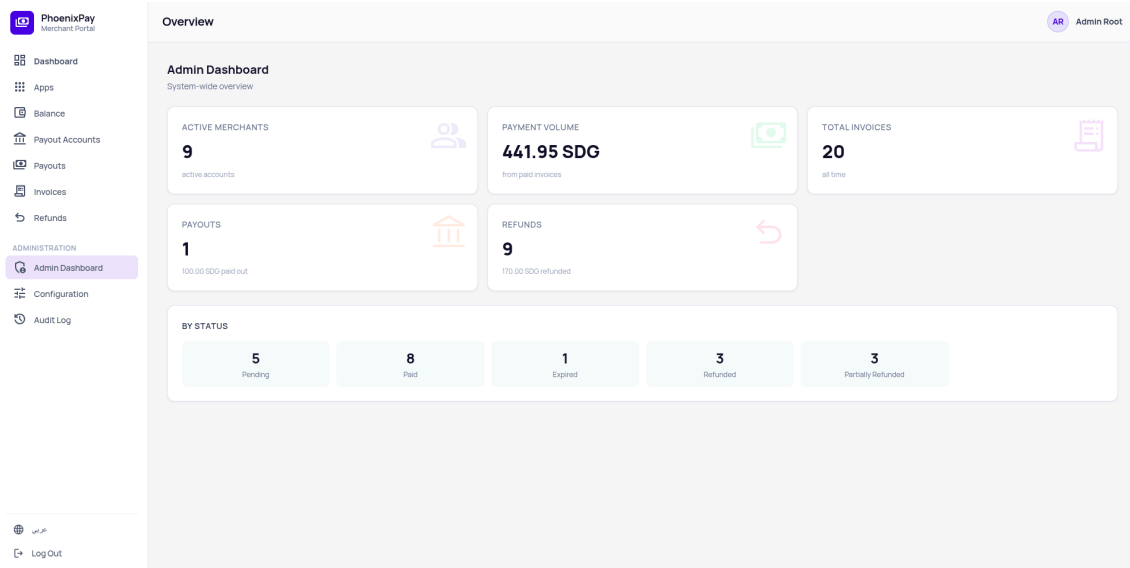


Figure 4.14 — Admin Dashboard

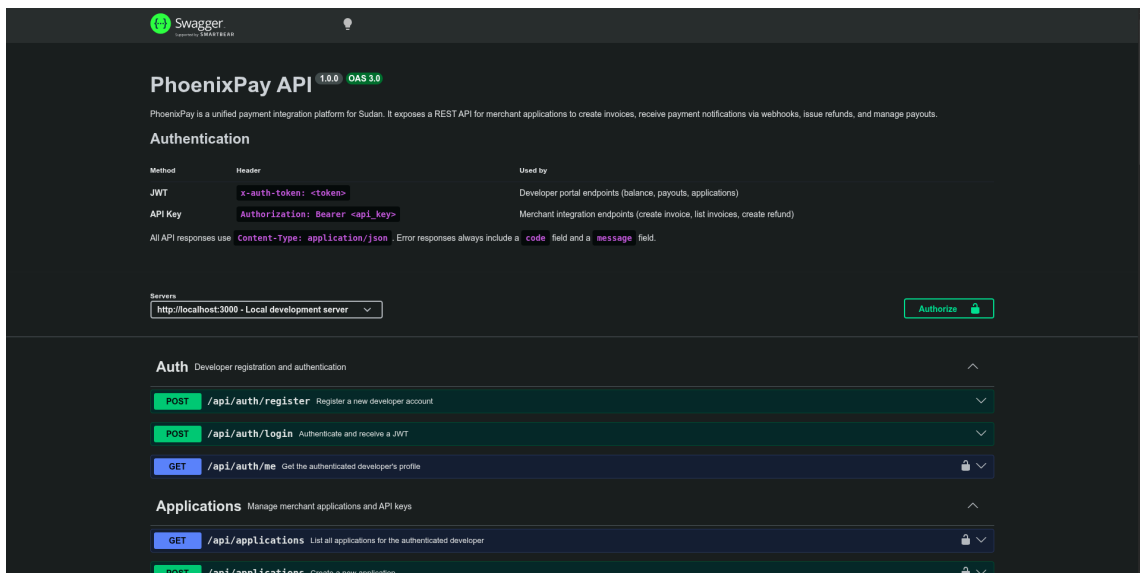


Figure 4.15 — Swagger UI (Interactive API Documentation)

4.4.4 Testing

All tests were executed manually against the running Docker Compose stack. Test cases are ordered by functional area (see Appendix B).

4.5 Component 4: TaskMaster — Demo Merchant Application

4.5.1 Overview

TaskMaster is an independently implemented task-management SaaS application that serves as a demonstration of a complete PhoenixPay merchant integration. Its Node.js/Express backend and Vue 3 frontend are deployed as separate containers from the rest of the system, mirroring the way a real third-party merchant application would be structured. TaskMaster exercises invoice creation, payment detection via both webhooks and polling, and premium cancellation with a partial refund.

4.5.2 Technology Stack

Backend:

Layer	Technology	Version
Runtime	Node.js	20 LTS
Framework	Express	5.2.1
Database Driver	MySQL2	3.16.2
Authentication	jsonwebtoken	9.0.3
Password Hashing	bcryptjs	3.0.3
HTTP Client	Node.js native fetch API	Built-in
Containerisation	Docker (node:20-alpine)	—

Frontend:

Layer	Technology	Version
Framework	Vue 3	3.5.24
Build Tool	Vite	7.2.4
State Management	Pinia	3.0.4
Routing	Vue Router	5.0.1
HTTP Client	Axios	1.13.4

CSS Framework	Tailwind CSS	4.1.18
Internationalisation	vue-i18n	10.0.8

Database: MariaDB 11.4 (MySQL-compatible) — 2 tables: `users` (includes `is_premium` flag and `phoenix_invoice_id` foreign reference) and `todos`

4.5.3 Execution Screenshots

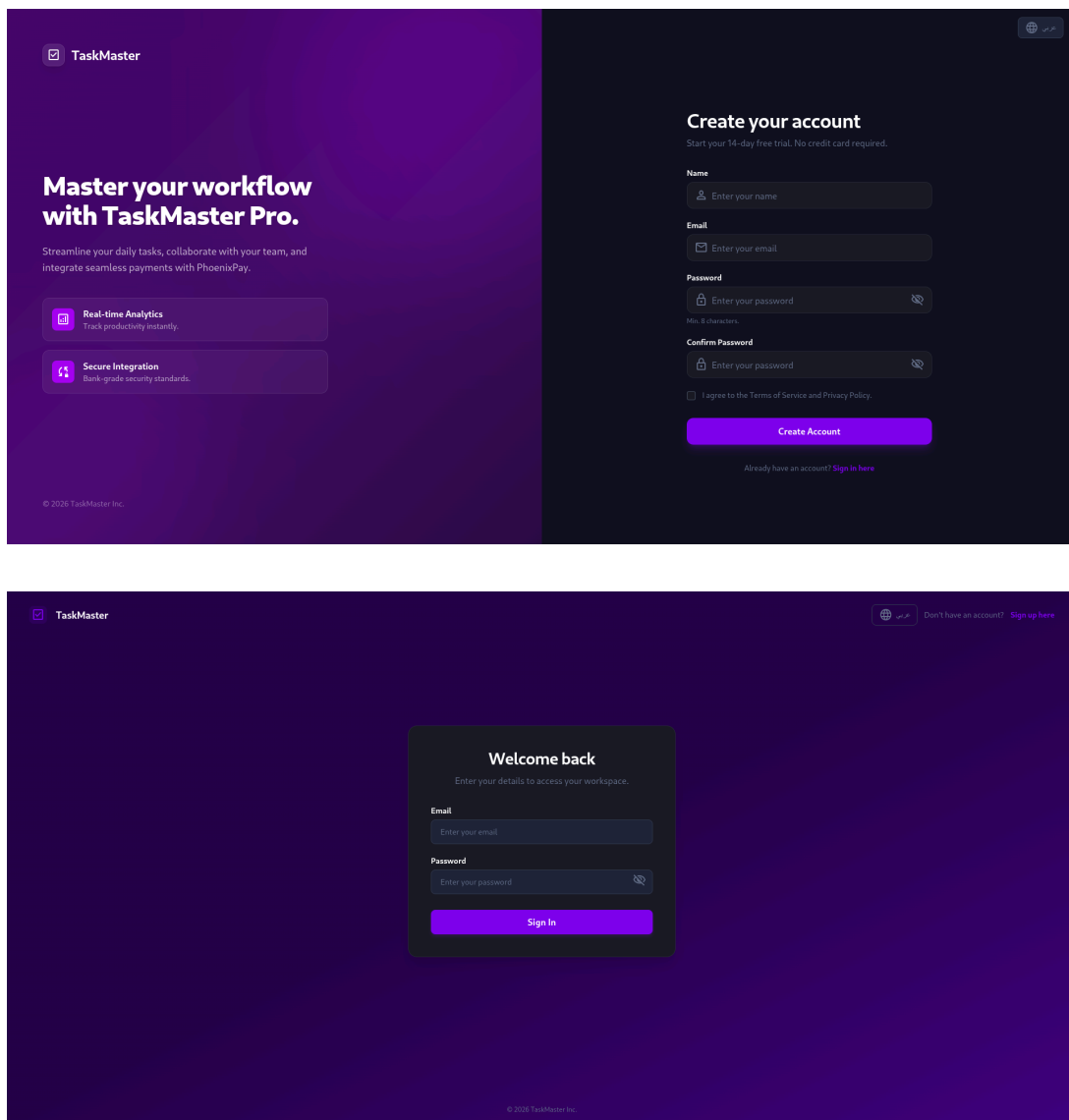


Figure 4.16 — TaskMaster Login / Registration

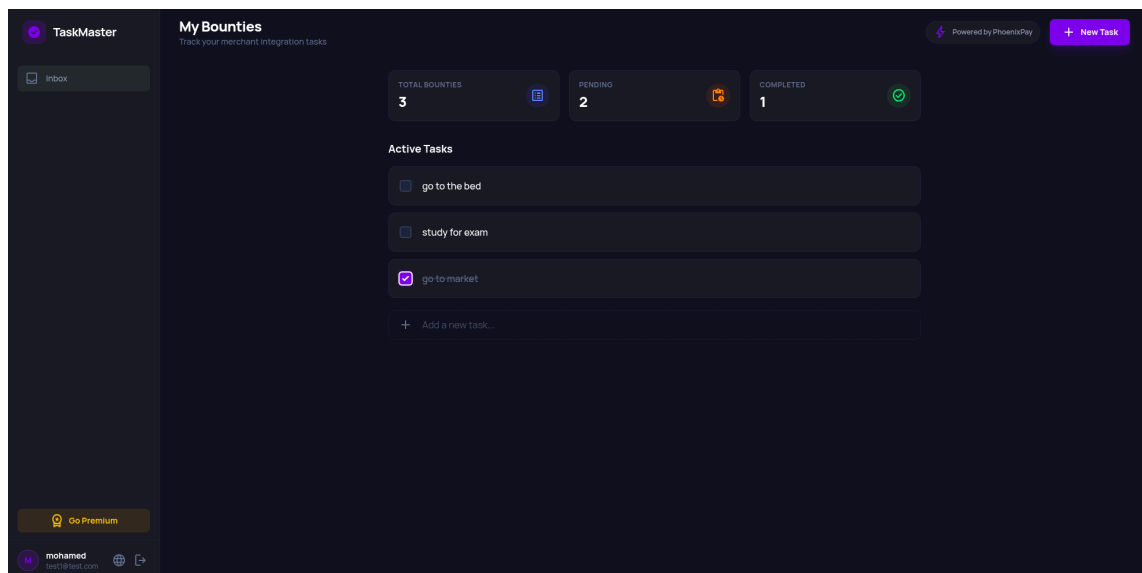


Figure 4.17 — Dashboard with Todo List

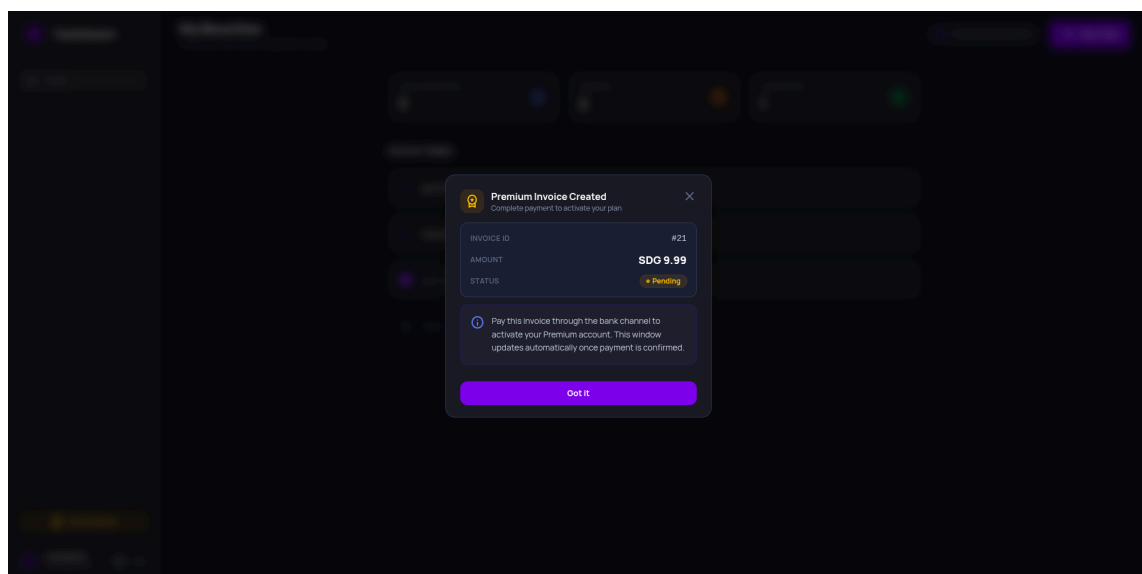


Figure 4.18 — Premium Subscription Modal (Invoice Displayed)

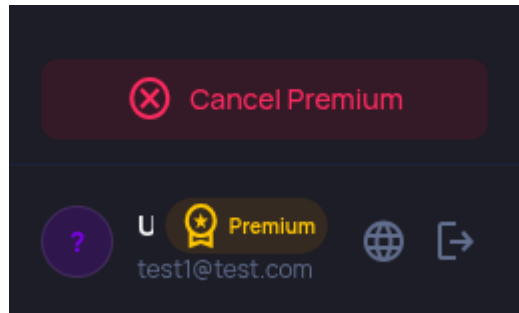


Figure 4.19 — Premium Features Unlocked

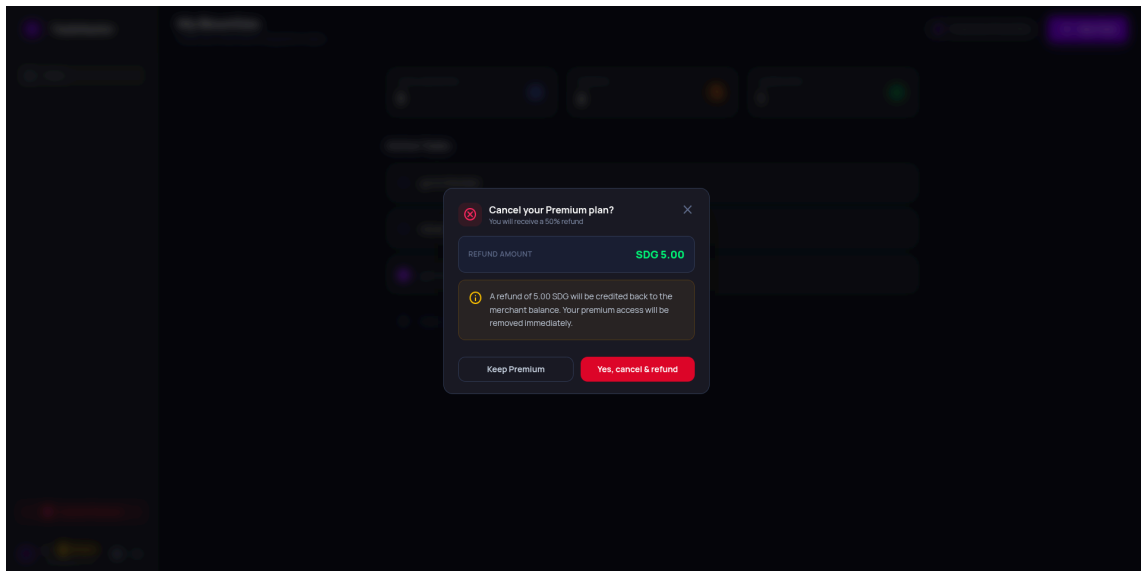


Figure 4.20 — Cancel Premium Modal

4.5.4 Testing

All tests were executed manually against the running Docker Compose stack (see Appendix C).

Chapter 5: Results, Conclusion, and Recommendations

5.1 Results

5.1.1 Functional Requirement Coverage

The platform was designed against twenty-four functional requirements. All twenty-four requirements were fully implemented and verified during system testing. The table below maps each requirement to its description and implementation status.

Requirement ID	Description	Status
FREQ-1	Developer registration with email validation and secure password (8+ characters, mixed case, digits, and symbols)	Implemented
FREQ-2	Developer authentication via JWT with 8-hour token lifetime and proactive refresh	Implemented
FREQ-3	Merchant application creation, update, activation, and deactivation with audit logging	Implemented
FREQ-4	API key generation (bcrypt-hashed, 16-character prefix for fast lookup), rotation, and revocation	Implemented
FREQ-5	Per-application webhook configuration: URL, signing secret, and enabled/disabled flag	Implemented
FREQ-6	Invoice creation with positive SDG amount, description, optional merchant reference, and optional expiry date	Implemented
FREQ-7	Invoice listing with status and date-range filters and pagination	Implemented
FREQ-8	Invoice cancellation (pending invoices only) by the issuing merchant application	Implemented
FREQ-9	Automatic invoice expiry: status set to <code>expired</code> when <code>due_date</code> passes, checked lazily on GET and on confirmation	Implemented
FREQ-10	Bank Channel invoice inquiry authenticated by channel shared secret, returning amount and merchant name	Implemented

FREQ-11	Idempotent payment confirmation with a unique database constraint on (<code>invoice_id</code> , <code>bank_reference</code>) to prevent duplicate ledger credits	Implemented
FREQ-12	Atomic ledger credit on payment confirmation: Sequelize transaction wraps invoice status update and balance increment	Implemented
FREQ-13	HMAC-SHA256 signed <code>invoice.paid</code> webhook delivery carrying the full transaction payload	Implemented
FREQ-14	Exponential-backoff webhook retry with delivery attempts at 0 s, 30 s, 5 min, 10 min, and 15 min; marked <code>failed</code> after 5 attempts	Implemented
FREQ-15	Startup reconciliation: stale pending webhook deliveries are re-queued when the server boots	Implemented
FREQ-16	Per-developer balance ledger tracking his <code>available_balance</code>	Implemented
FREQ-17	Payout account registration: store a merchant's Fineract client ID and account ID with a label	Implemented
FREQ-18	Payout initiation with minimum-threshold enforcement and available-balance validation	Implemented
FREQ-19	Payout execution via Bank Channel <code>/api/phoenix/payout</code> → Fineract POST <code>/accounttransfers</code> ; Fineract transfer ID stored on the payout record	Implemented
FREQ-20	Refund initiation by a merchant application with amount, reason, and available-balance validation	Implemented
FREQ-21	Refund execution via Bank Channel <code>/api/phoenix/refund</code> → Fineract transfer; invoice status updated to <code>refunded</code> or <code>partially_refunded</code>	Implemented
FREQ-22	Admin system configuration management:	Implemented

	payout_min_threshold and invoice_default_expiry_days configurable at runtime	
FREQ-23	Comprehensive audit trail: every sensitive action recorded with actor email, IP address, resource type and ID, action name, metadata JSON, and UTC timestamp	Implemented
FREQ-24	Admin audit log query with date-range and action-type filters and pagination	Implemented

5.1.2 Non-Functional Requirement Coverage

All six non-functional requirements were satisfied. The table below maps each requirement to its implementation.

Requirement ID	Requirement	Implementation
NFREQ-1	Multi-layer security controls	JWT authentication, bcrypt-hashed passwords, HMAC-SHA256 webhook signatures with timing-safe verification, channel shared secret authenticated via <code>crypto.timingSafeEqual()</code> , RBAC admin access, and per-endpoint rate limiting (login: 10 req/15 min; invoice creation: 100 req/15 min; payouts: 20 req/15 min; channel confirmation: 30 req/15 min)
NFREQ-2	Stateless backend design	JWT-based sessions with no server-side session state; all persistent state stored in MariaDB
NFREQ-3	Durable, ACID-compliant financial storage	MariaDB with Sequelize database transactions for all balance updates; <code>DECIMAL(15, 2)</code> for every monetary column; UTC timestamps; Docker volume-mounted database directories
NFREQ-4	Reliable webhook delivery	Five-attempt exponential-backoff retry; startup reconciliation re-queues any deliveries that were still in

		pending status when the server last stopped
NFREQ-5	Idempotent payment confirmation	Unique database constraint on (invoice_id, bank_reference) in the transactions table prevents duplicate ledger credits even under concurrent retries
NFREQ-6	Complete audit trail	Every sensitive and money-related operation is logged to audit_logs with actor email, IP address, resource type, resource ID, action name, metadata JSON field, and UTC timestamp

5.1.3 Test Results Summary

Chapter 4 documents 54 manual test cases executed against the running Docker Compose stack. All 54 cases passed, yielding a 100% pass rate. The tables below summarise results by component and by test type.

Table 5.1 — Test Results by Component:

Component	Test Cases	Passed	Failed	Pass Rate
Bank Channel	13	13	0	100%
PhoenixPay System	26	26	0	100%
TaskMaster	15	15	0	100%
Total	54	54	0	100%

Table 5.2 — Test Results by Test Type:

Test Type	Count	Passed	Purpose
Functional	20	20	Positive-path feature verification
Integration	17	17	Cross-component interaction verification
Security	10	10	Authentication, authorisation, and signature checks

Negative	7	7	Input validation and error-condition handling
Total	54	54	

5.1.4 System Evaluation — Strengths and Limitations

Strengths:

- **End-to-end payment lifecycle.** The platform implements the complete payment flow — from invoice creation through bank-channel payment confirmation, balance crediting, webhook delivery, and payout or refund — without any manual operator intervention. Every step is automated and observable via the audit log and the webhook delivery log.
- **Production-grade security patterns.** Four authentication layers (JWT for developer sessions, API key for merchant applications, channel shared secret for bank-to-platform calls, and RBAC for admin operations), HMAC-SHA256 signed webhooks with timing-safe verification, bcrypt-hashed credentials, and per-endpoint rate limiting bring the security design to production quality rather than prototype quality.
- **Financial correctness.** The combination of `DECIMAL(15, 2)` monetary columns, Sequelize ACID transactions for all balance updates, and a unique idempotency constraint on `(invoice_id, bank_reference)` eliminates the three most common sources of financial data corruption in payment systems: floating-point rounding errors, concurrent double-credits, and retry-induced duplicates.
- **Developer experience.** The self-service Merchant Dashboard, auto-generated OpenAPI 3.0 documentation served at `/api-docs`, and a fully integrated demonstration application (TaskMaster) give an integrating

developer everything needed to understand, test, and integrate PhoenixPay without requiring external support.

- **Localisation.** Full English and Arabic support with RTL layout and SDG currency formatting across both the Merchant Dashboard and TaskMaster reflects genuine commitment to the Sudanese target market rather than a token internationalisation stub.
- **Observability.** Every audit-log entry captures the actor's identity, IP address, resource type and ID, action name, and a metadata JSON field, providing a financial trail that would satisfy basic compliance review requirements.

Limitations:

- **No live bank integration.** The Bank Channel simulates a real banking interface via Apache Fineract but does not connect to any actual Sudanese payment network or interbank system. A production deployment would require negotiating and implementing integration protocols with one or more licensed financial institutions.
- **Single-node deployment.** The Docker Compose setup lacks database replication, load balancing, and automated failover. It is appropriate for a demonstration prototype but cannot sustain production traffic or provide the uptime guarantees required in a financial system.
- **Manual Fineract account provisioning.** Creating Fineract client accounts requires manual steps in the MifosX web UI. A production system would need an automated provisioning workflow that creates Fineract entities programmatically as part of the merchant onboarding flow.
- **Manual testing only.** All 54 test cases were executed manually. The absence of an automated test suite means that regressions are not

automatically detected and the testing effort cannot scale as the codebase grows.

5.2 Conclusion

The central motivation behind this project was the fragmented state of online payment integration in Sudan. Developers seeking to add payment functionality to their applications face a landscape with no unified API, inconsistent documentation, and no sandbox environments. Each financial institution requires a separate, bespoke integration, creating duplicated effort, inconsistent security practices, and barriers that are prohibitive for small development teams.

PhoenixPay was designed and built to address this gap directly. Modelled on the developer-platform pattern established by production payment processors such as Stripe and PayPal, it provides a single REST API through which any merchant application can create invoices, receive payment confirmation via signed webhooks, track balances, and initiate payouts and refunds — all without the merchant needing to interact with the underlying banking infrastructure.

The technical substance of the project extends well beyond a conceptual prototype. The payment confirmation endpoint is genuinely idempotent, enforced at the database level by a unique constraint rather than application-layer checks. Webhook delivery is reliable through a five-attempt exponential-backoff retry loop and server-startup reconciliation. All financial operations use ACID database transactions and fixed-point arithmetic to prevent ledger corruption. The authentication design applies four distinct security layers appropriate to four distinct caller types. The audit log captures every sensitive and money-related event with enough context for compliance review. Taken together, these are not prototype conveniences — they are the same engineering patterns that production

payment systems apply, and they would carry forward unchanged into a live deployment.

PhoenixPay is the core deliverable of this project; the supporting components exist to prove that it works. The three-subsystem architecture — a REST API backend, a Vue 3 developer portal, and a demonstration merchant application — is a deliberate design choice. The developer portal gives the project a real user interface that any developer can use without writing code. The demonstration application, TaskMaster, closes the loop by exercising every major integration point: invoice creation, out-of-band payment detection via both webhook and polling, and partial refund on cancellation. Together they prove that the system works end-to-end in a realistic scenario, not just in isolation.

The project's primary constraint is the absence of a connection to a real bank. Apache Fineract — deployed as-is from its official Docker image and used to simulate core-banking operations — is a mature, production-grade engine, but it represents a simulation of the banking relationship rather than the banking relationship itself. Reaching production would require regulatory engagement with the Central Bank of Sudan, technical certification with at least one Sudanese bank or mobile-money provider, and the implementation of AML/KYC workflows. None of these are obstacles that the current architecture cannot accommodate; they are milestones on a credible roadmap.

Sudan's digital economy is growing, and the businesses and developers building within it need payment infrastructure that matches modern expectations: clear documentation, sandbox environments, signed webhooks, and a single integration point rather than ten. PhoenixPay demonstrates that this infrastructure can be built — correctly, securely, and with the open-source tools available today.

5.3 Recommendations for Future Work

The following recommendations are ordered by priority, from the improvements that would have the greatest immediate impact on production readiness to those that address longer-term strategic capability.

1. Automated test suite. The absence of an automated test suite is the most significant gap between the current prototype and a production-ready system. The recommended approach is three tiers: unit tests for business logic (ledger calculations, state machine transitions, idempotency checks) using Jest; API integration tests using Supertest against a test-isolated MariaDB database; and end-to-end tests for the Merchant Dashboard and TaskMaster using Playwright. A target of 80% line coverage on the PhoenixPay backend should be reached before any production deployment.

2. Live bank channel integration. The Bank Channel must be connected to at least one real Sudanese financial institution. Mobile-money providers — such as Cashi or Bravo — are the most accessible first integration targets because they expose REST APIs without requiring full core-banking certification. The `/api/channel/*` endpoint group in the PhoenixPay backend provides the precise integration surface; adding a new channel adapter requires only a new `channelAuth` middleware variant and a transfer executor that maps to the provider's protocol.

3. Message queue for webhook delivery. The current in-process webhook retry loop is adequate for a prototype but will not survive process restarts or horizontal scaling in production. Replacing it with BullMQ backed by Redis provides durable retry storage that survives crashes, dead-letter queues for persistently failing endpoints, and out-of-the-box Grafana dashboards that expose webhook delivery latency and failure rates without additional application code.

4. Redis-backed rate limiting. The current `express-rate-limit` configuration uses in-process memory, which resets on server restart and cannot be shared across multiple API server instances. Switching to a Redis store (via the `rate-limit-redis` adapter) enables accurate, consistent per-IP and per-user rate limits across a horizontally scaled deployment.

5. Database high availability. The Docker Compose setup uses a single MariaDB instance with no replication. A production deployment should configure MariaDB primary-replica replication, Sequelize read/write splitting, automated daily backups with point-in-time recovery, and container orchestration via Docker Swarm or Kubernetes with health-checked restart policies.

6. Multi-currency support. As Sudan's payment landscape matures and cross-border payments become commercially relevant, extending PhoenixPay to support USD and EUR alongside SDG would significantly increase the platform's utility. This requires changes to the `invoices` and `transactions` models to store the settlement currency, integration with an exchange-rate data source for real-time conversion rates, and currency-specific number formatting in both the Merchant Dashboard and TaskMaster i18n locale files.

7. Monitoring and alerting. Production operation requires real-time visibility into system health. The backend should be instrumented with Prometheus metrics covering request rate, error rate, webhook delivery success rate, payout processing latency, and Fineract availability. These metrics should drive a Grafana dashboard and PagerDuty or Opsgenie alerts that page the on-call engineer when webhook failure spikes, Fineract becomes unavailable, or abnormal payout volumes are detected.

8. Regulatory alignment. Any production deployment handling real money in Sudan must engage the Central Bank of Sudan to understand applicable payment service provider licensing requirements, anti-money-laundering (AML) and know-your-customer (KYC) obligations, and data-residency rules for

financial transaction records. The developer onboarding flow should be extended to collect and verify the identity documents required for KYC, and all audit logs and transaction records must be retained in accordance with the applicable statutory minimum retention period.

References

- [1] Apache Software Foundation, "Apache Fineract: An Open Source Core Banking Platform," 2024. [Online]. Available: <https://fineract.apache.org>. [Accessed: May 10, 2026].
- [2] OpenJS Foundation, "Node.js Documentation — v20 LTS," 2024. [Online]. Available: <https://nodejs.org/docs/latest-v20.x/api/>. [Accessed: May 10, 2026].
- [3] OpenJS Foundation, "Express.js — Fast, Unopinionated, Minimalist Web Framework for Node.js (v4/v5)," 2024. [Online]. Available: <https://expressjs.com>. [Accessed: May 10, 2026].
- [4] Sequelize Contributors, "Sequelize — Feature-Rich ORM for Modern TypeScript and JavaScript," 2024. [Online]. Available: <https://sequelize.org/docs/v6/>. [Accessed: May 10, 2026].
- [5] E. You and Vue.js Contributors, "Vue.js 3 Documentation," 2024. [Online]. Available: <https://vuejs.org/guide/introduction>. [Accessed: May 10, 2026].
- [6] Vite Contributors, "Vite — Next Generation Frontend Tooling," 2024. [Online]. Available: <https://vitejs.dev>. [Accessed: May 10, 2026].
- [7] Docker Inc., "Docker Documentation — Docker Compose," 2024. [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: May 10, 2026].
- [8] OWASP Foundation, "OWASP Application Security Verification Standard (ASVS) 4.0," 2021. [Online]. Available: <https://owasp.org/www-project-application-security-verification-standard/>. [Accessed: May 10, 2026].
- [9] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," IETF, RFC 2104, Feb. 1997. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc2104>. [Accessed: May 10, 2026].

- [10] J. Franks, P. Hallam-Baker, J. Hostetler, S. Lawrence, P. Leach, A. Luotonen, and L. Stewart, "HTTP Authentication: Basic and Digest Access Authentication," IETF, RFC 2617, Jun. 1999. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc2617>. [Accessed: May 10, 2026].
- [11] *Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.
- [12] OpenAPI Initiative, "OpenAPI Specification 3.0.3," 2021. [Online]. Available: <https://spec.openapis.org/oas/v3.0.3>. [Accessed: May 10, 2026].
- [13] Stripe, Inc., "Stripe API Reference," 2024. [Online]. Available: <https://stripe.com/docs/api>. [Accessed: May 10, 2026].
- [14] K. E. Wiegers and J. Beatty, *Software Requirements*, 3rd ed. Redmond, WA, USA: Microsoft Press, 2013.
- [15] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018.
- [16] K. C. Laudon and C. G. Traver, *E-Commerce 2021–2022: Business, Technology, Society*, 17th ed. Hoboken, NJ, USA: Pearson, 2021.
- [17] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.
- [18] A. Demirgüç-Kunt, L. Klapper, D. Singer, and S. Ansar, "The Global Findex Database 2021: Financial Inclusion, Digital Payments, and Resilience in the Age of COVID-19," World Bank, Washington, DC, 2022. [Online]. Available: <https://www.worldbank.org/en/publication/globalfindex>. [Accessed: May 10, 2026].
- [19] European Centre for Development Policy Management (ECDPM), "Instant and Inclusive? The State of Instant Payment Systems in Africa," ECDPM, 2024. [Online]. Available: <https://ecdpm.org>. [Accessed: May 10, 2026].

- [20] A. H. Ahmad, C. Green, and F. Jiang, "Mobile Money, Financial Inclusion and Development: A Review With Reference to African Experience," *J. Econ. Surv.*, vol. 34, no. 4, pp. 753–792, 2020, doi: 10.1111/joes.12372.
- [21] Moyasar, "Moyasar Payment API Documentation," 2023. [Online]. Available: <https://moyasar.com/docs/>. [Accessed: May 10, 2026].
- [22] PayPal Developer, "PayPal REST API Documentation," 2023. [Online]. Available: <https://developer.paypal.com/api/rest/>. [Accessed: May 10, 2026].
- [23] Flutterwave, "Flutterwave Developer Documentation," 2023. [Online]. Available: <https://developer.flutterwave.com>. [Accessed: May 10, 2026].
- [24] Safaricom Developer Portal, "Daraja API Documentation," 2023. [Online]. Available: <https://developer.safaricom.co.ke>. [Accessed: May 10, 2026].
- [25] Cashi, "CashiPay — Online Payment Solution," Alsoug, 2024. [Online]. Available: <https://getcashi.com/cashi-pay>. [Accessed: May 10, 2026].
- [26] Business Wire, "alsoug Raises \$5M in First Foreign Tech Investment Into Sudan Since Lifting of International Sanctions," Oct. 2021. [Online]. Available: <https://www.businesswire.com/news/home/20211012005512/en/>. [Accessed: May 10, 2026].
- [27] SyberPay, "SyberPay — Official Website," Syber Group, 2024. [Online]. Available: <https://syberpay.com/>. [Accessed: May 10, 2026].
- [28] Syber Group, "Syber Group — Company Profile," 2024. [Online]. Available: <https://syber-group.com/>. [Accessed: May 10, 2026].
- [29] kataki-kh, "Syber Pay Payment API Integration Package," GitHub, 2023. [Online]. Available: https://github.com/kataki-kh/Syber_pay-package. [Accessed: May 10, 2026].
- [30] A. Copestake, D. Kirti, M. S. Martinez Peria, and Y. Zeng, "Integrating Fragmented Networks: Interoperability in Money and Payments," IMF Working Paper WP/25/126, International Monetary Fund, 2025. [Online]. Available:

<https://www.imf.org/-/media/files/publications/wp/2025/english/wpiea2025126-print-pdf.pdf>

- [31] L. Hornuf, K. Safari, and J. Voshaar, "Mobile Fintech Adoption in Sub-Saharan Africa: A Systematic Literature Review and Meta-Analysis," *Research in International Business and Finance*, vol. 73, p. 102529, 2025.

Appendices

Appendix A: Bank Channel Test Case Results

TC ID	Test Type	Objective	Steps	Expected Result	Actual Result	Status
TC-BC-01	Functional	Valid user login	POST /api/login with username Mohamed, password password	200 OK; JWT token returned; fineract_client_id present in response	200 OK; 8-hour token issued	Pass
TC-BC-02	Negative	Invalid password rejected	POST /api/login with correct username and wrong password	401 Unauthorized; no token issued	401 returned	Pass
TC-BC-03	Functional	JWT token refresh	POST /api/refresh with valid token in Authorization header	New 8-hour JWT token returned	New token issued	Pass
TC-BC-04	Integration	Account dashboard — balance display	GET /api/accounts with valid JWT	Savings account list with account numbers and SDG balances sourced from Fineract	Account list with correct balances returned	Pass
TC-BC-05	Integration	Transaction history	GET /api/accounts/:id/transactions with valid JWT and an account ID	Statement lines (date, amount, type) for the selected account	Transaction history returned	Pass
TC-BC-06	Integration	Recipient account lookup	GET /api/transfer/lookup/:accNo with valid	Account holder name, account ID, and office	Correct account details returned	Pass

			Fineract account number	ID returned from Fineract GET /search		
TC-BC-07	Integration	Peer-to-peer transfer — execution	POST /api/transfer with valid recipient account details and amount 50 SDG	Fineract POST /accounttransfers executed; Fineract resourceId returned; local transactions row inserted	Transfer successful; Fineract balances updated	Pass
TC-BC-08	Integration	Bill payment — invoice lookup	GET /api/billpay/lookup/:invoiceId with a valid pending PhoenixPay invoice ID	Invoice details (amount, merchant name, status) returned from PhoenixPay via channel secret	Invoice details returned correctly	Pass
TC-BC-09	Integration	Bill payment — full execution	POST /api/billpay with a valid pending invoice ID	Fineract transfer payer→PhoenixPay settlement account executed; PhoenixPay confirmation POSTed (up to 3 retries); invoice marked paid; local transactions row inserted	Payment confirmed; invoice status paid	Pass
TC-BC-10	Negative	Bill payment on already-paid invoice	POST /api/billpay with an invoice already in paid status	PhoenixPay returns error on lookup; no Fineract transfer initiated	Error returned; no duplicate transfer or credit	Pass

TC-BC-11	Security	Unauthenticated access to protected endpoint	GET /api/accounts without Authorization header	401 Unauthorized	401 returned	Pass
TC-BC-12	Security	Invalid channel secret on PhoenixPay-facing endpoint	POST /api/phoenix/payout with wrong X-Channel-Secret value	401 Unauthorized	401 returned	Pass
TC-BC-13	Integration	PhoenixPay payout proxy — execution	POST /api/phoenix/payout with valid channel secret, merchant Fineract client ID and account ID, amount 20 SDG	Fineract POST /accounttransfers from settlement account to merchant account executed; fineract_transfer_id returned to PhoenixPay	Transfer executed; transfer ID returned	Pass

Appendix B: PhoenixPay Test Case Results

TC ID	Test Type	Objective	Steps	Expected Result	Actual Result	Status
TC-PP-01	Functional	Developer registration	POST /api/auth/register with unique email and password of 8 or more characters	201 Created; JWT returned; developer role assigned; balance row seeded	Account created; token issued	Pass
TC-PP-02	Negative	Duplicate email rejected	POST /api/auth/register with an email address already in use	400 Bad Request; descriptive error message	Duplicate rejected	Pass
TC-PP-03	Negative	Weak password rejected	POST /api/auth/register with a password of fewer than 8 characters	400 Bad Request; validation error listing the password rule	Validation failed	Pass
TC-PP-04	Functional	Developer login	POST /api/auth/login with valid credentials	200 OK; JWT token returned	JWT issued	Pass
TC-PP-05	Functional	Token refresh	POST /api/auth/refresh with valid JWT	New JWT issued before expiry	New token returned	Pass
TC-PP-06	Functional	Application creation	POST /api/applications with JWT; application name provided	201 Created; application persisted with status inactive	Application created	Pass
TC-PP-07	Functional	Application activation	PUT /api/applications/:id with status set to active	Application status updated to active; audit log entry created	Status updated	Pass

TC-PP-08	Functional	API key generation	POST /api/applications/:id/key/reset with JWT	Full API key returned once; bcrypt hash stored; 16-character prefix saved for subsequent lookups	Key generated and returned	Pass
TC-PP-09	Functional	Invoice creation via API key	POST /api/invoices with valid API key, amount 9.99, currency SDG	201 Created; invoice ID returned; status pending	Invoice created	Pass
TC-PP-10	Negative	Invoice with unsupported currency	POST /api/invoices with currency USD	400 Bad Request; error message stating currency must be SDG	Validation error returned	Pass
TC-PP-11	Functional	Invoice list with status filter	GET /api/invoices?status=pending with API key	Only pending invoices returned; pagination applied	Filtered list returned	Pass
TC-PP-12	Functional	Invoice cancellation	POST /api/invoices/:id/cancel with API key	Invoice status set to cancelled	Cancelled successfully	Pass
TC-PP-13	Integration	Payment confirmation — first call	POST /api/channel/invoices/:id/confirm with valid channel secret, bank_reference, and amount	Invoice marked paid; available_balance incremented; webhook delivery queued	Invoice paid; balance credited atomically	Pass
TC-PP-14	Integration	Idempotent payment confirmation	POST /api/channel/invoices/:id/confirm a second time with	Original success response returned; balance not incremented	Idempotency enforced; no double credit	Pass

			identical bank_refe rence	a second time		
TC-PP-15	Integration	Webhook delivery on payment	Inspect webhook_d eliveries table after TC-PP-13	HMAC-SH A256 signed invoice.p aid POST delivered to configured webhook URL; delivery attempt logged with HTTP response status	Delivery logged; merchant endpoint received event	Pass
TC-PP-16	Functional	Balance query	GET /api/bala nce with JWT	available _balance returned; payout eligibility flag reflects current threshold	Balances returned correctly	Pass
TC-PP-17	Functional	Payout account registration	POST /api/payo ut-accoun ts with JWT, Fineract client ID, account ID, and a label	Account record persisted; fineract_ client_id and fineract_ account_i d stored as provided; no real-time Fineract validation at registration time — validation occurs implicitly when a payout is executed	Account registered and linked to user	Pass
TC-PP-18	Integration	Payout request	POST /api/payo uts with amount within available balance	Payout record created; Bank Channel /api/phoe nix/payou t called;	Funds disbursed; payout record updated	Pass

				Fineract transfer executed; available_balance decremented; Fineract transfer ID stored on payout record		
TC-PP-19	Integration	Refund initiation	POST /api/refunds with API key, invoice ID, refund amount, and reason	Refund record created; Bank Channel /api/phoenix/refund called; Fineract transfer executed; invoice status updated to refunded; available_balance decremented	Refund completed; invoice status updated	Pass
TC-PP-20	Security	Rate limit on login endpoint	POST /api/auth/login more than 10 times within 15 minutes from the same IP	429 Too Many Requests returned after the 10th attempt within the window	Rate limit triggered	Pass
TC-PP-21	Security	Invalid API key	POST /api/invoices with an incorrect or revoked API key	401 Unauthorized	Request rejected	Pass
TC-PP-22	Security	Non-admin user accessing admin endpoint	GET /api/admin/audit-logs with a developer-role JWT (no admin role in user_roles)	403 Forbidden	Access denied	Pass

TC-PP-23	Security	Missing channel secret header	POST /api/channel/invoices/:id/confirm without X-Channel-Secret header	401 Unauthorized	Request rejected	Pass
TC-PP-24	Functional	Admin configuration update	PUT /api/admin/config with admin JWT; update payout_min_threshold value	Config updated; new value returned on subsequent GET /api/admin/config	Config updated and persisted	Pass
TC-PP-25	Functional	Audit log query with action filter	GET /api/admin/audit-logs?action=invoice.create with admin JWT	Paginated log entries matching the action filter returned	Filtered log returned	Pass
TC-PP-26	Negative	Payment confirmation for expired invoice	POST /api/channel/invoices/:id/confirm for an invoice whose due_date has passed	Payment rejected with an error response; invoice status remains expired	Expired invoice rejected	Pass

Appendix C: TaskMaster Test Case Results

TC ID	Test Type	Objective	Steps	Expected Result	Actual Result	Status
TC-TM-01	Functional	User registration	POST /api/auth/register with name, email, and password of 8 or more characters	201 Created; user record created in MariaDB	Account created	Pass
TC-TM-02	Negative	Weak password rejected	POST /api/auth/register with a password of 5 characters	400 Bad Request; validation error	Validation failed	Pass
TC-TM-03	Functional	User login	POST /api/auth/login with valid credentials	200 OK; 24-hour JWT returned	JWT issued	Pass
TC-TM-04	Functional	Create todo item	POST /api/todos with task text "Write the report"	Todo created; ID returned; completed = false	Todo saved in DB	Pass
TC-TM-05	Functional	Toggle todo completion	PUT /api/todos/:id with completed: true	Todo marked completed; updated state returned	Status updated	Pass
TC-TM-06	Functional	Delete todo item	DELETE /api/todos/:id	Todo removed from the list	Todo deleted	Pass
TC-TM-07	Integration	Premium subscription — invoice creation	POST /api/premium/subscribe	PhoenixPay POST /api/invoices called with API key; 9.99 SDG invoice created; invoice ID stored in users.phoenix_invoice_id; invoice	Invoice created; ID persisted locally	Pass

				details returned to SPA		
TC-TM-08	Integration	Premium status polling — before payment	GET /api/premium/status immediately after TC-TM-07	PhoenixPay invoice status is pending; is_premium is false	Status pending returned	Pass
TC-TM-09	Integration	Premium status polling — after bank channel confirms payment	GET /api/premium/status after the bank channel has confirmed the invoice payment to PhoenixPay	PhoenixPay returns invoice status paid; users.is_premium set to TRUE in local DB	Premium activated via polling	Pass
TC-TM-10	Integration	Webhook receipt — valid HMAC signature	POST /api/webhooks/phoenix with correctly HMAC-SHA256 signed invoice.paid payload using the configured PHOENIX_WEBHOOK_SECRET	Signature verified using crypto.timingSafeEqual(); users.is_premium set to TRUE; 200 OK returned to PhoenixPay	Premium unlocked via webhook	Pass
TC-TM-11	Security	Webhook receipt — invalid HMAC signature	POST /api/webhooks/phoenix with a tampered payload or wrong X-PhoenixPay-Signature header	401 Unauthorized; users.is_premium unchanged	Tampered webhook rejected	Pass
TC-TM-12	Security	Webhook receipt — missing signature header	POST /api/webhooks/phoenix without the X-PhoenixPay-Signature header	401 Unauthorized	Request rejected	Pass

			ture header			
TC-TM-13	Security	Unauthenticated access to protected endpoint	GET /api/todos without Authorization header	401 Unauthorized	Request rejected	Pass
TC-TM-14	Security	Cross-user todo isolation	Create todos as User A; then GET /api/todos authenticated as User B	Only User B's todos returned; User A's todos are not exposed	Per-user isolation enforced	Pass
TC-TM-15	Integration	Premium cancellation and partial refund	POST /api/premium/cancel for a user with active premium	PhoenixPay POST /api/refunds called with 50% of 9.99 SDG (= 4.995 SDG); users.is_premium set to FALSE; PhoenixPay invoice status updated to partially_refunded	Refund initiated; premium disabled	Pass